



Autonomous design of structured representations

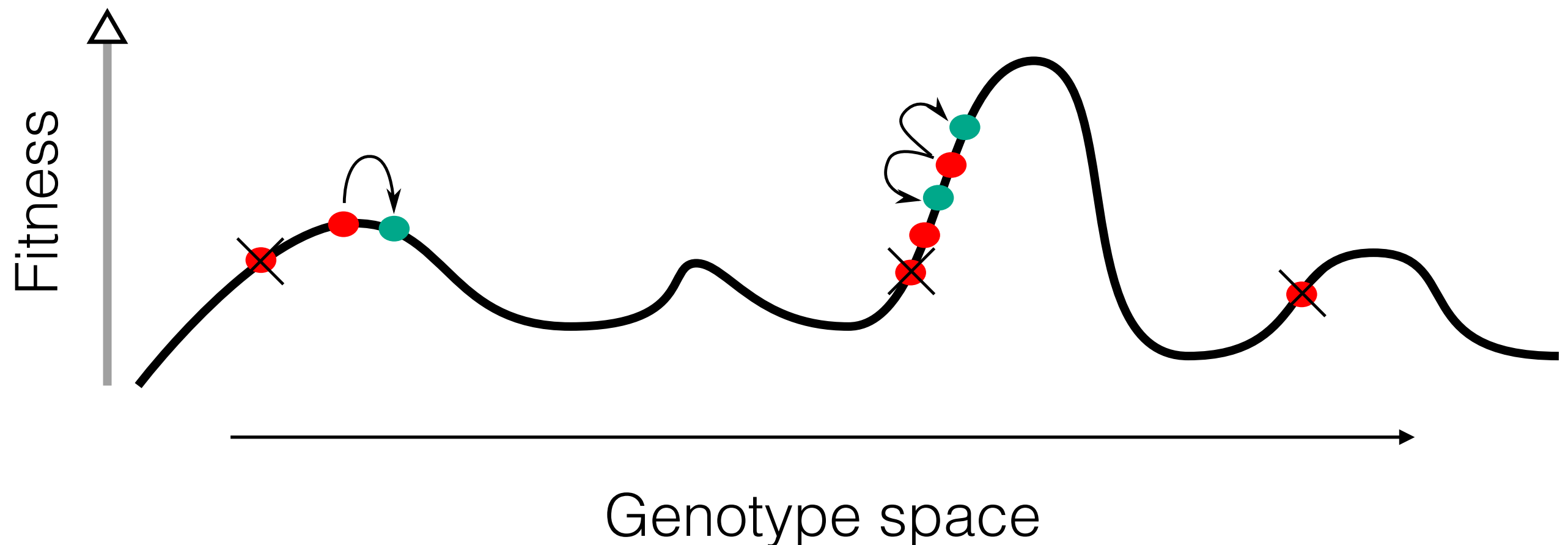
Stéphane Doncieux
Professor



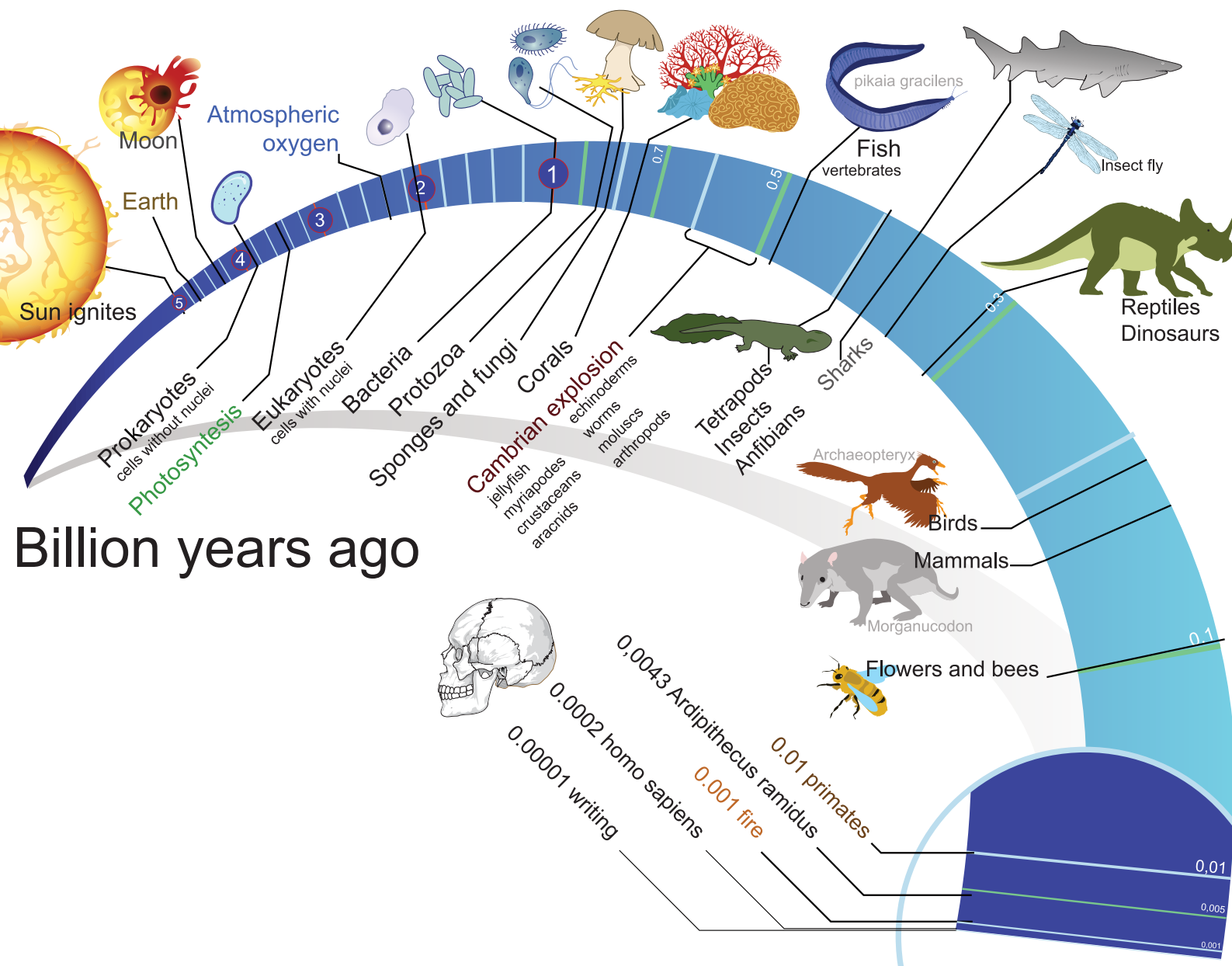
Introduction

- Evolution as a numerical optimisation process:

Find $\mathbf{x} \in \mathbb{R}^n$ maximizing/minimizing $f(\mathbf{x})$



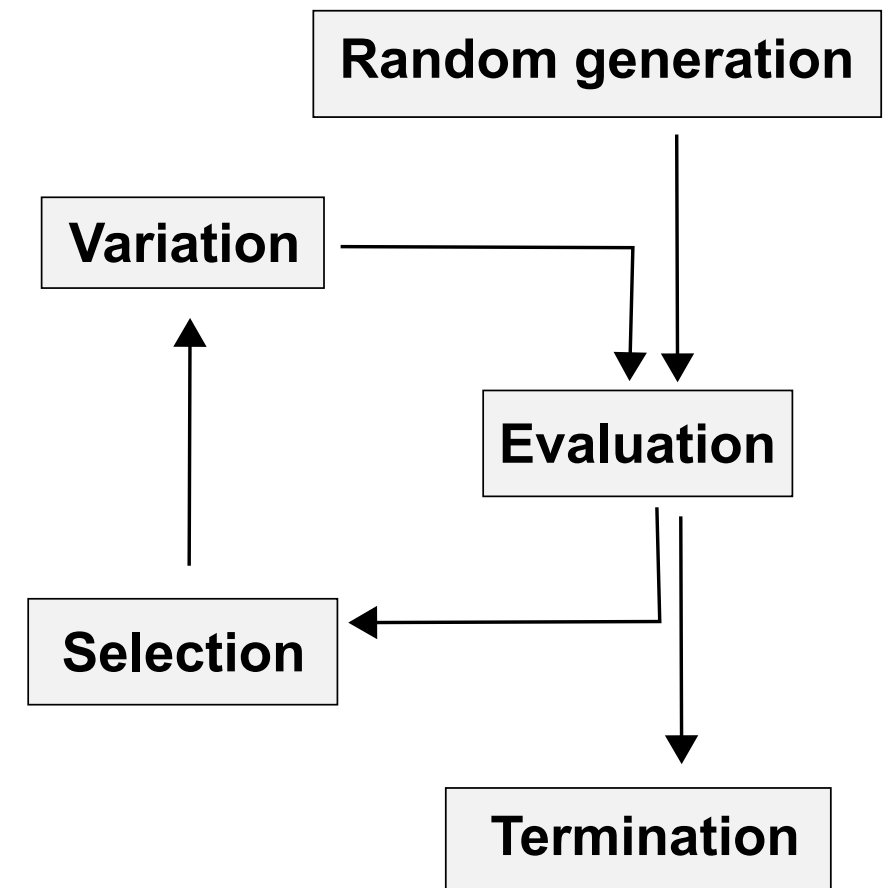
Introduction



Evolution can do more than parameter optimization...

Introduction

- From parameter to structure optimization:
 - random generation operator
 - variation operators
 - generic selection process (or specific one)

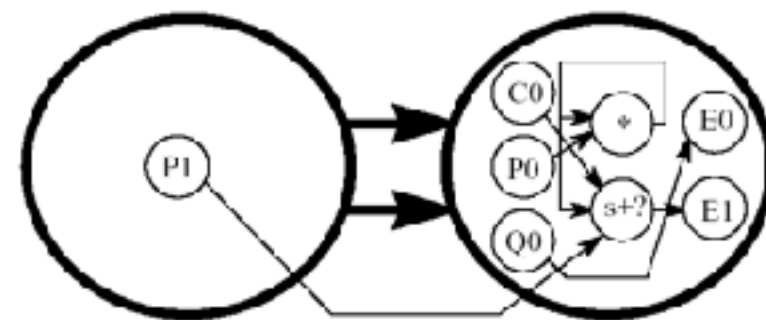
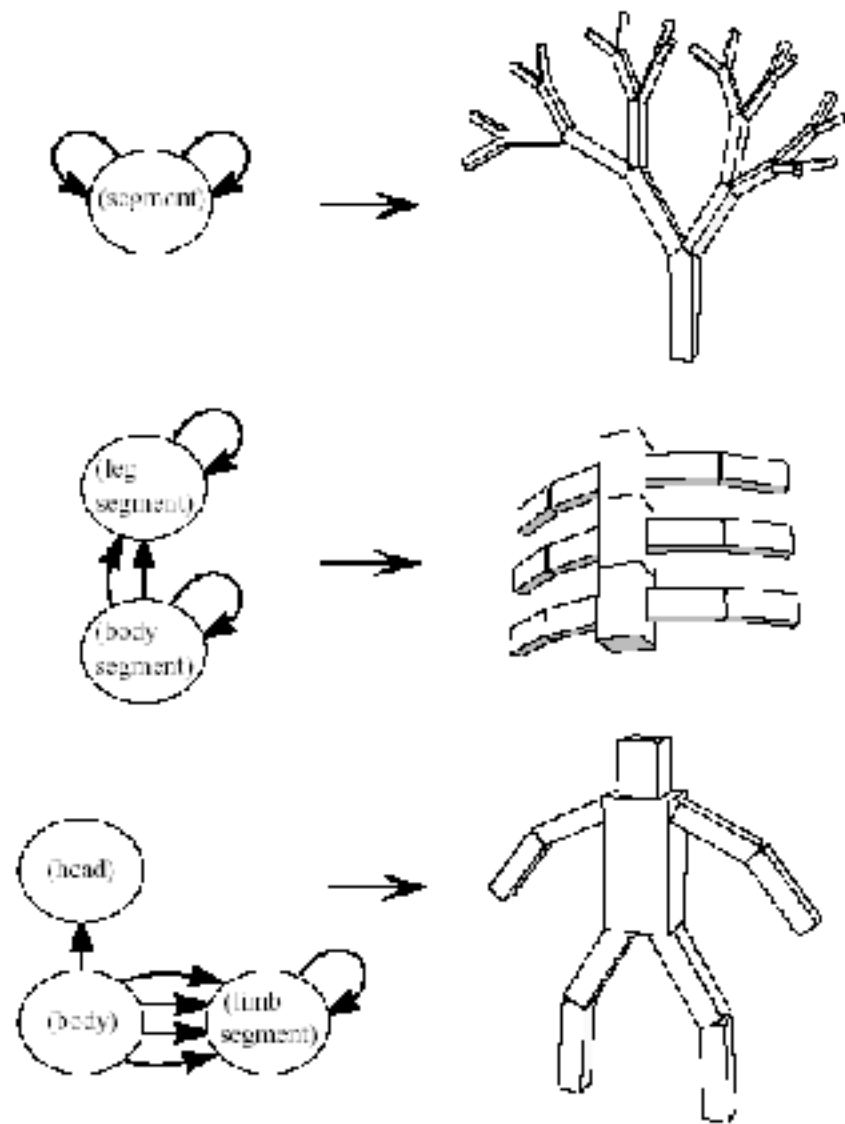


Overview

- Evolving Morphologies
- Genetic Programming
- Neuroevolution
- Evolutionary algorithms & selective pressures
- Neuroevolution/GP in Deep learning era
- Building you own representation



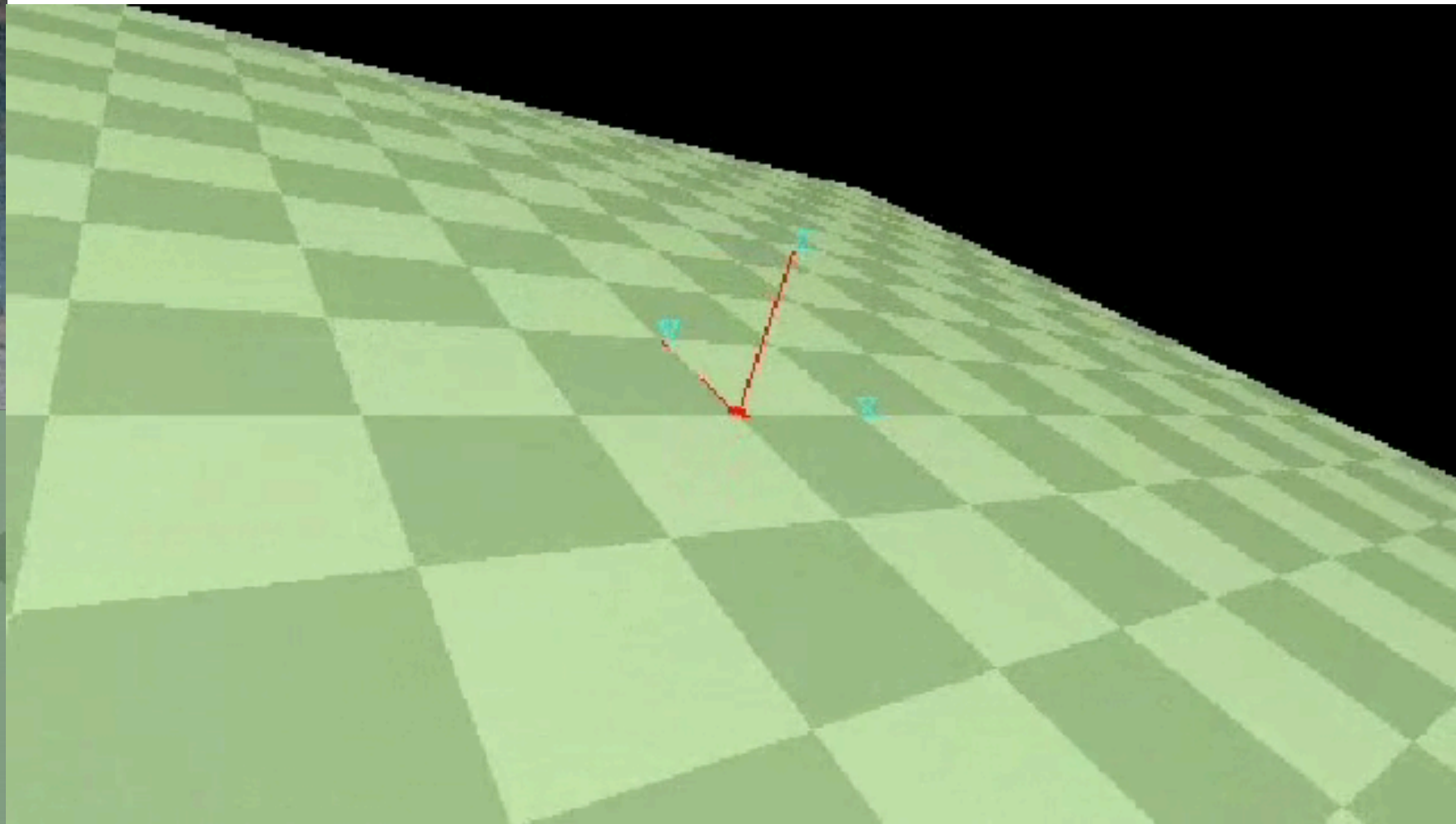
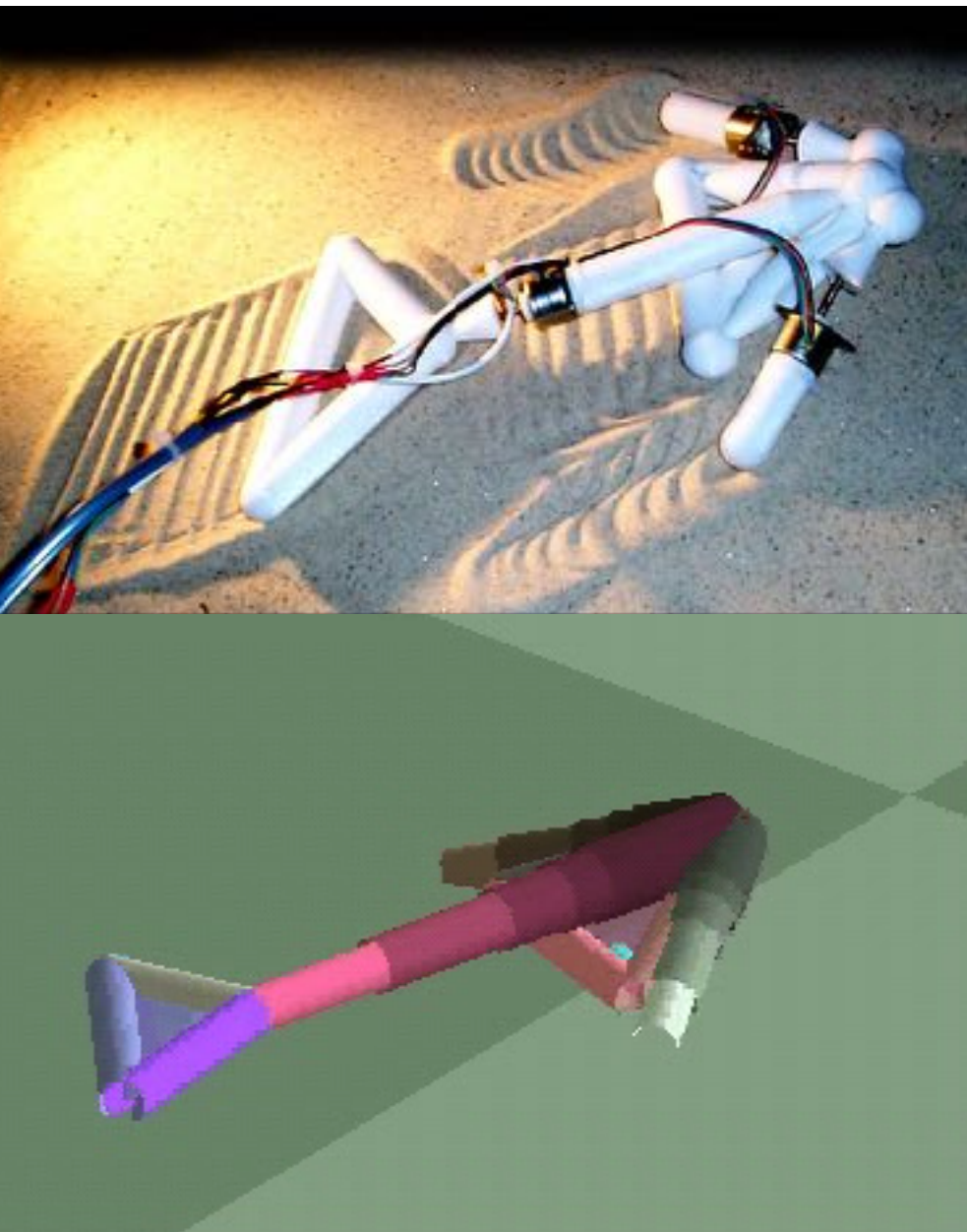
Evolution of morphologies



"Evolving Virtual Creatures" K.Sims, Computer Graphics (Siggraph '94 Proceedings), July 1994, pp.15-22.

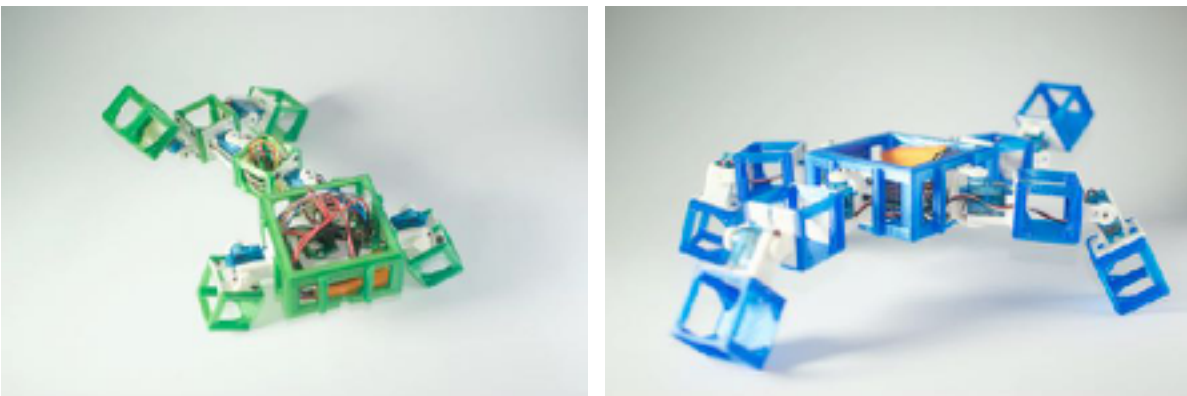
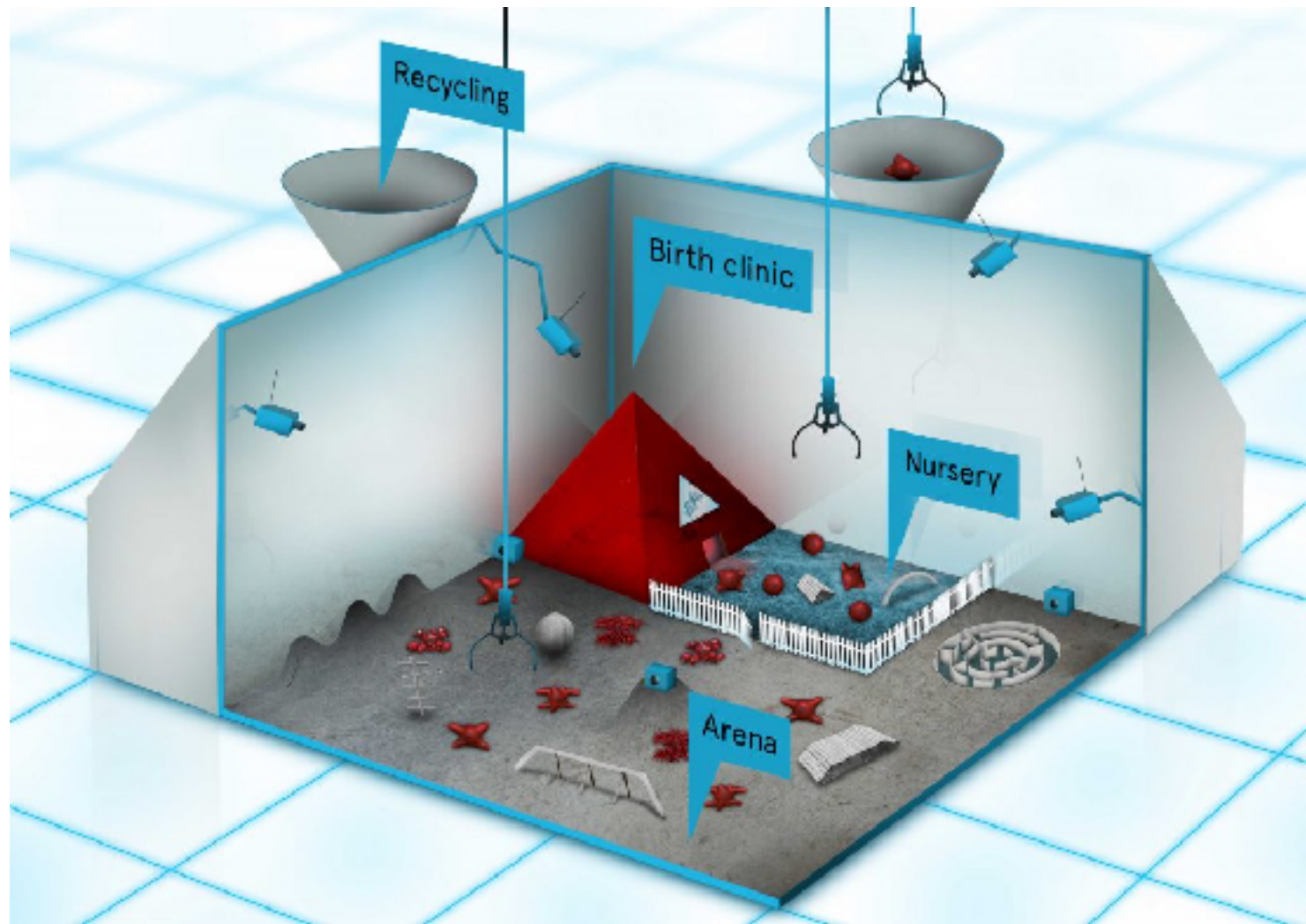
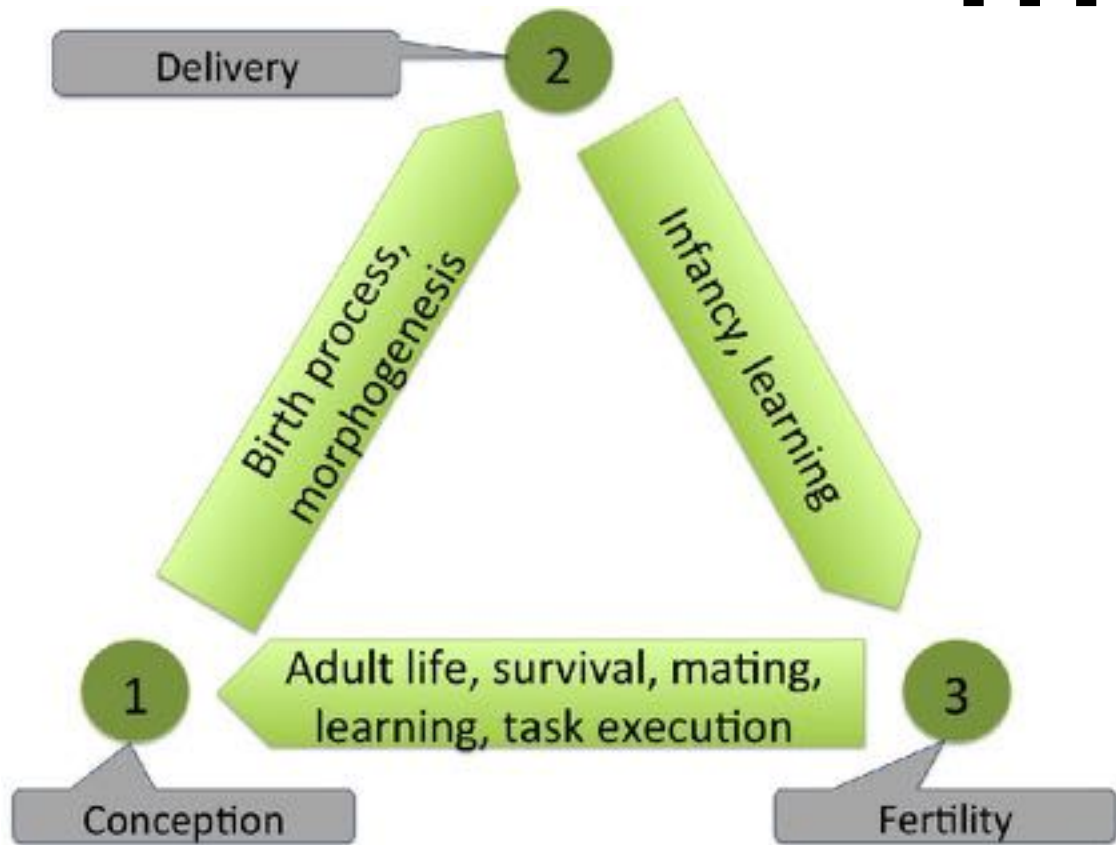
"Evolving 3D Morphology and Behavior by Competition" K.Sims, Artificial Life IV Proceedings, ed.by Brooks & Maes, MIT Press, 1994, pp.28-39.

Evolution of morphologies



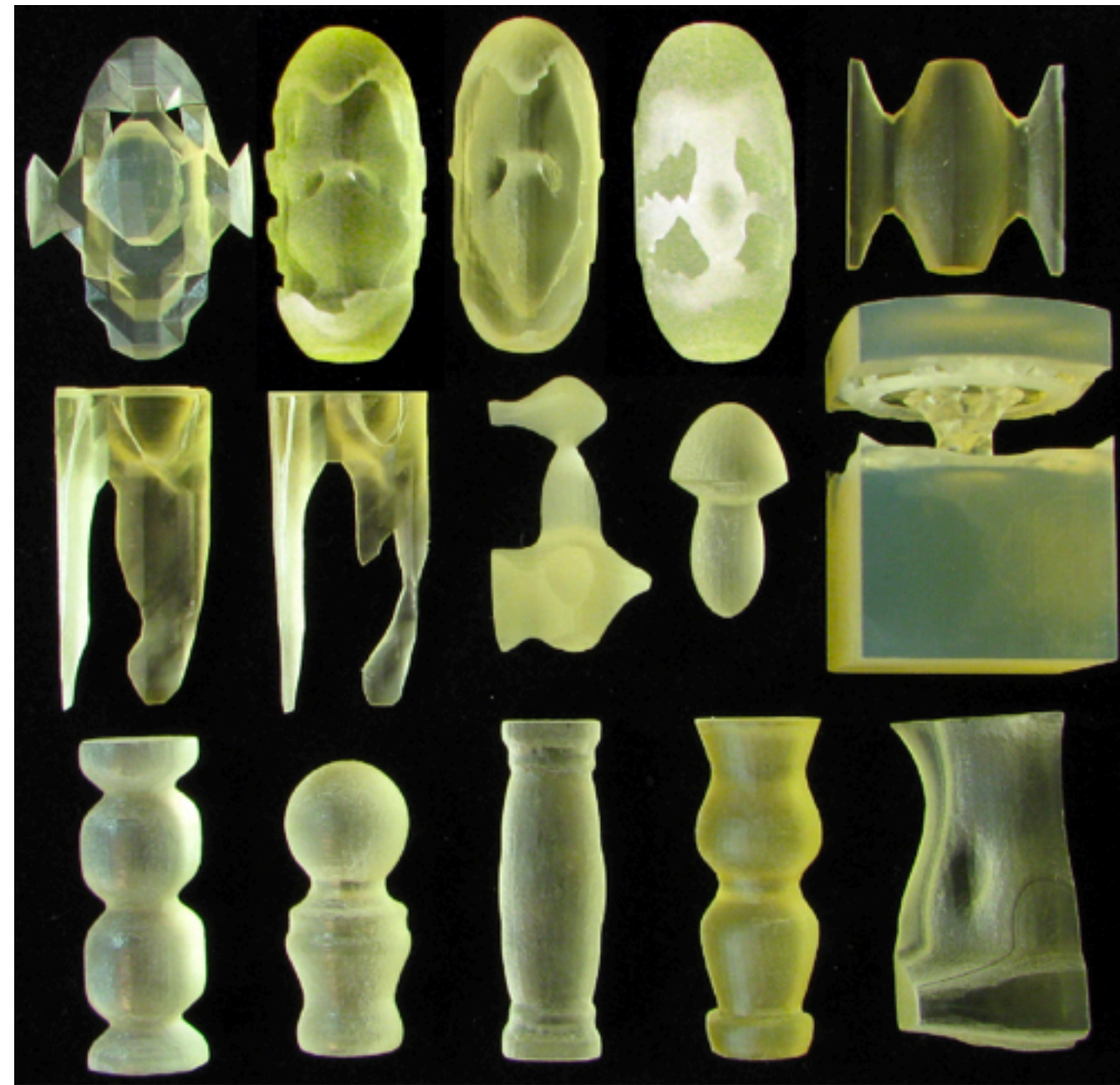
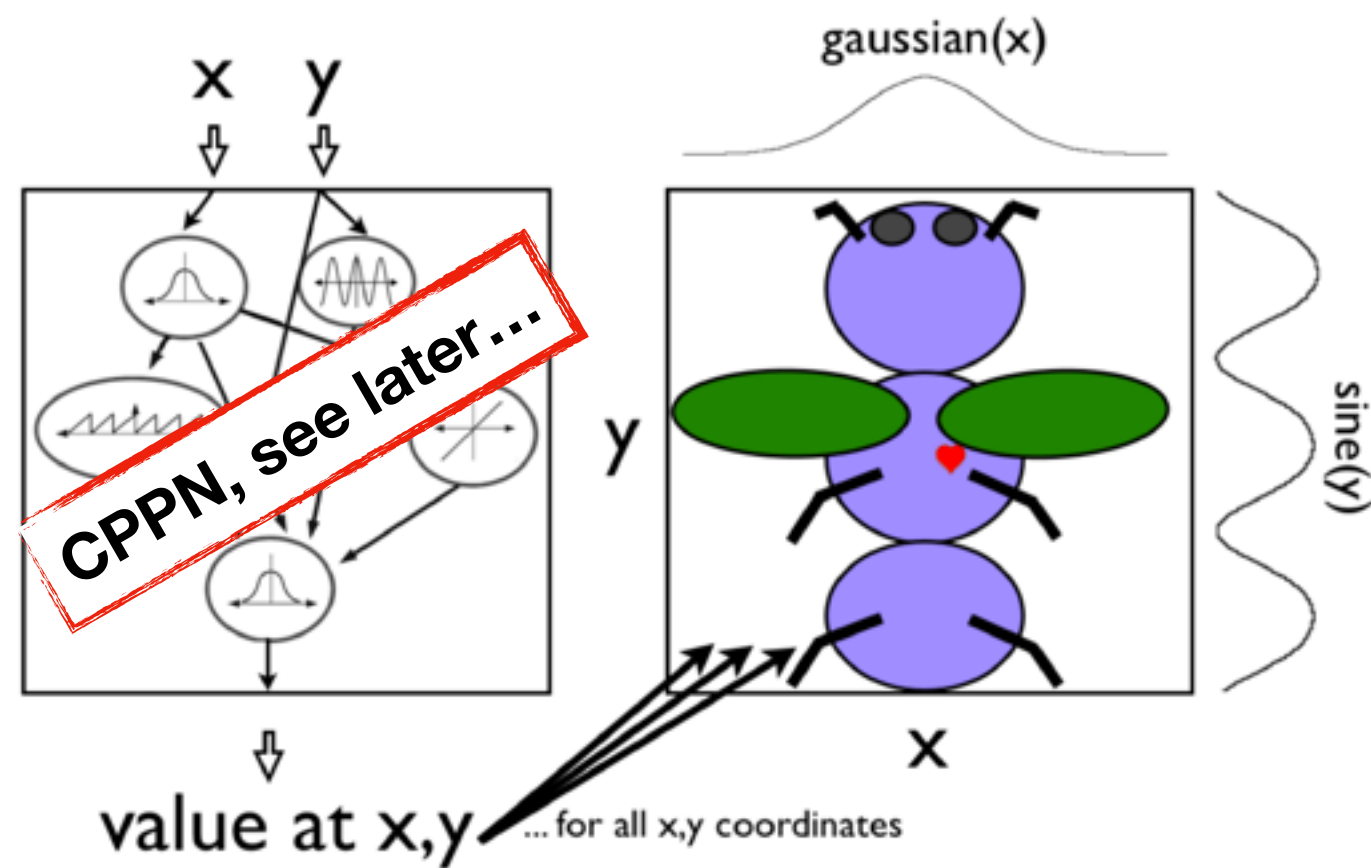
Projet GOLEM

Evolution of morphologies



A.E. Eiben, EvoSphere: the World of Robot Evolution, A.-H. Dediu et al. (Eds.), Proceedings of the Theory and Practice of Natural Computing 2015 (TPNC 2015), LNCS 9477, Springer, 2015, pp. 1-17.

Evolution of 3D structures



Clune, J., & Lipson, H. (2011). Evolving 3d objects with a generative encoding inspired by developmental biology. *ACM SIGEVOlution*, 5(4), 2-12.

Auerbach, J. E., & Bongard, J. C. (2010, July). Evolving CPPNs to grow three-dimensional physical structures. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation* (pp. 627-634).

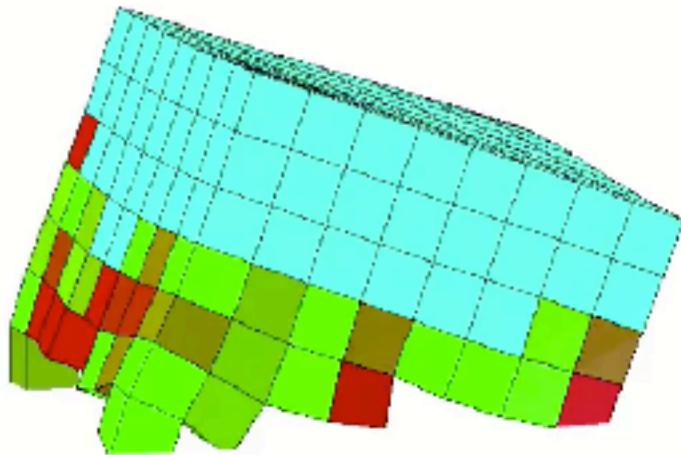
Evolution of morphologies

Cheney, N., MacCurdy, R., Clune, J., & Lipson, H. (2014). Unshackling evolution: evolving soft robots with multiple materials and a powerful generative encoding. *ACM SIGEVOlution*, 7(1), 11-23.

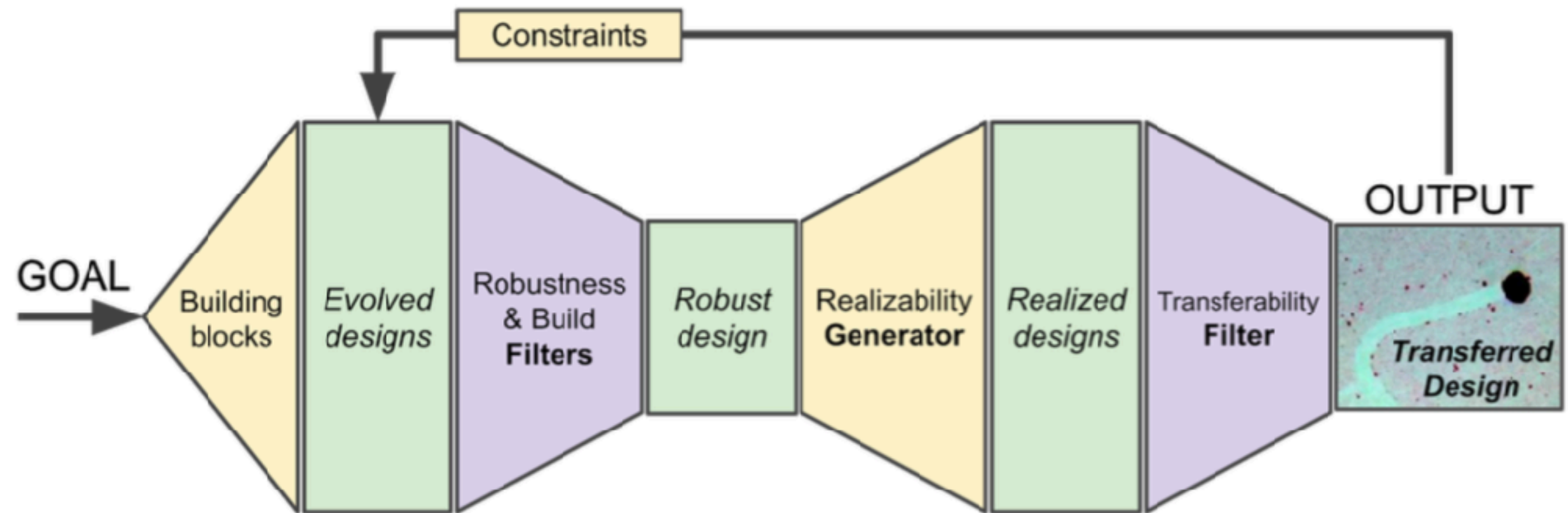
Evolution of morphologies

A scalable pipeline for
designing reconfigurable organisms.

Sam Kriegman, Douglas Blackiston, Michael Levin, Josh Bongard
University of Vermont, Tufts University.



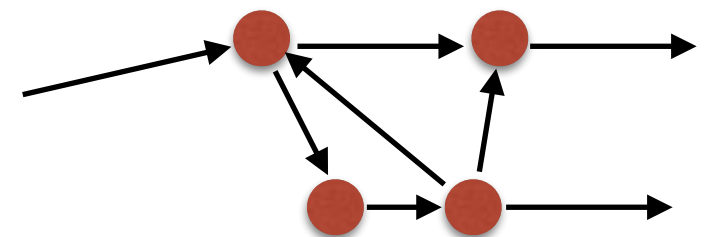
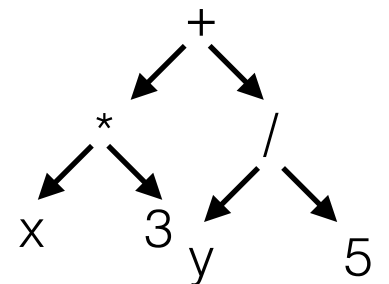
Evolution of morphologies



- Key challenges:
 - Representing a 3D structure: **CPPN (see later...)**
 - Evolution of the representation: **Age-Fitness Pareto Optimisation (see later...)**
 - Transfer from simulation to reality **(see later...)**

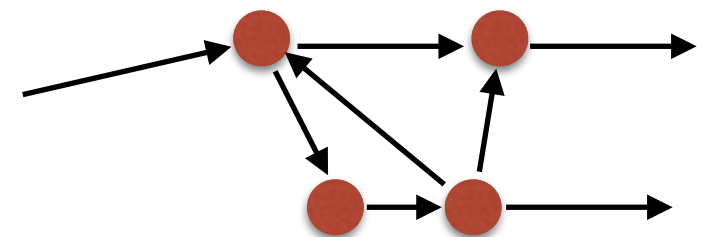
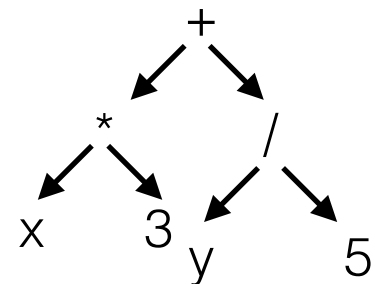
Autonomous design of structured representations

- Focus on two kinds of structures:
 - Programs (trees, linear)
 - Neural networks (oriented graphs)



Autonomous design of structured representations

- What for ?
 - Programs: **symbolic regression (...)**
 - Neural networks: **direct policy search**



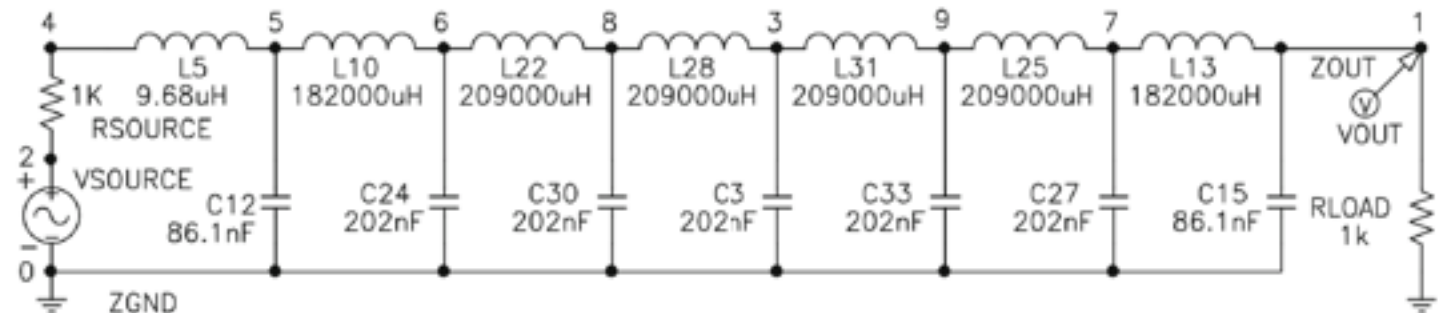
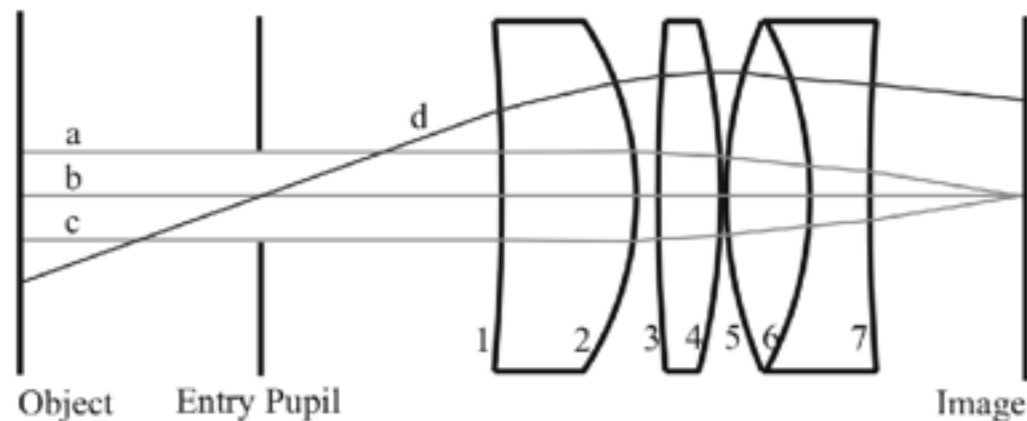
Overview



- Evolving Morphologies
- Genetic Programming
- Neuroevolution
- Evolutionary algorithms & selective pressures
- Neuroevolution/GP in Deep learning era
- Building you own representation

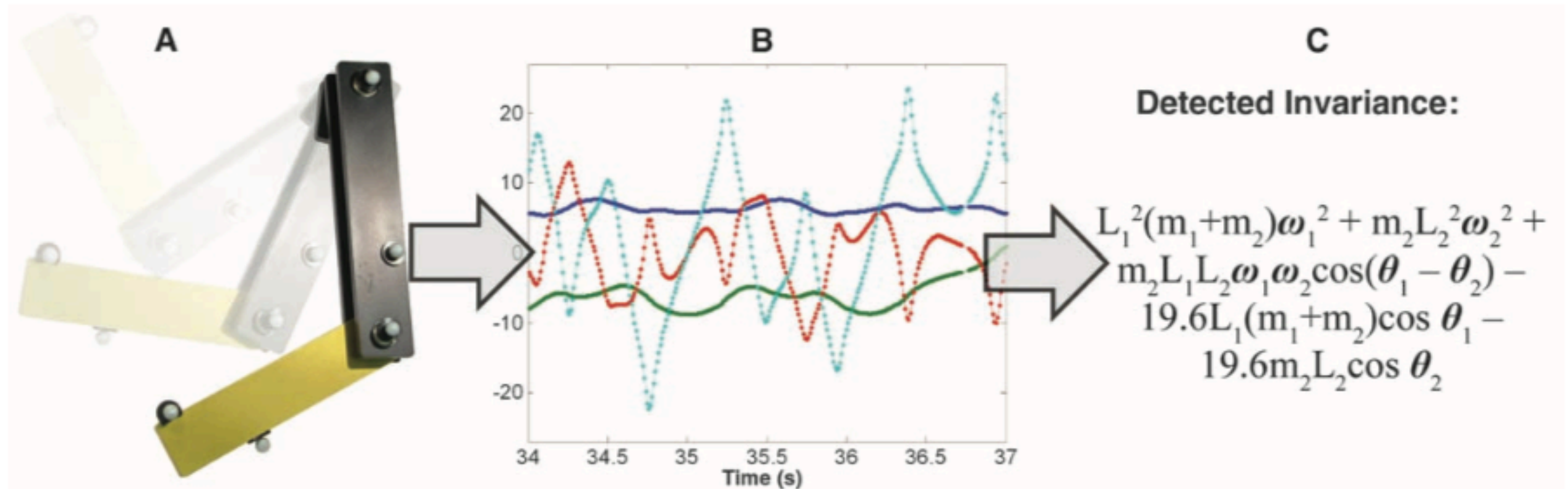
Genetic programming

- Developed since the 1990's by John R. Koza
- Evolution of *programs*

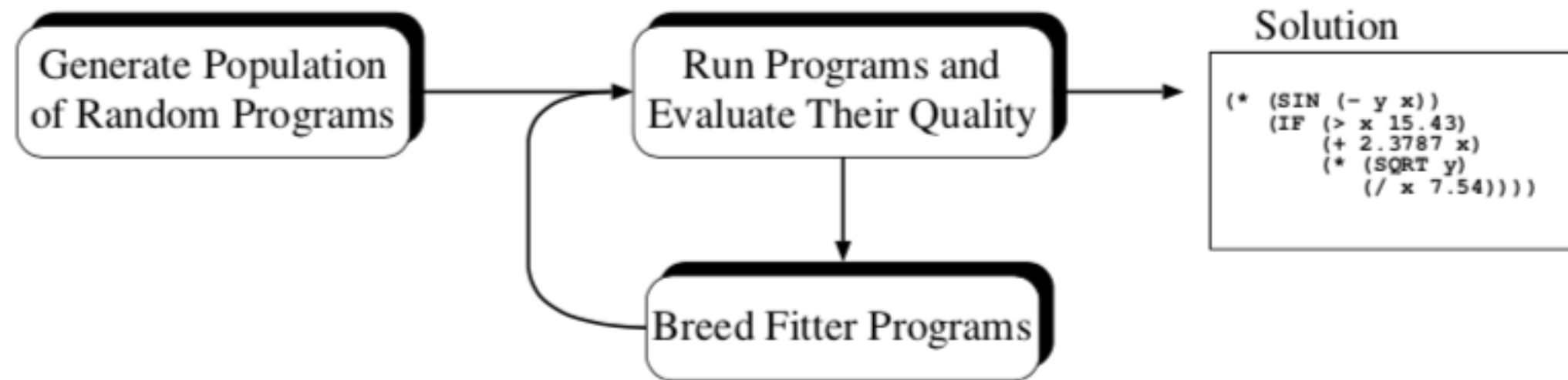


Koza, J. R. (2010). Human-competitive results produced by genetic programming. *Genetic programming and evolvable machines*, 11(3-4), 251-284.

Symbolic regression



Genetic programming



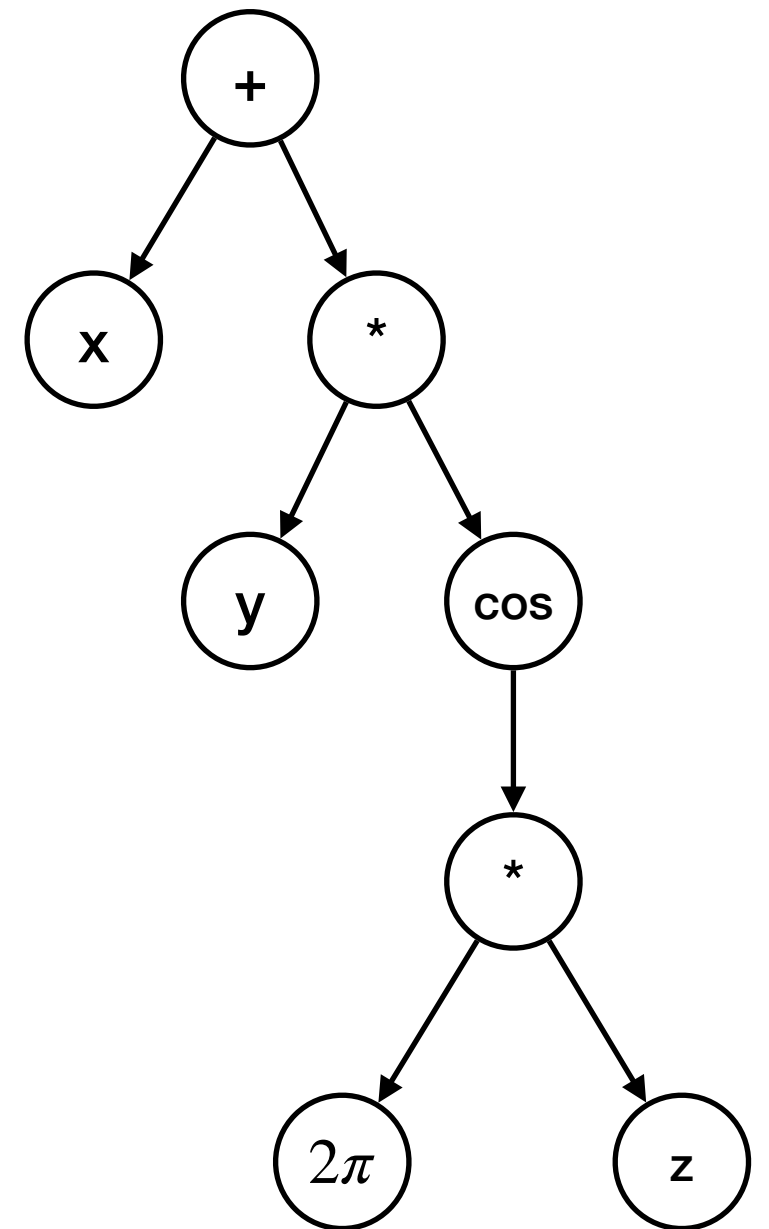
- Principle:
 - The genotype represents a program or an equation
 - Random generation
 - Generation of new programs through exploration operators:
 - Mutation
 - Crossover
 - Selection of the parents of the next generation

Representations

- Abstract syntax tree

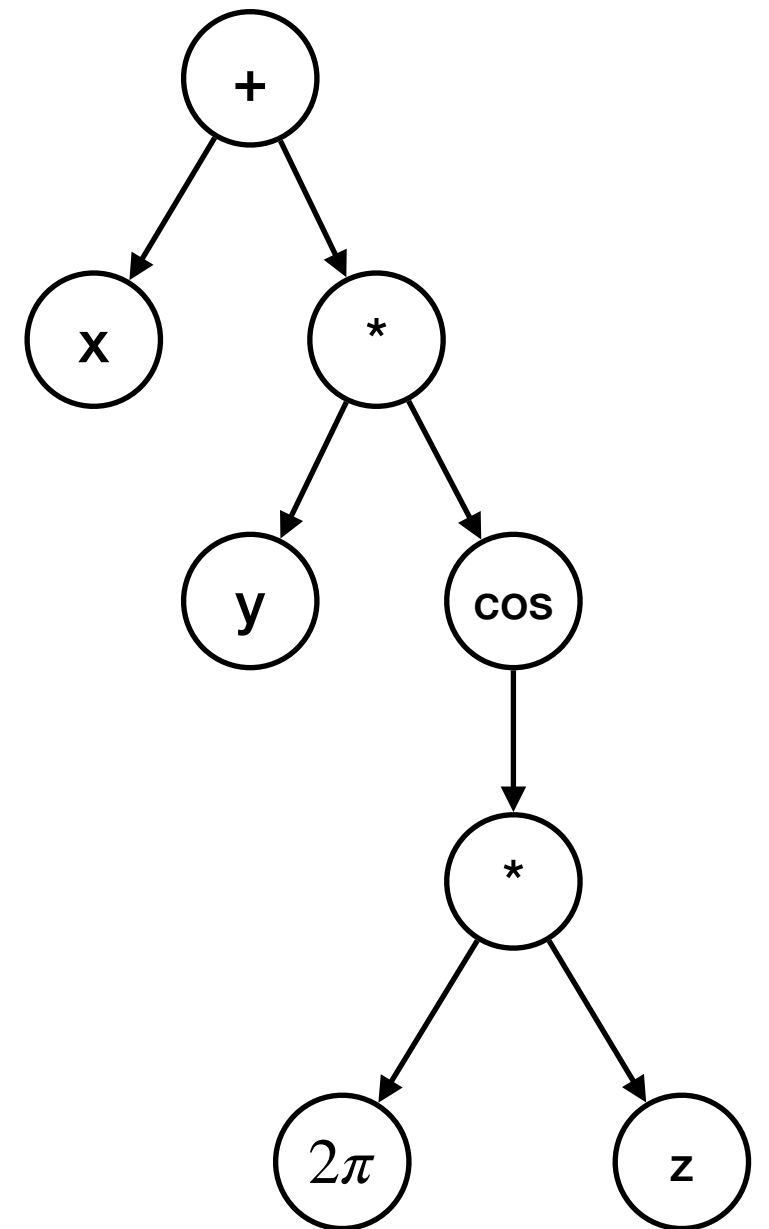
- Example:

$$x + y * \cos(2\pi z)$$



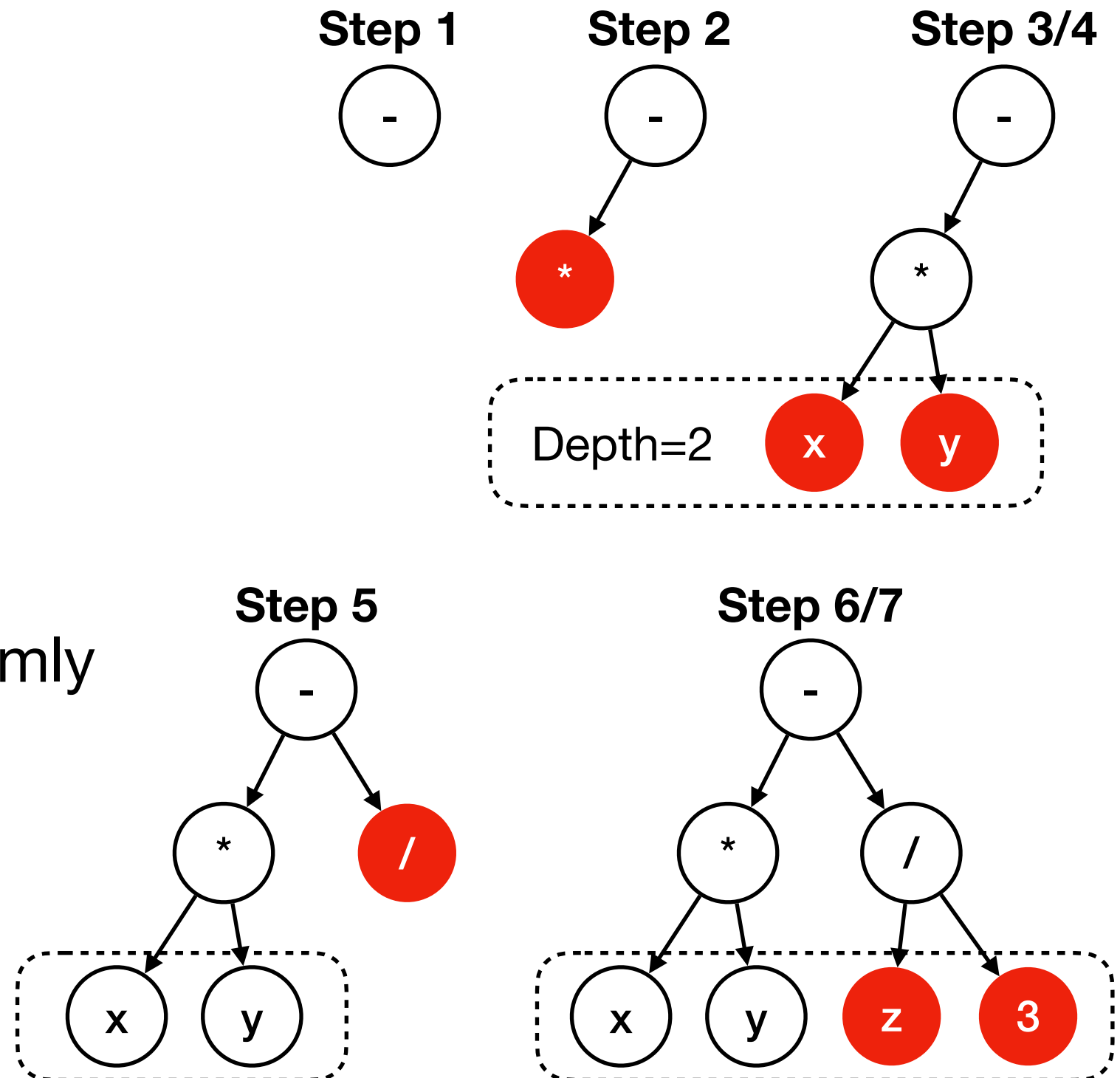
Representations

- Example: $x + y * \cos(2\pi z)$
- **Terminals:** input variables and constants
 - In the example: $x, y, 2\pi, z$
- **Primitive set:** set of available functions
 - In the example: $+, *, \cos$



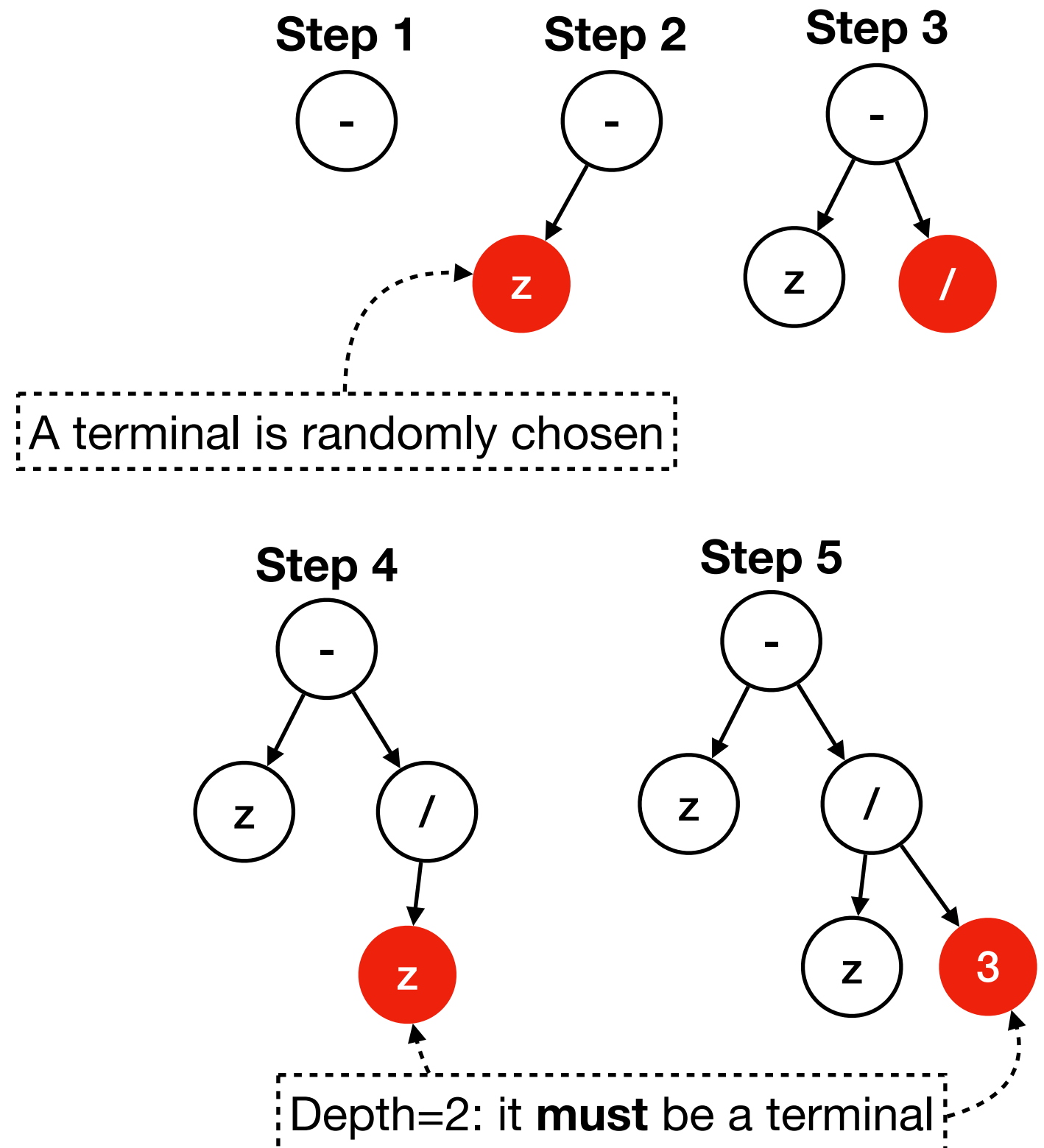
Random initialisation

- Parameter: maximum depth $\mathcal{D}$
- Different methods:
 - **Full:** functions are added until $\mathcal{D}$ is reached, then randomly select terminals
- Example: $\mathcal{D} = 2$



Random initialisation

- Parameter: maximum depth $\mathcal{D}$
- Different methods:
 - **Grow**: functions/terminals are added until $\mathcal{D}$ is reached, then finish with terminals only
 - Example: $\mathcal{D} = 2$

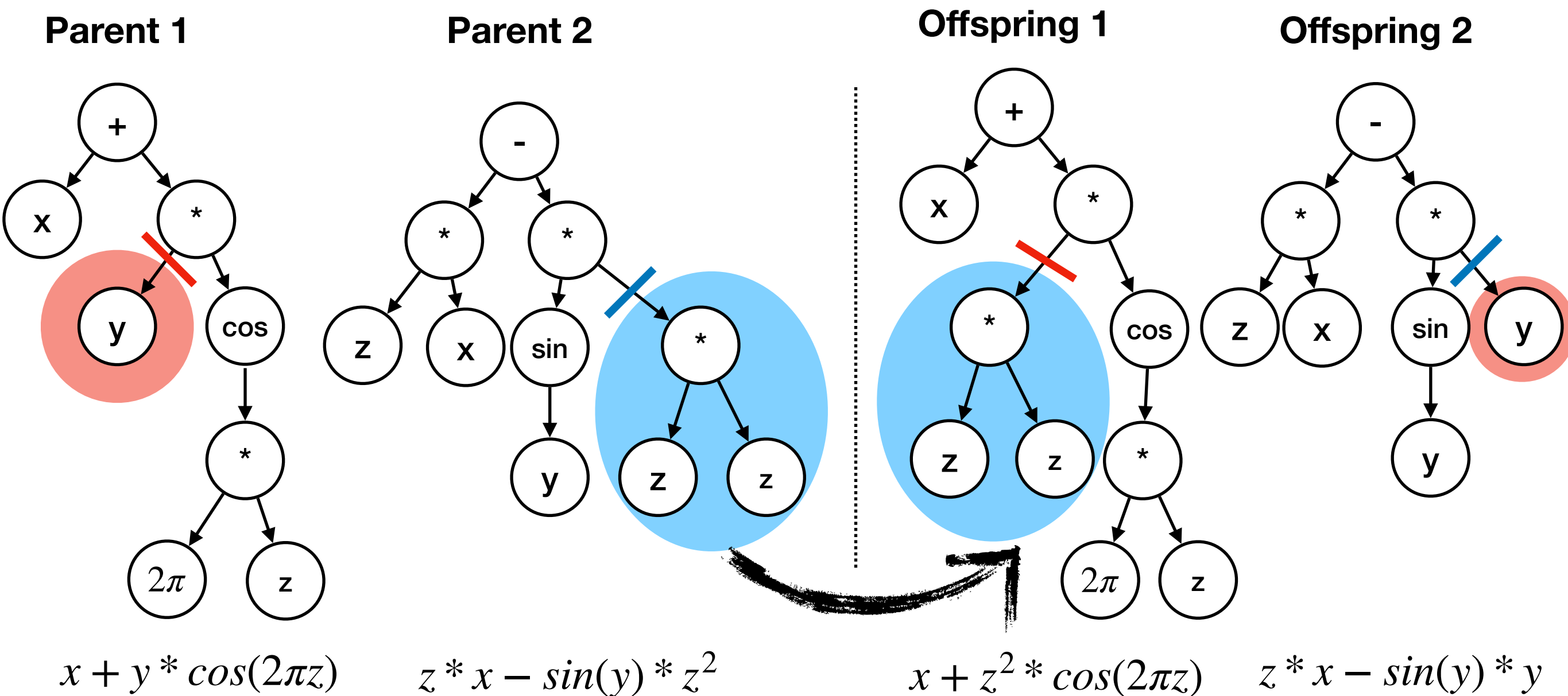


Random initialisation

- Parameter: maximum depth $\mathcal{D}$
- Different methods:
 - **Full:** functions are added until $\mathcal{D}$ is reached, then randomly select terminals
 - **Grow:** functions/terminals are added until $\mathcal{D}$ is reached, then finish with terminals only
 - **Ramped half-and-half:** half the population with full, half with grow.

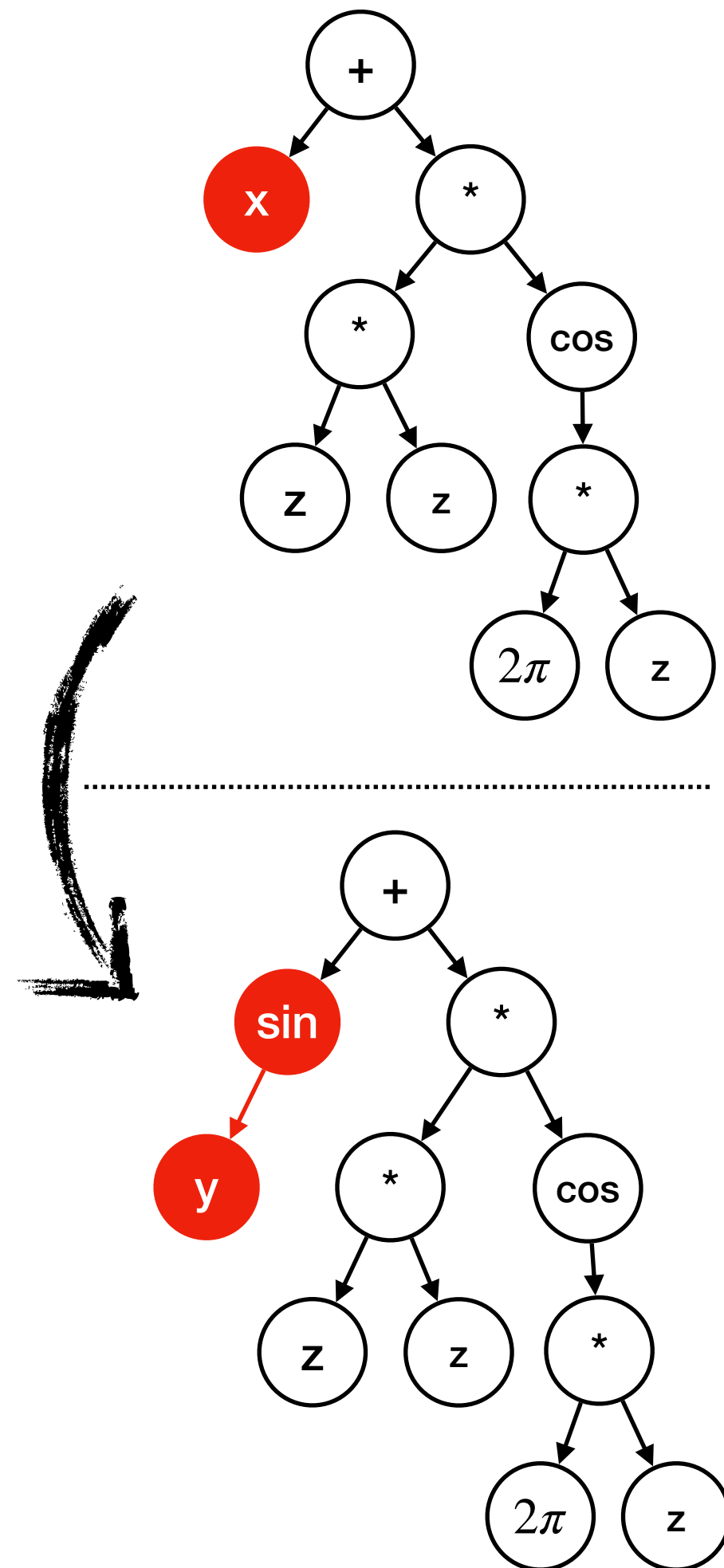
Crossover

- Most important operator in GP, typical crossover rate: 90%
- **Subtree crossover:** single point crossover



Mutation

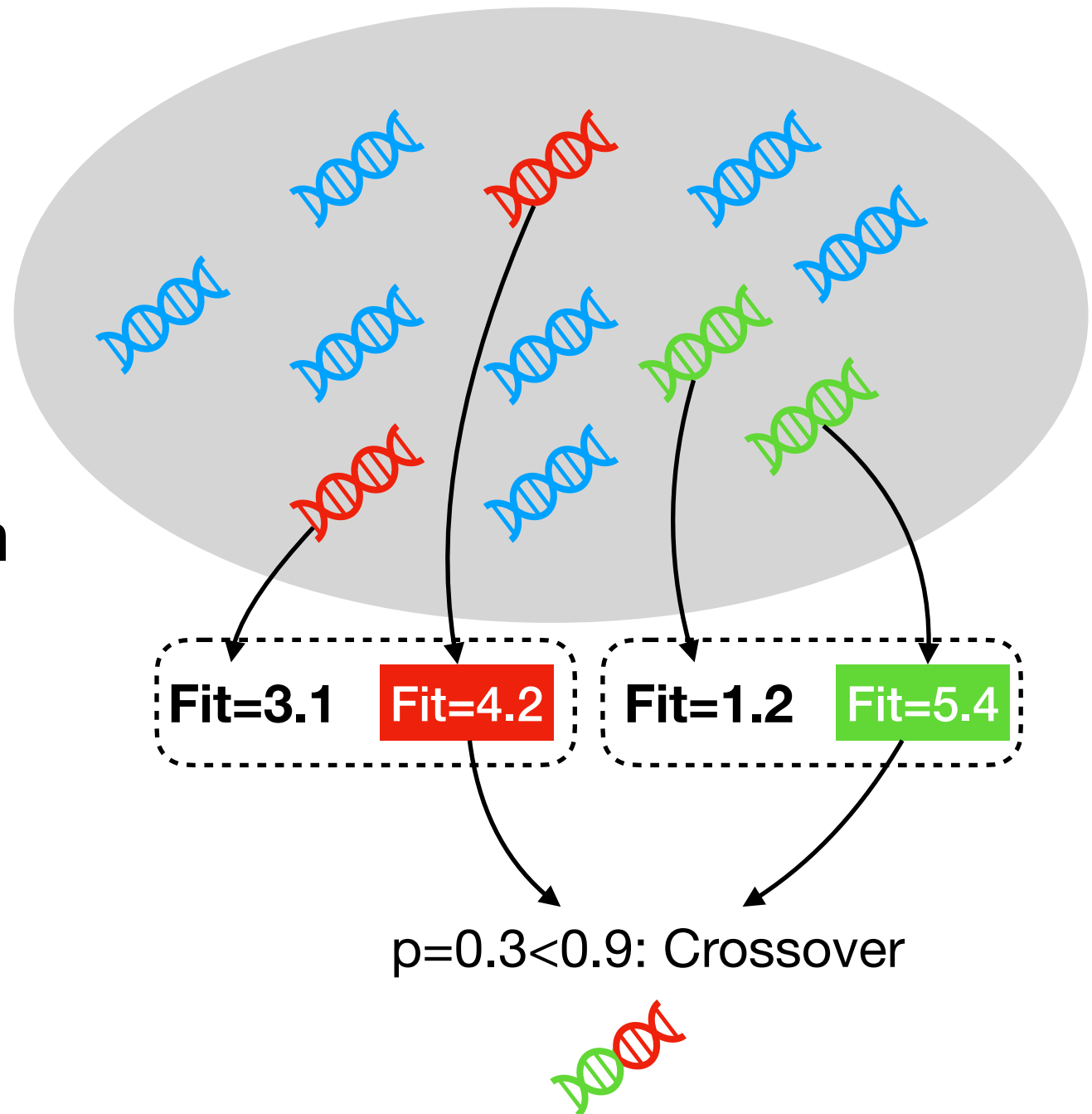
- Less frequent operator, typical mutation rate: 10%
- **Subtree mutation:** selection of a mutation point and replacement of the subtree by a randomly generated one
- Other mutation operators: randomly change function or terminal



Selection

- Any selection scheme can be used (fitness proportionate, ...)
- Frequently used: tournament-based selection
- then generation of offspring (in general either with crossover or mutation, not both at the same time)

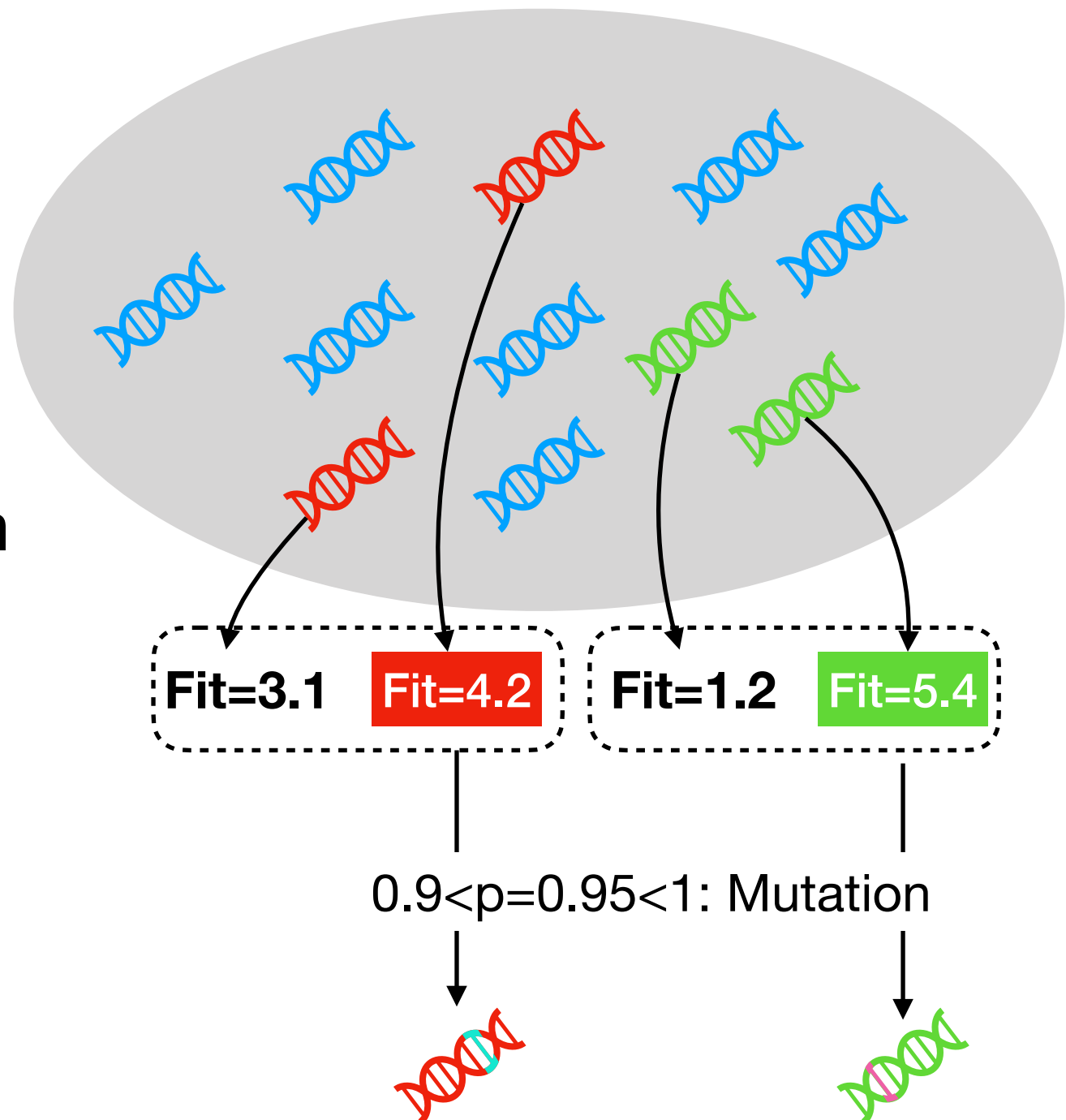
Tournament based selection



Selection

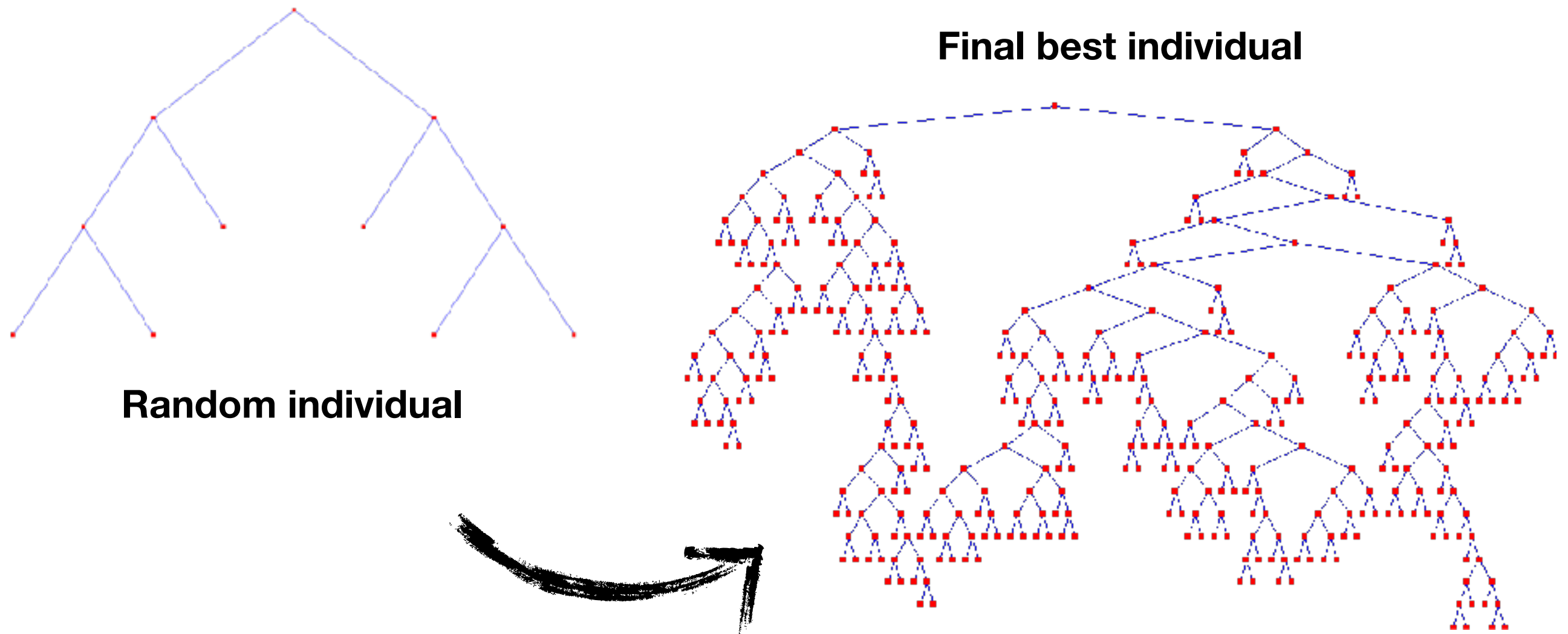
- Any selection scheme can be used (fitness proportionate, ...)
- Frequently used: tournament-based selection
- then generation of offspring (in general either with crossover or mutation, not both at the same time)

Tournament based selection



GP main challenge: bloat

- **Bloat:** uncontrolled and unbounded growth of genotypes



GP main challenge: bloat

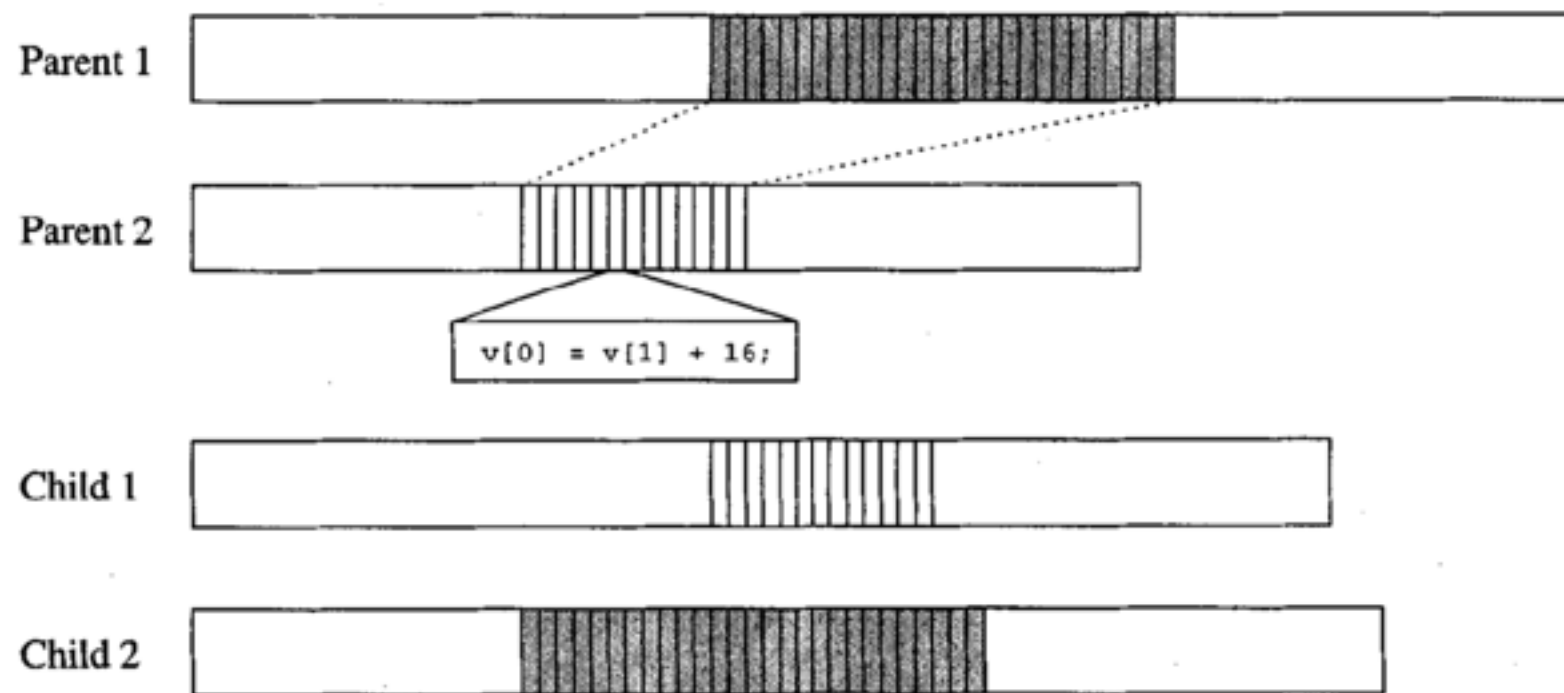
- How to deal with it ?
 - Put a limit on depth (for instance reject children with a depth exceeding a threshold) together with ...
 - **Multi-objective approach:** (fitness, parsimony)
 - **Double tournament:** 2 tournaments selecting on fitness and selection of the most parsimonious
 - ...

Preparing a GP run

- Questions:
 - What is the fitness measure ?
 - What is the primitive set ?
 - What are the terminals ?
 - What are the GP parameters (pop size, , nb_gen, mutation and crossover rate, ...) ?
 - Typical pop size > 500
 - Typical nb_gen= 10 to 50

Different flavours of GP

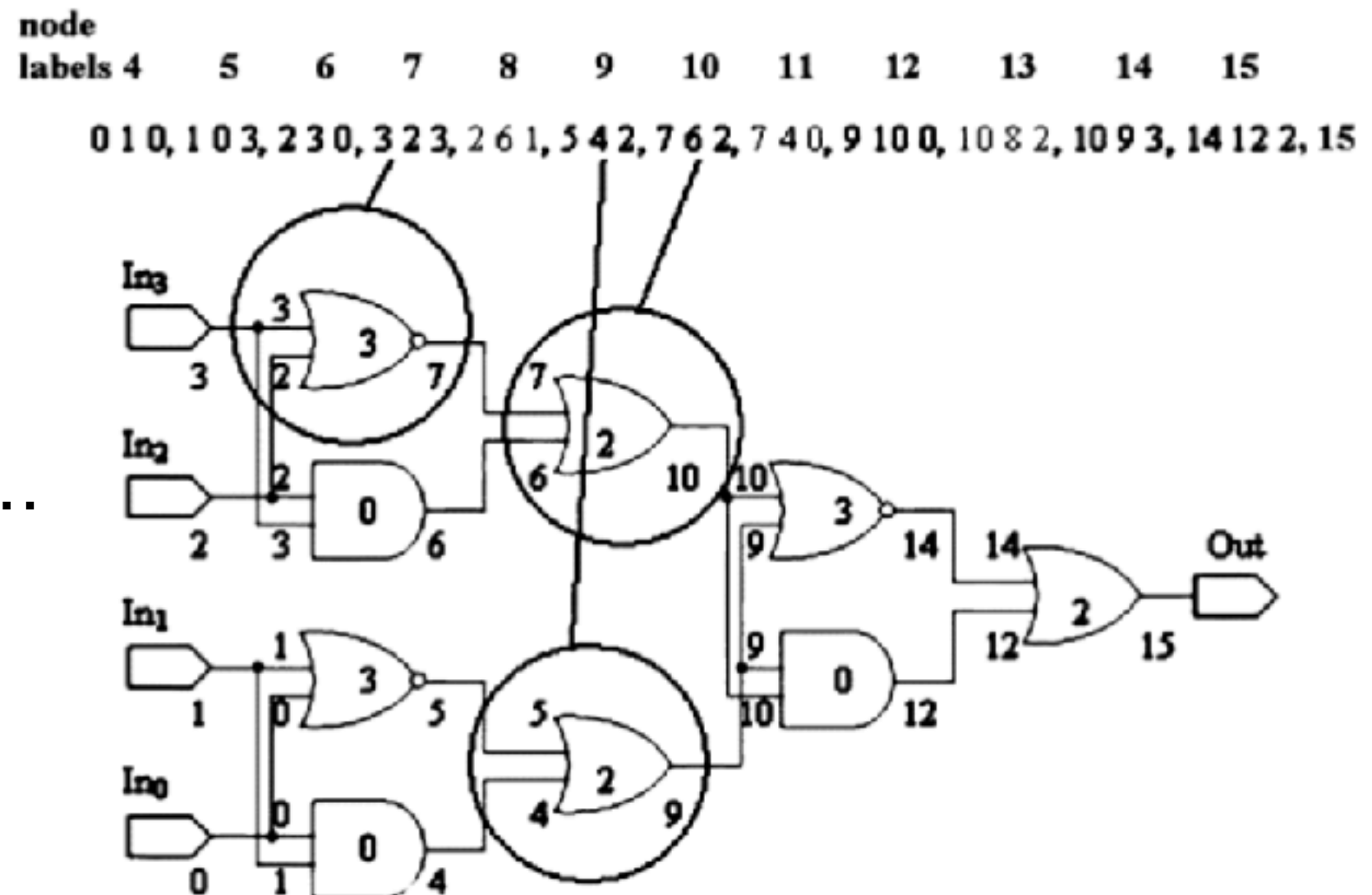
- Tree-like structures is not the only alternative:
- Linear GP: sequence of instructions



```
void ind(v)
  double v[8];
{
  ...
  v[0] = v[5] + 73;
  v[7] = v[0] - 59;      (I)
  if (v[1] > 0)
  if (v[5] > 21)
    v[4] = v[2] * v[1];
  v[2] = v[5] + v[4];    (I)
  v[6] = v[0] * 25;
  v[6] = v[4] - 4;
  v[1] = sin(v[6]);
  if (v[0] > v[1])      (I)
    v[3] = v[5] * v[5]; (I)
  v[7] = v[6] * 2;
  v[5] = v[7] + 115;    (I)
  if (v[1] <= v[6])
    v[1] = sin(v[7]);
}
```

Different flavours of GP

- Cartesian GP
- Advantage: no bloat...



Miller, J. F., & Smith, S. L. (2006). Redundancy and computational efficiency in cartesian genetic programming. *IEEE Transactions on Evolutionary Computation*, 10(2), 167-174.

Miller, J. (2001, July). What bloat? cartesian genetic programming on boolean problems. In *2001 Genetic and Evolutionary Computation Conference Late Breaking Papers* (pp. 295-302). San Francisco, California, USA.

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

$N = \{\text{expr}, \text{op}, \text{pre_op}\}$

$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$

$S = \langle \text{expr} \rangle$

```

(1) <expr> ::= <expr><op><expr>      (0)
              | (<expr><op><expr>)    (1)
              | <pre-op>(<expr>)    (2)
              | <var>                (3)
(2) <op> ::= +      (0)
              | -      (1)
              | /      (2)
              | *      (3)
(3) <pre-op> ::= Sin  (0)
              | Cos   (1)
              | Exp   (2)
              | Log   (3)
(4) <var> ::= X      (0)
              | 1.0   (1).
  
```

Genotype

220	240	220	203	101	53	202	203	102	55	220	202	241	130	37	202	203	140	39	202	203	102
-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----

S
 $\langle \text{expr} \rangle$
→
 $220\%4=0:$
 $\langle \text{expr} \rangle$
→
 $\langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

$N = \{\text{expr}, \text{op}, \text{pre_op}\}$

$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$

$S = \langle \text{expr} \rangle$

```

(1) <expr> ::= <expr><op><expr>      (0)
              | (<expr><op><expr>)    (1)
              | <pre-op>(<expr>)     (2)
              | <var>                (3)
(2) <op> ::= +      (0)
              | -      (1)
              | /      (2)
              | *      (3)
(3) <pre-op> ::= Sin  (0)
              | Cos   (1)
              | Exp   (2)
              | Log   (3)
(4) <var> ::= X      (0)
              | 1.0   (1).
    
```

Genotype

220	240	220	203	101	53	202	203	102	55	220	202	241	130	37	202	203	140	39	202	203	102
-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----

$240 \% 4 = 0:$
 $\langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$
→
 $\langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

$N = \{\text{expr}, \text{op}, \text{pre_op}\}$

$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$

$S = \langle \text{expr} \rangle$

```

(1) <expr> ::= <expr><op><expr>      (0)
              | (<expr><op><expr>)      (1)
              | <pre-op>(<expr>)      (2)
              | <var>                 (3)
(2) <op> ::= +      (0)
              | -      (1)
              | /      (2)
              | *      (3)
(3) <pre-op> ::= Sin  (0)
              | Cos   (1)
              | Exp   (2)
              | Log   (3)
(4) <var> ::= X      (0)
              | 1.0   (1).
  
```

Genotype

220	240	220	203	101	53	202	203	102	55	220	202	241	130	37	202	203	140	39	202	203	102
-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----

220%4=0:

<expr><op><expr><op><expr>



<expr><op><expr><op><expr><op><expr>

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

$N = \{\text{expr}, \text{op}, \text{pre_op}\}$

$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$

$S = \langle \text{expr} \rangle$

```

(1) <expr> ::= <expr><op><expr>      (0)
              | (<expr><op><expr>)    (1)
              | <pre-op>(<expr>)    (2)
              | <var>                (3)
(2) <op> ::= +      (0)
              | -      (1)
              | /      (2)
              | *      (3)
(3) <pre-op> ::= Sin  (0)
              | Cos   (1)
              | Exp   (2)
              | Log   (3)
(4) <var> ::= X      (0)
              | 1.0   (1).
    
```

Genotype

220	240	220	203	101	53	202	203	102	55	220	202	241	130	37	202	203	140	39	202	203	102
-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----

203%4=3:

<expr>**<op>****<expr>****<op>****<expr>****<op>****<expr>**



<var>**<op>****<expr>****<op>****<expr>****<op>****<expr>**

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

$$N = \{\text{expr}, \text{op}, \text{pre_op}\}$$

$$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$$

$$S = \langle \text{expr} \rangle$$

$$\begin{aligned} (1) \langle \text{expr} \rangle &::= \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle & (0) \\ &| (\langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle) & (1) \\ &| \langle \text{pre-op} \rangle (\langle \text{expr} \rangle) & (2) \\ &| \langle \text{var} \rangle & (3) \\ (2) \langle \text{op} \rangle &::= + & (0) \\ &| - & (1) \\ &| / & (2) \\ &| * & (3) \\ (3) \langle \text{pre-op} \rangle &::= \text{Sin} & (0) \\ &| \text{Cos} & (1) \\ &| \text{Exp} & (2) \\ &| \text{Log} & (3) \\ (4) \langle \text{var} \rangle &::= X & (0) \\ &| 1.0 & (1). \end{aligned}$$

Genotype

220	240	220	203	101	53	202	203	102	55	220	202	241	130	37	202	203	140	39	202	203	102
-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----

$101 \% 2 = 1$
 $\langle \text{var} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$



$1.0 \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

$N = \{\text{expr}, \text{op}, \text{pre_op}\}$

$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$

$S = \langle \text{expr} \rangle$

```

(1) <expr> ::= <expr><op><expr>      (0)
              | (<expr><op><expr>)    (1)
              | <pre-op>(<expr>)    (2)
              | <var>                (3)
(2) <op> ::= +      (0)
              | -      (1)
              | /      (2)
              | *      (3)
(3) <pre-op> ::= Sin  (0)
              | Cos   (1)
              | Exp   (2)
              | Log   (3)
(4) <var> ::= X      (0)
              | 1.0   (1).
  
```

Genotype

220	240	220	203	101	53	202	203	102	55	220	202	241	130	37	202	203	140	39	202	203	102
-----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----	-----	----	-----	-----	-----	----	-----	-----	-----

$53 \% 4 = 1$

1.0 <op> <expr> <op> <expr> <op> <expr>



1.0 - <expr> <op> <expr> <op> <expr>

Different flavours of GP: Grammatical Evolution

- Advantage: add constraints, types, ...

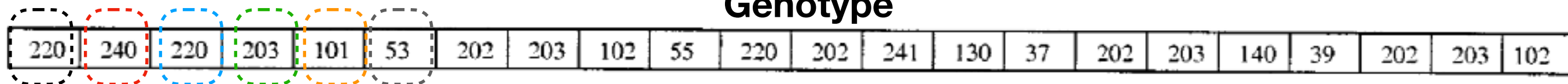
$$N = \{\text{expr}, \text{op}, \text{pre_op}\}$$

$$T = \{\text{Sin}, \text{Cos}, \text{Exp}, \text{Log}, +, -, /, *, X, 1.0, (,)\}$$

$$S = \langle \text{expr} \rangle$$

$$\begin{aligned} (1) \langle \text{expr} \rangle &::= \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle & (0) \\ &| (\langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle) & (1) \\ &| \langle \text{pre-op} \rangle (\langle \text{expr} \rangle) & (2) \\ &| \langle \text{var} \rangle & (3) \\ (2) \langle \text{op} \rangle &::= + & (0) \\ &| - & (1) \\ &| / & (2) \\ &| * & (3) \\ (3) \langle \text{pre-op} \rangle &::= \text{Sin} & (0) \\ &| \text{Cos} & (1) \\ &| \text{Exp} & (2) \\ &| \text{Log} & (3) \\ (4) \langle \text{var} \rangle &::= X & (0) \\ &| 1.0 & (1). \end{aligned}$$

Genotype

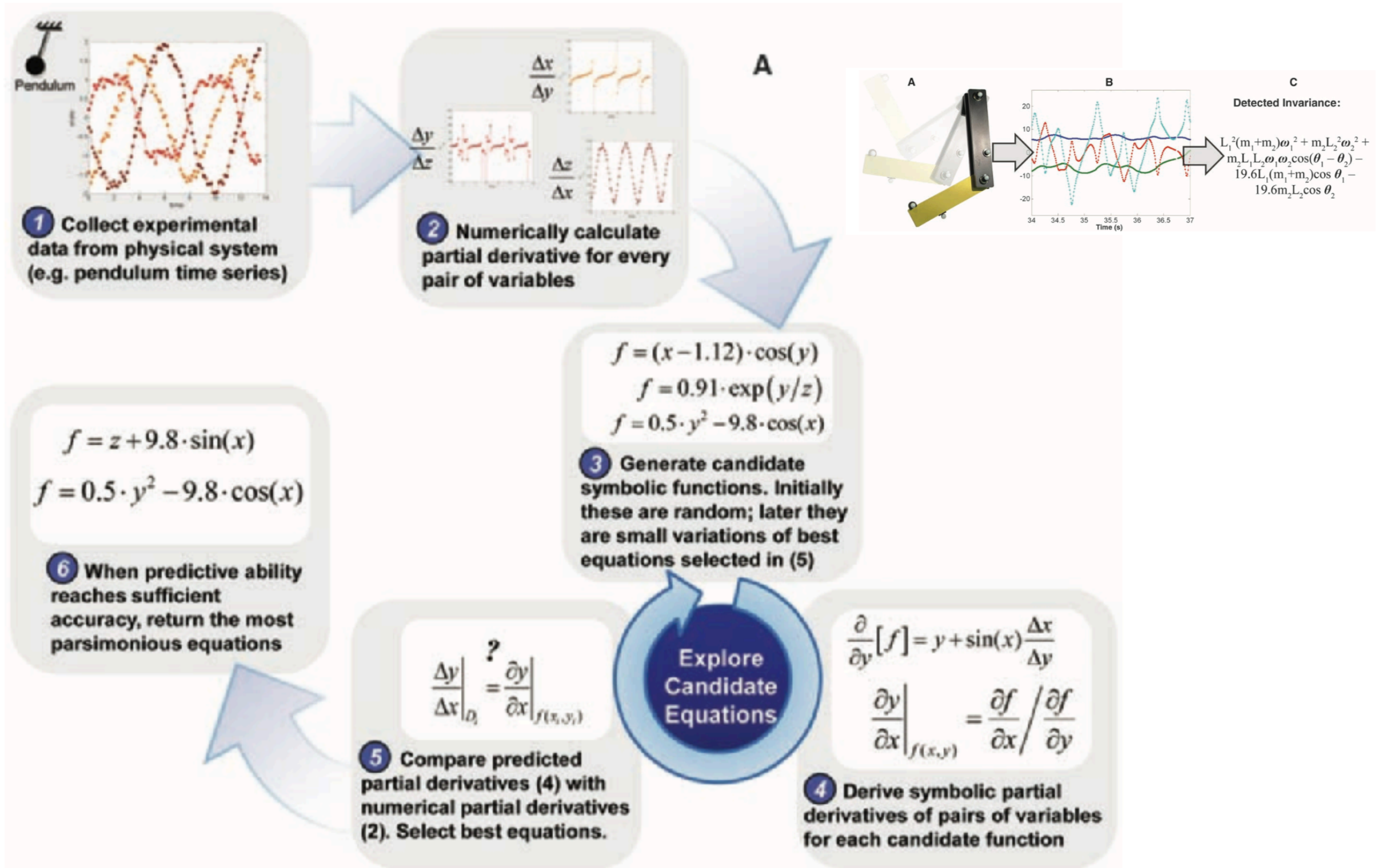


1.0 - $\langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle \langle \text{op} \rangle \langle \text{expr} \rangle$

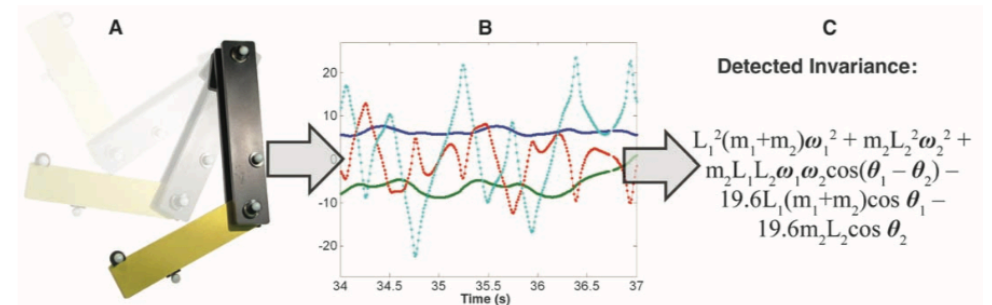
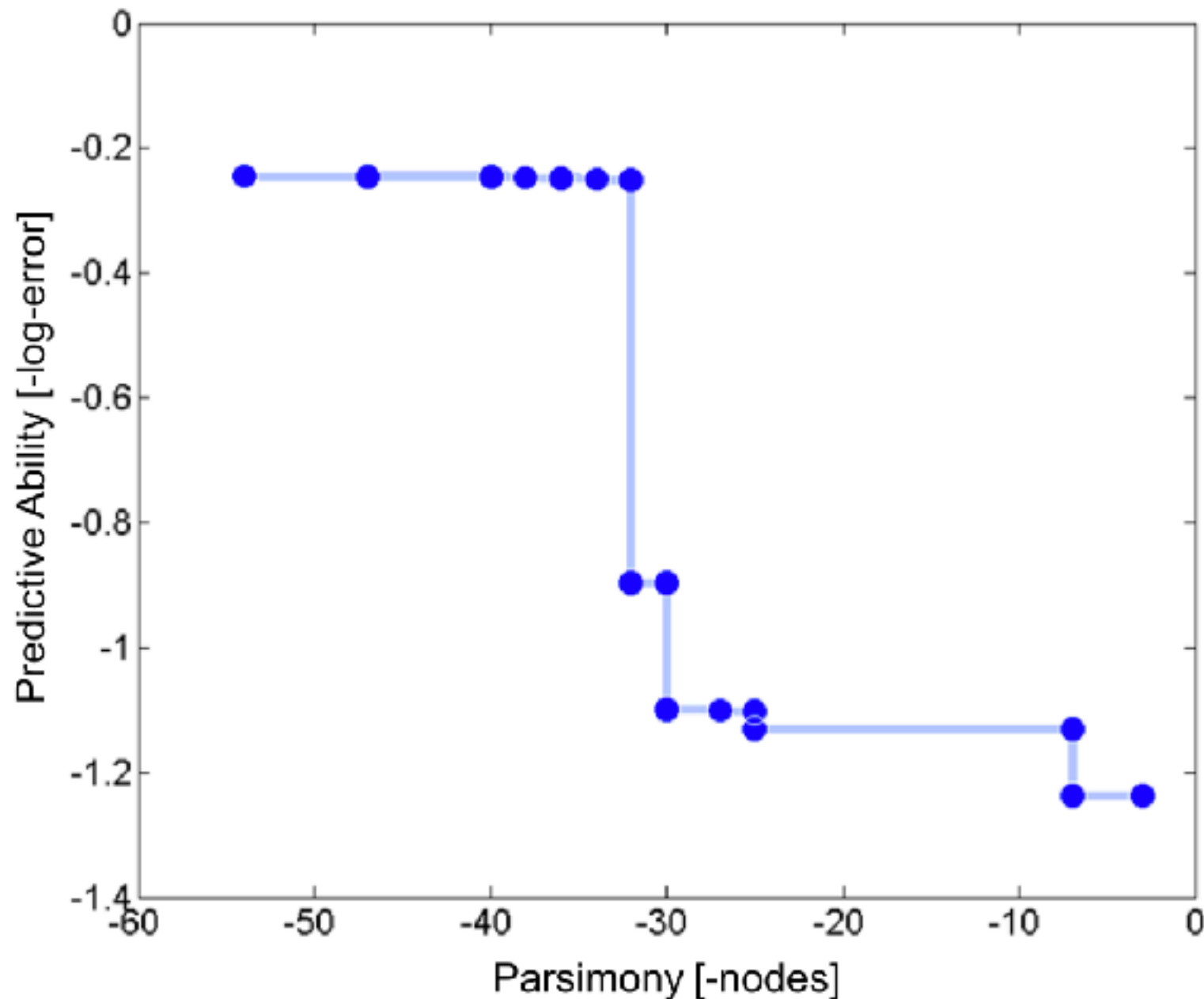


...

Back to Schmidt & Lipson work



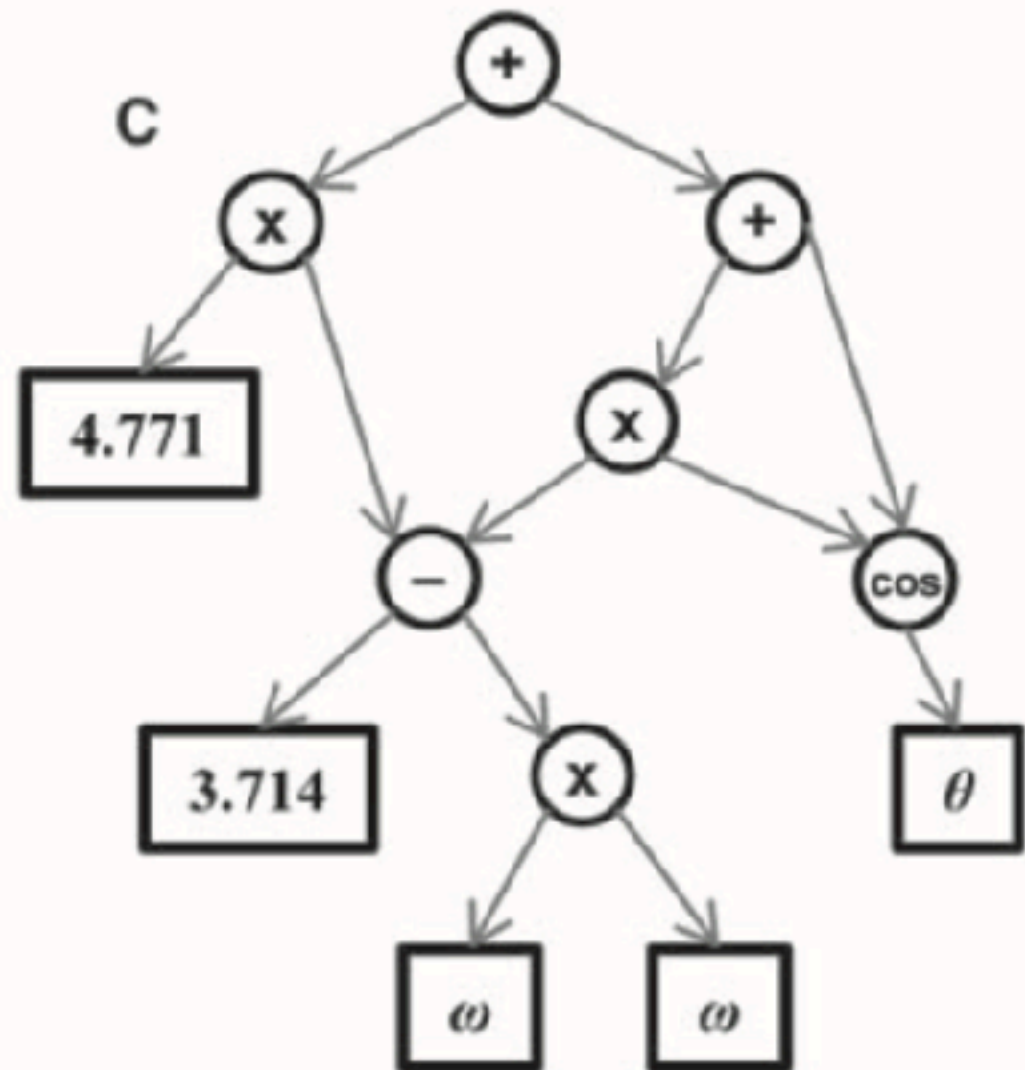
Back to Schmidt & Lipson work



Back to Schmidt & Lipson work

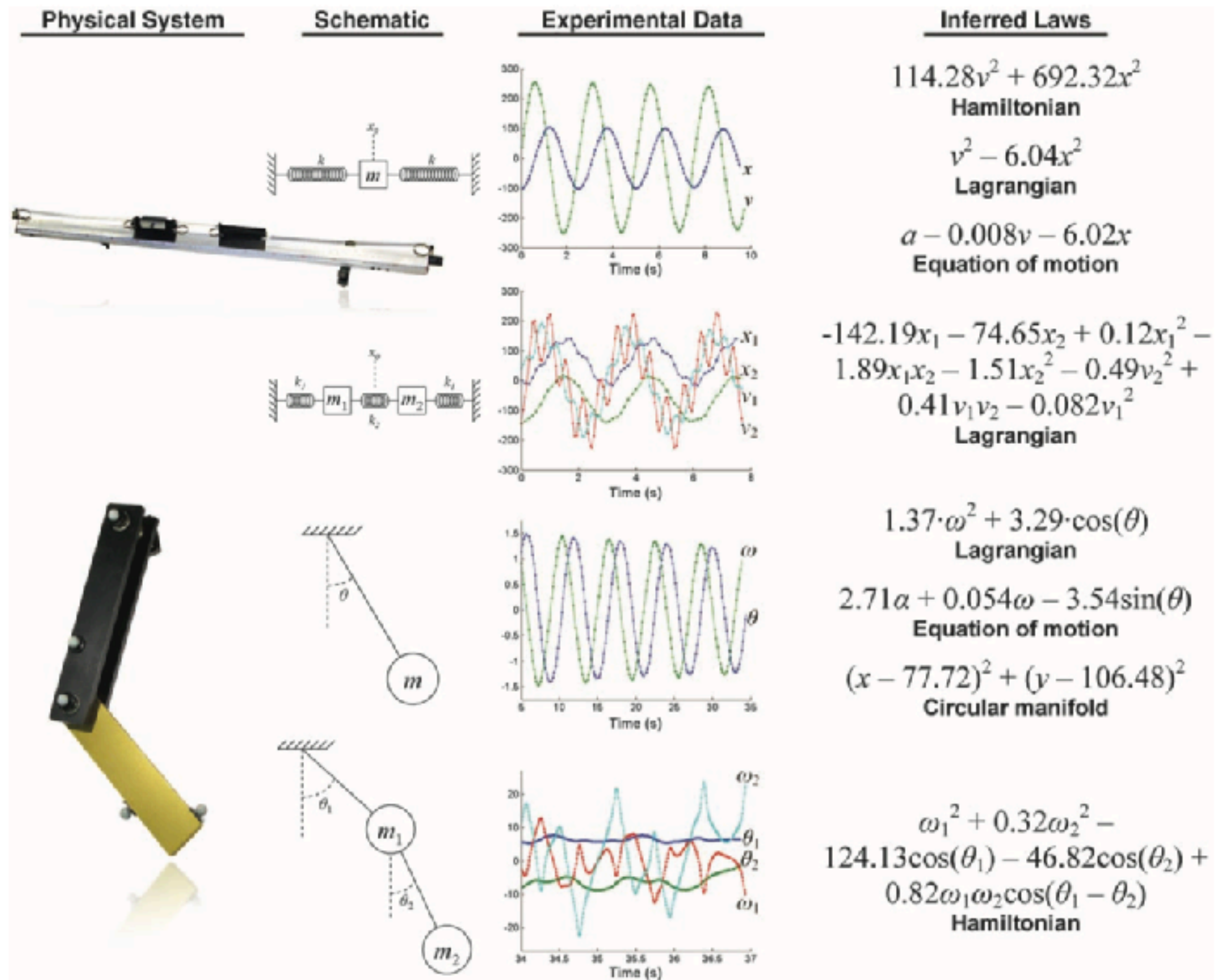
$$f(\theta, \omega) = 4.771 \cdot (3.714 - \omega^2) + \cos(\theta) + (3.714 - \omega^2) \cdot \cos(\theta)$$

```
(0) <- load [3.714]
(1) <- load [ $\omega$ ]
(2) <- mul (1), (1)
(3) <- sub (0), (2)
(4) <- load [ $\theta$ ]
(5) <- cos (4)
(6) <- mul (3), (5)
(7) <- load [4.771]
(8) <- mul (7), (3)
(9) <- add (8), (5)
(10) <- add (9), (6)
```



- Mutation: change operation or constant
- Crossover: 1 point crossover

Back to Schmidt & Lipson work



Quiz

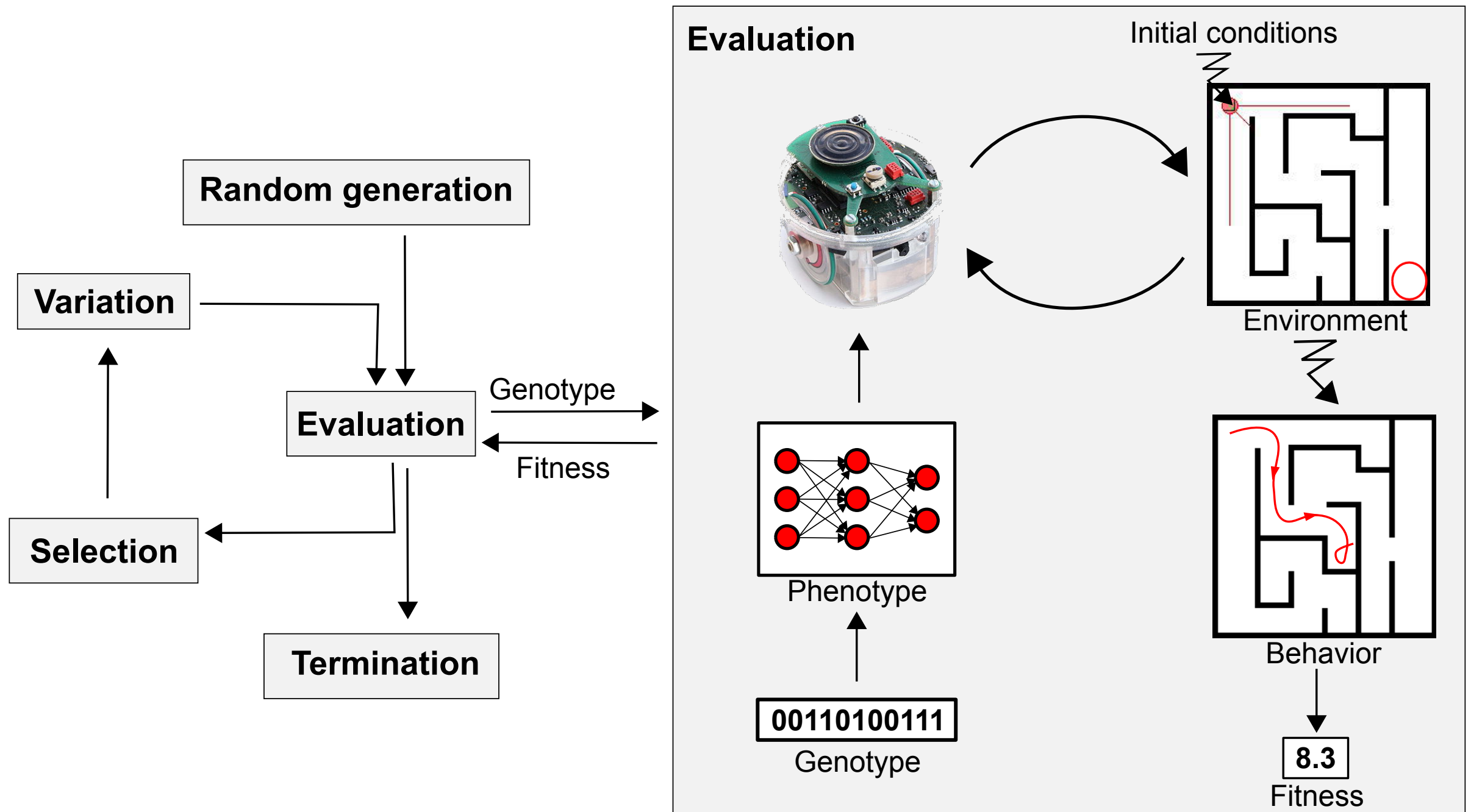
- What is « bloat » ? Does it appear with all GP methods ?
- How to represent a program ?

Overview



- Evolving Morphologies
- Genetic Programming
- Neuroevolution
- Evolutionary algorithms & selective pressures
- Neuroevolution/GP in Deep learning era
- Building you own representation

Neuroevolution & robotics



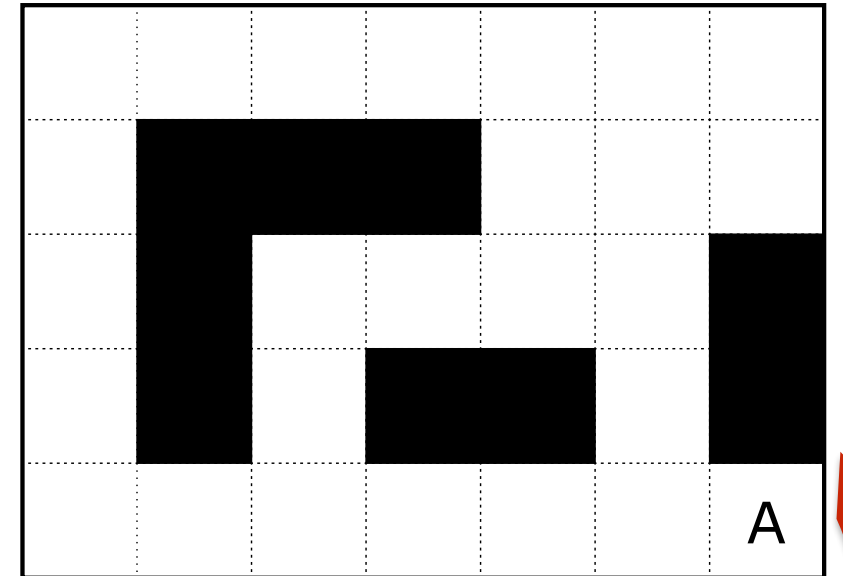
Neuroevolution: Motivation (1)

Reinforcement learning in robotics:

- Real World continuous and, frequently, partially observable
- High dimension of perceptions and actions
- Importance of time and dynamic of movement
- Imperfect and noisy sensors and effectors
- ...

→ Use direct policy search !

- Parameterized policy: $\pi_{\theta} : S \rightarrow A$
- Find θ maximizing the cumulated reward during an episode $f(\theta)$



Neuroevolution: Motivation (2)

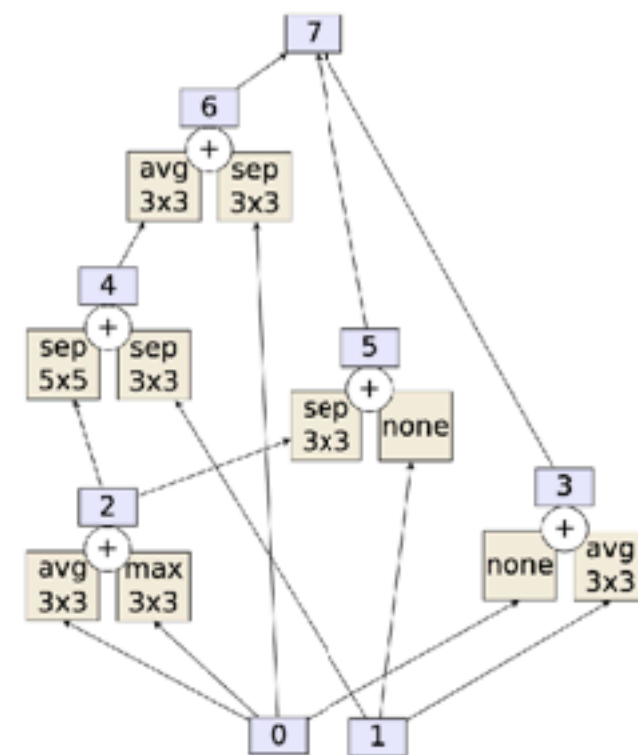
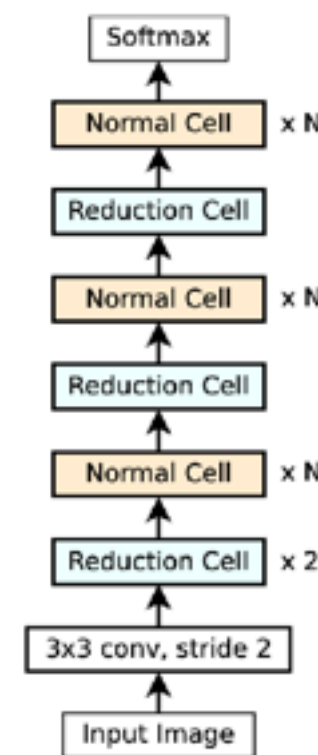
- Neuroevolution with natural ES competitive with deep learning:
 - Highly parallelizable: with 1440 processes working in parallel, MuJoCo 3D humanoid solved in less than 10min
 - Better exploration than TRPO on MuJoCo humanoid task
 - ES robust: a single set of hyper parameters for all Atari experiments and a single (but different) set for all MuJoCo experiments
 - Not that bad data efficiency, between 3x and 10x wrt A3C or TRPO, but no backprop...

Neuroevolution: Motivation (3)

- ... but EA can go beyond parameter tuning: architecture search !

Model	# Parameters	# Multiply-Adds	Top-1 / Top-5 Accuracy (%)
Incep-ResNet V2 [44]	55.8M	13.2B	80.4 / 95.3
ResNeXt-101 [48]	83.6M	31.5B	80.9 / 95.6
PolyNet [51]	92.0M	34.7B	81.3 / 95.8
Dual-Path-Net-131 [7]	79.5M	32.0B	81.5 / 95.8
GeNet-2 [47]*	156M	—	72.1 / 90.4
Block-QNN-B [52]*	—	—	75.7 / 92.6
Hierarchical [30]*	64M	—	79.7 / 94.8
NASNet-A [54]	88.9M	23.8B	82.7 / 96.2
PNASNet-5 [29]	86.1M	25.0B	82.9 / 96.2
AmoebaNet-A (N=6, F=190)*	86.7M	23.1B	82.8 / 96.1
AmoebaNet-A (N=6, F=448)*	469M	104B	83.9 / 96.6

Results on ImageNet

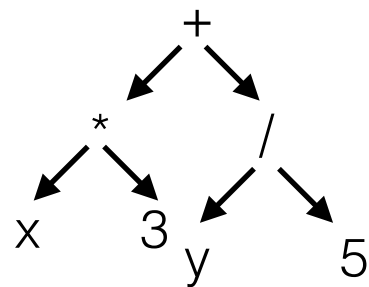


How to evolve a neural network ?

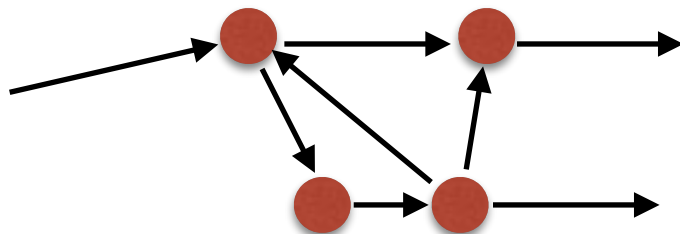
EA common genotypes

01001110

[1.23 42.3 7.53 5.8]



???



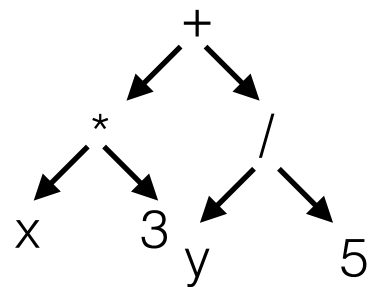
- Bit string (Genetic algorithms)
- Vector of real parameters (Evolution Strategies)
- Programs: trees, linear programs (Genetic Programming)
- Specific representations ...
- Graphs (Evolutionary Programming)

How to evolve a neural network ?

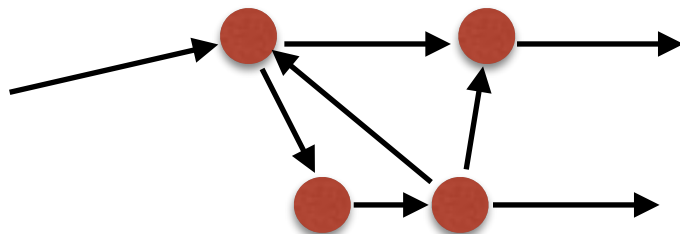
EA common genotypes

01001110

[1.23 42.3 7.53 5.8]



???



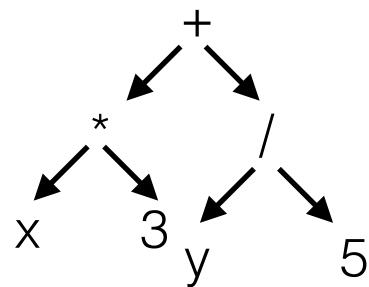
- Bit string (Genetic algorithms)
- Vector of real parameters (Evolution Strategies)
- Programs: trees, linear programs (Genetic Programming)
- Specific representations ...
- Graphs (Evolutionary Programming)

How to evolve a neural network ?

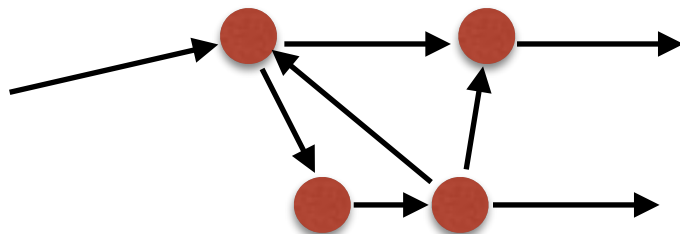
EA common genotypes

01001110

[1.23 42.3 7.53 5.8]



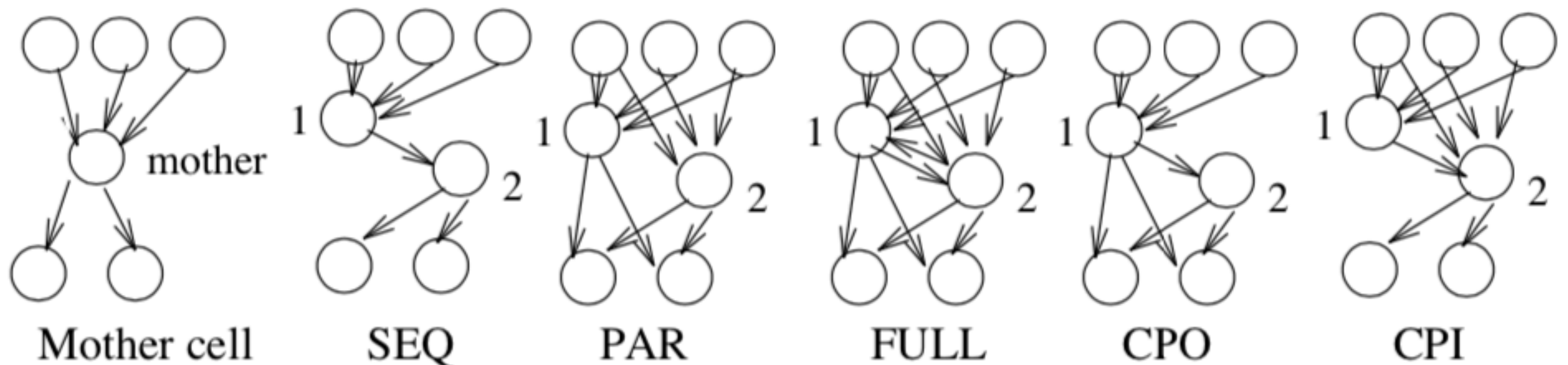
???



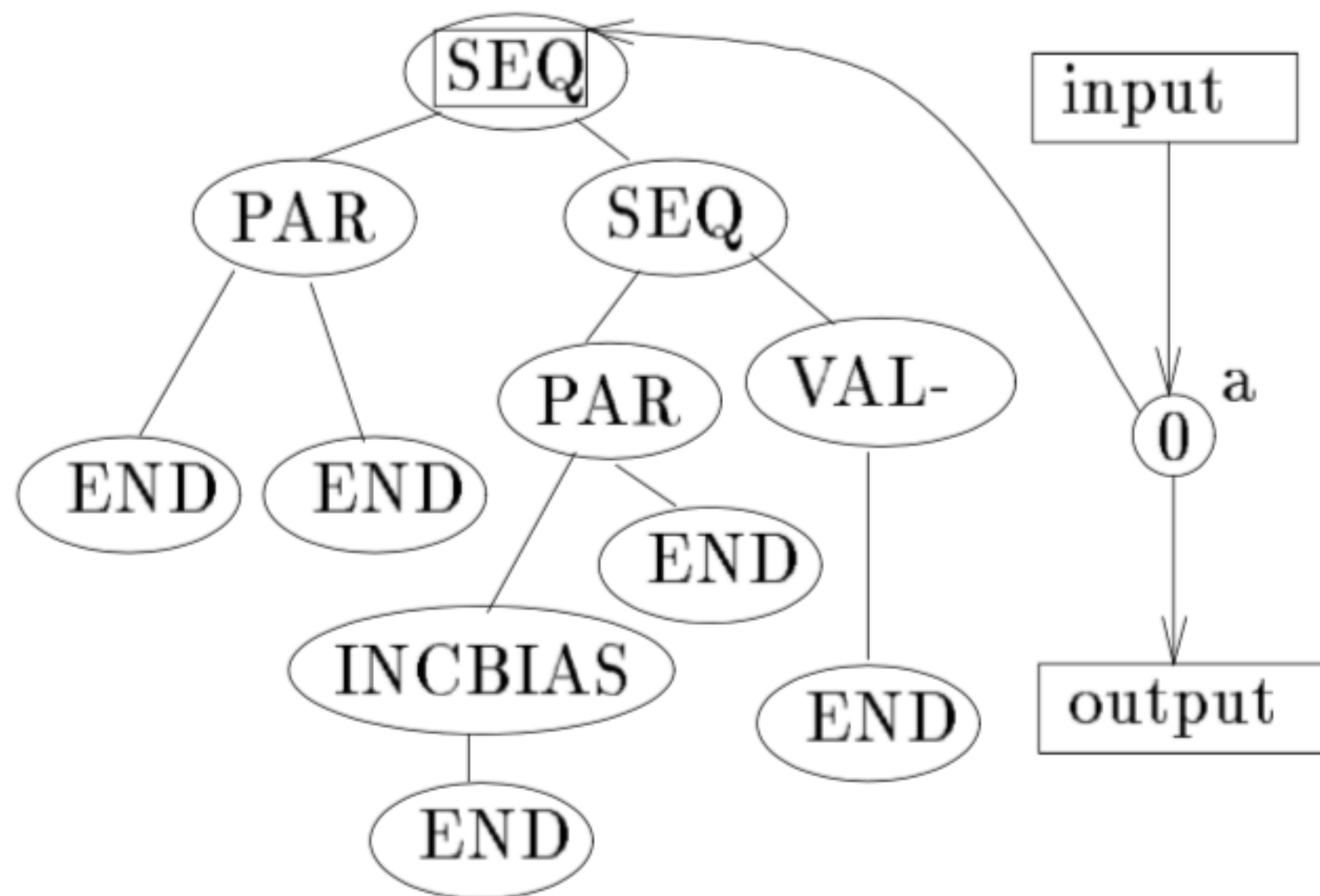
- Bit string (Genetic algorithms)
- Vector of real parameters (Evolution Strategies)
- Programs: trees, linear programs (Genetic Programming)
- Specific representations ...
- Graphs (Evolutionary Programming)

A program to describe a neural network: Cellular Encoding

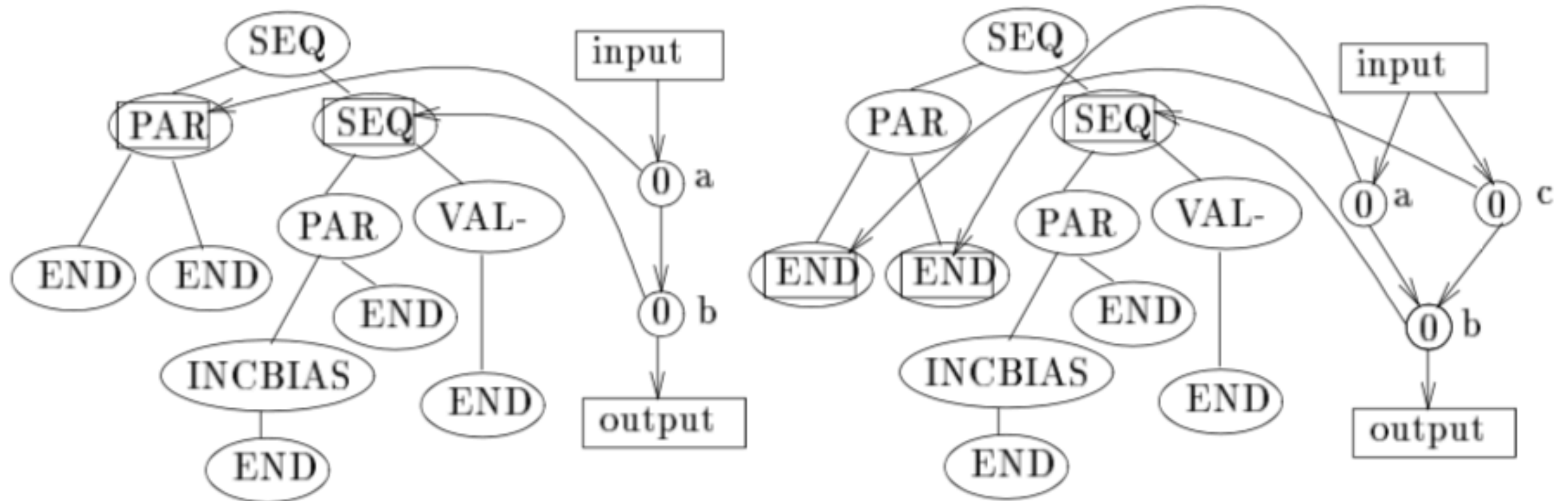
- Neuroevolution based on Genetic Programming
 - Primitive function set:



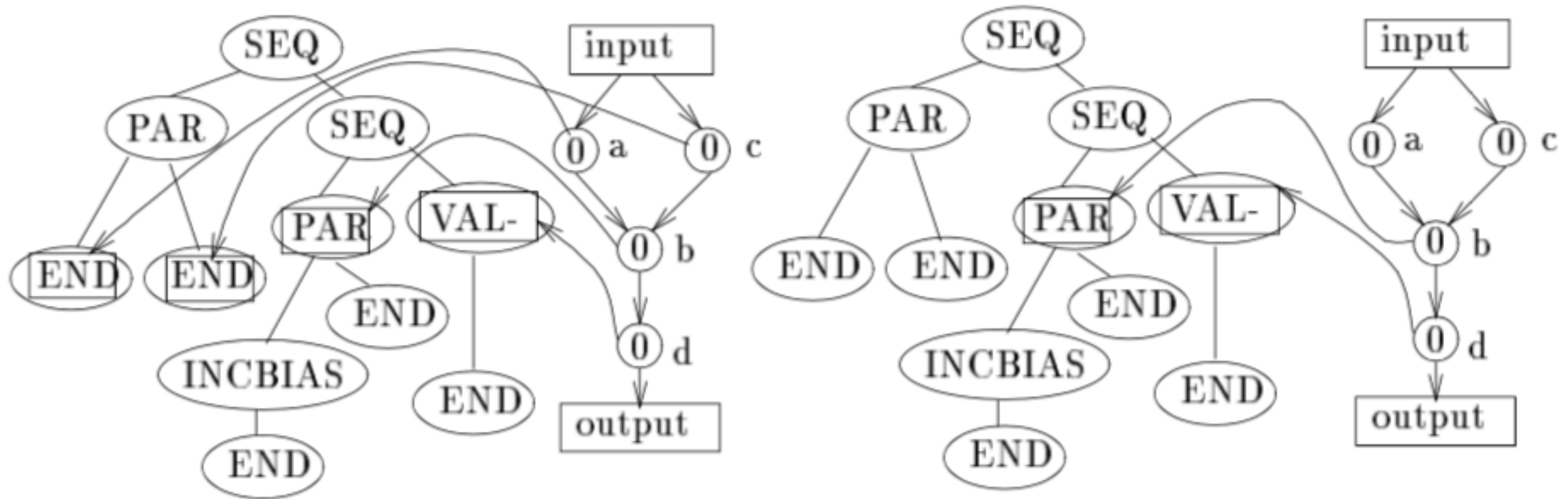
A program to describe a neural network: Cellular Encoding

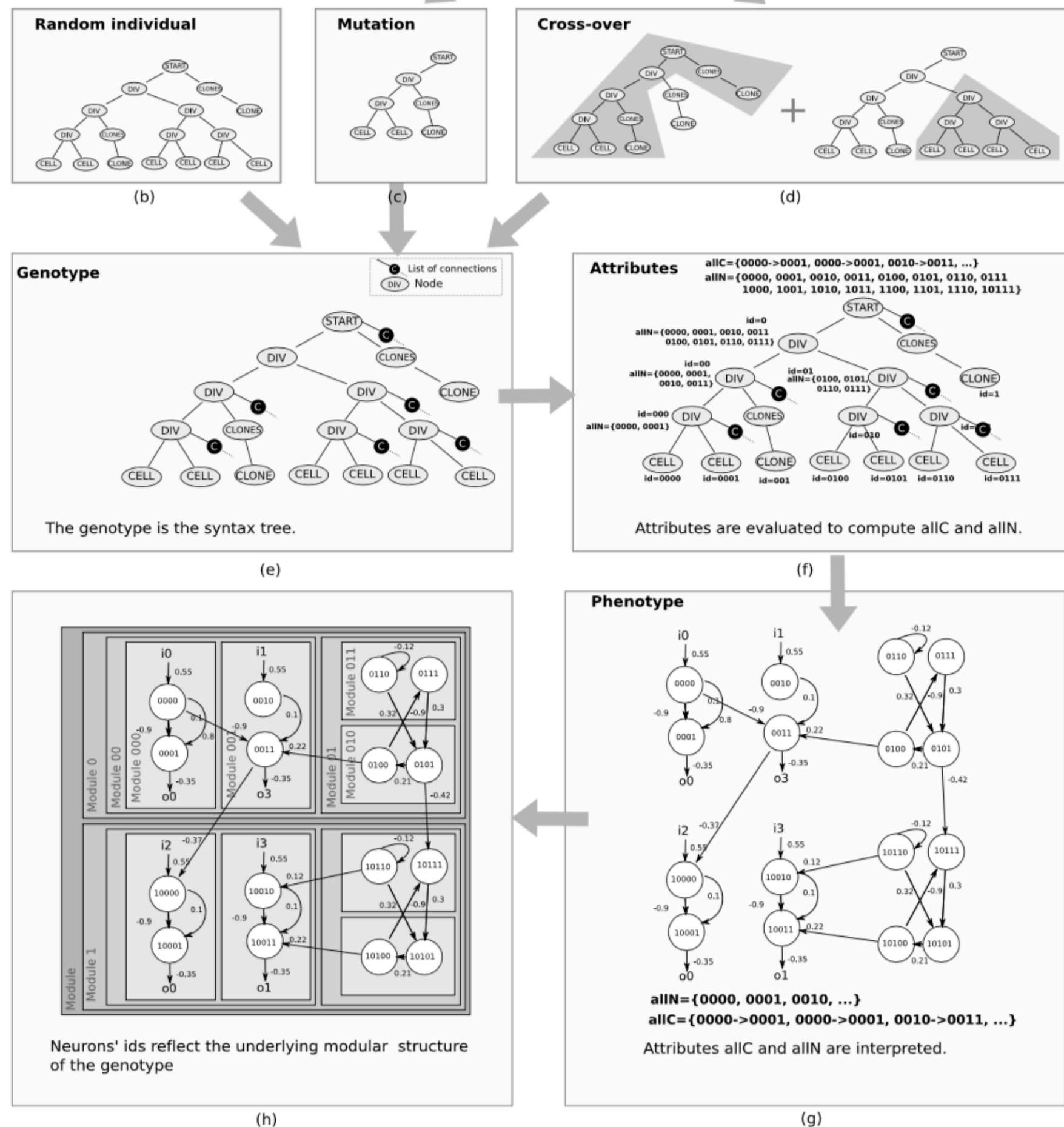
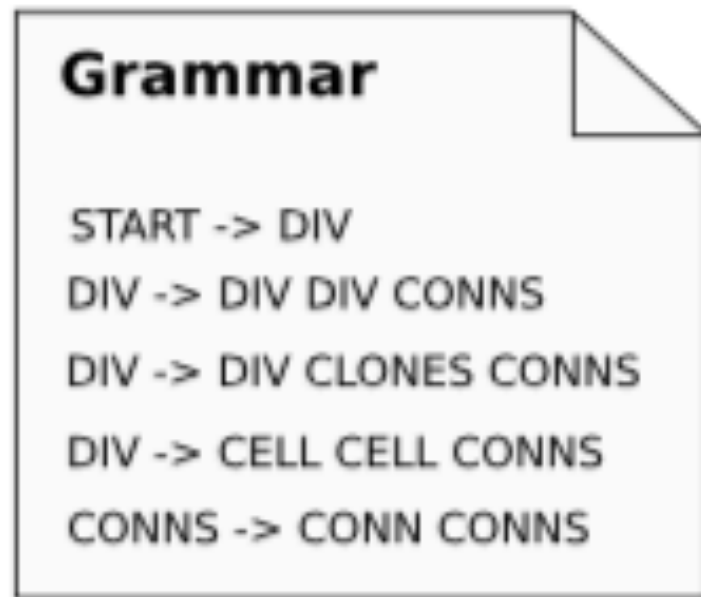


A program to describe a neural network: Cellular Encoding



A program to describe a neural network: Cellular Encoding



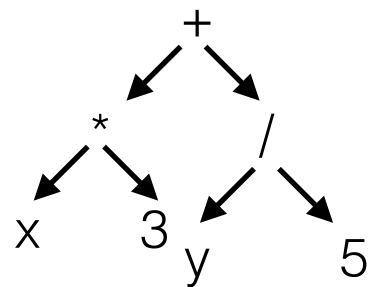


How to evolve a neural network ?

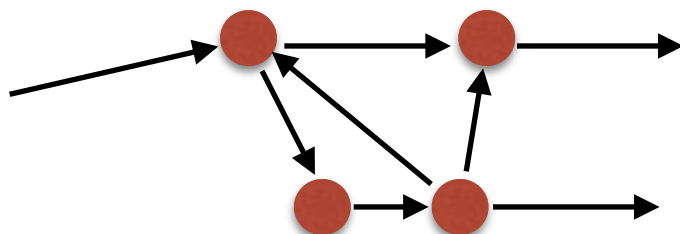
EA common genotypes

01001110

[1.23 42.3 7.53 5.8]



???



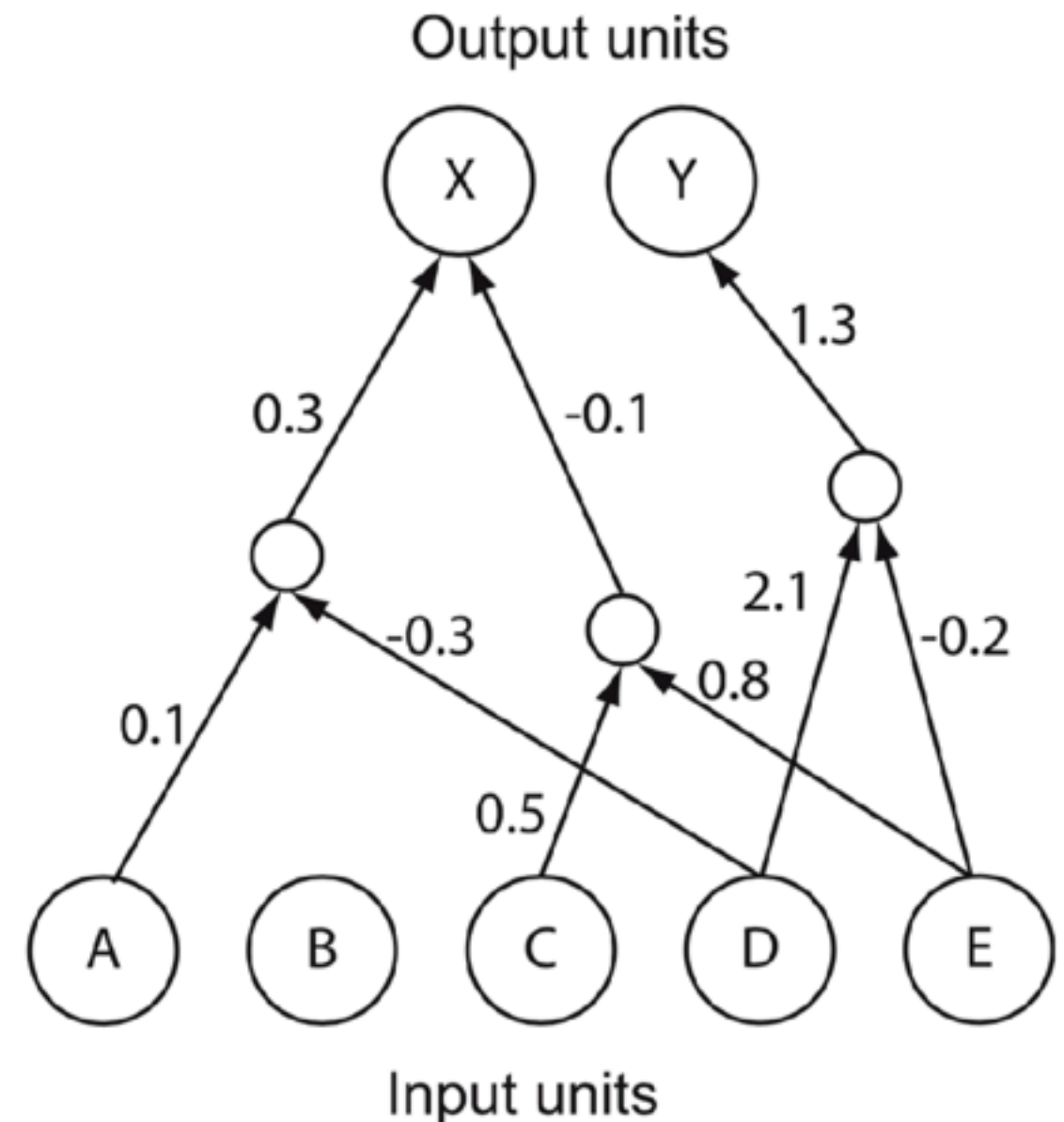
- Bit string (Genetic algorithms)
- Vector of real parameters (Evolution Strategies)
- Programs: trees, linear programs (Genetic Programming)
- Specific representations ...
- Graphs (Evolutionary Programming)

Specific representations: SANE

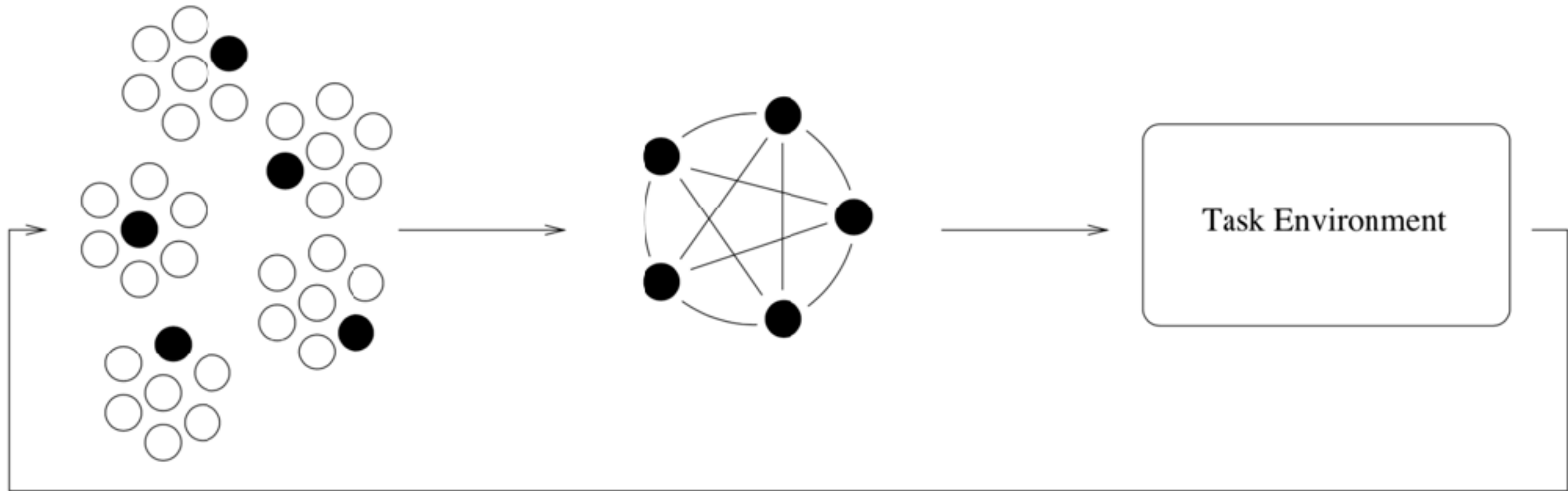
Label Weight

A	0.1	D	-0.3	X	0.3
C	0.5	E	0.8	X	-0.1
D	2.1	E	-0.2	Y	1.3

Decoding
→



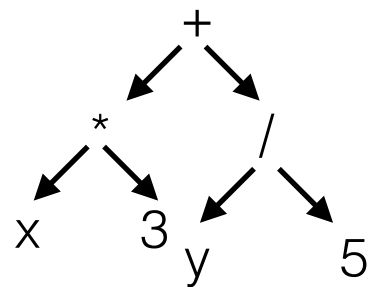
Specific representations: ESP



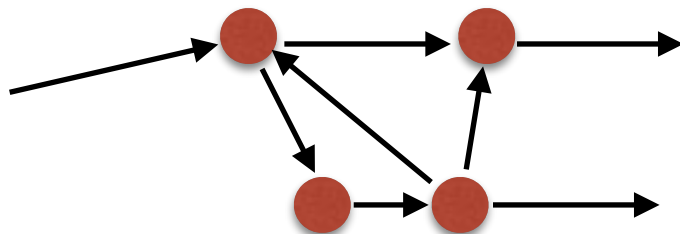
EA common genotypes

01001110

[1.23 42.3 7.53 5.8]



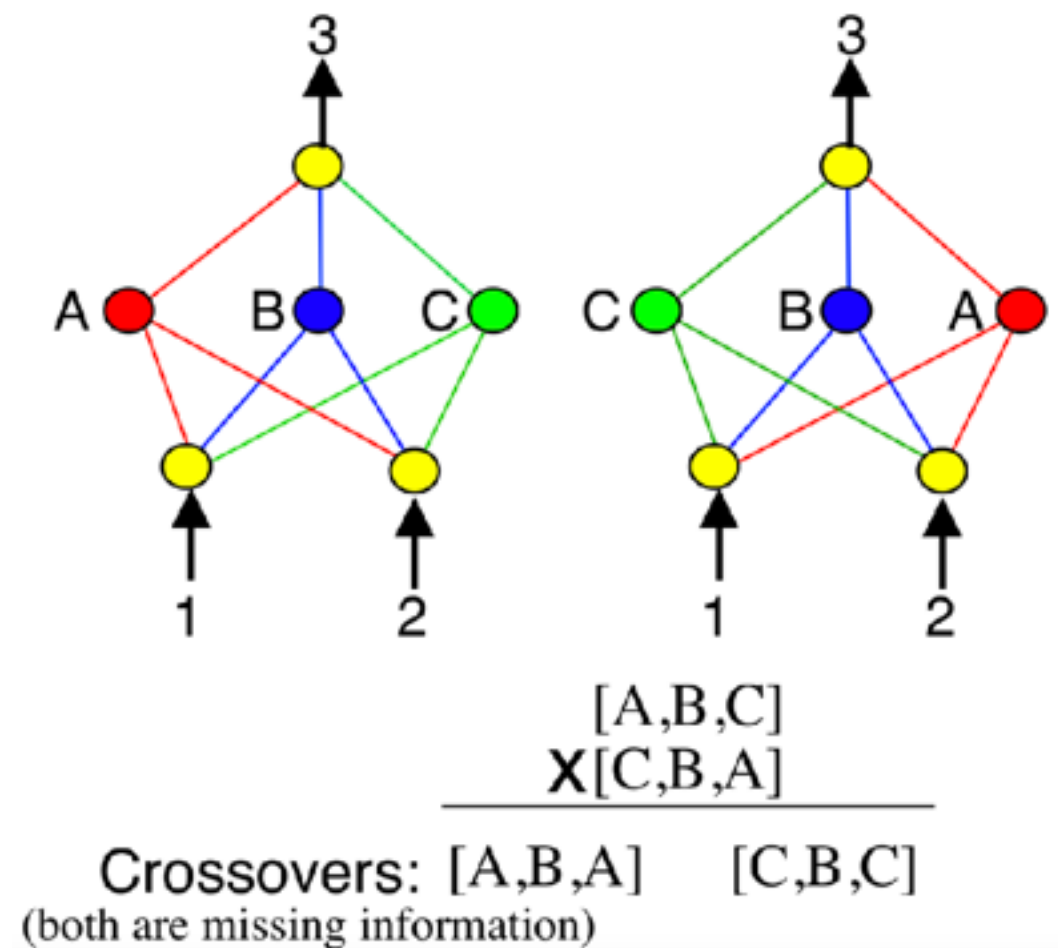
???



- Bit string (Genetic algorithms)
- Vector of real parameters (Evolution Strategies)
- Programs: trees, linear programs (Genetic Programming)
- Specific representations ...
- **Graphs (Evolutionary Programming)**

Challenges in Neuroevolution

1. Competing convention
2. Protecting innovation



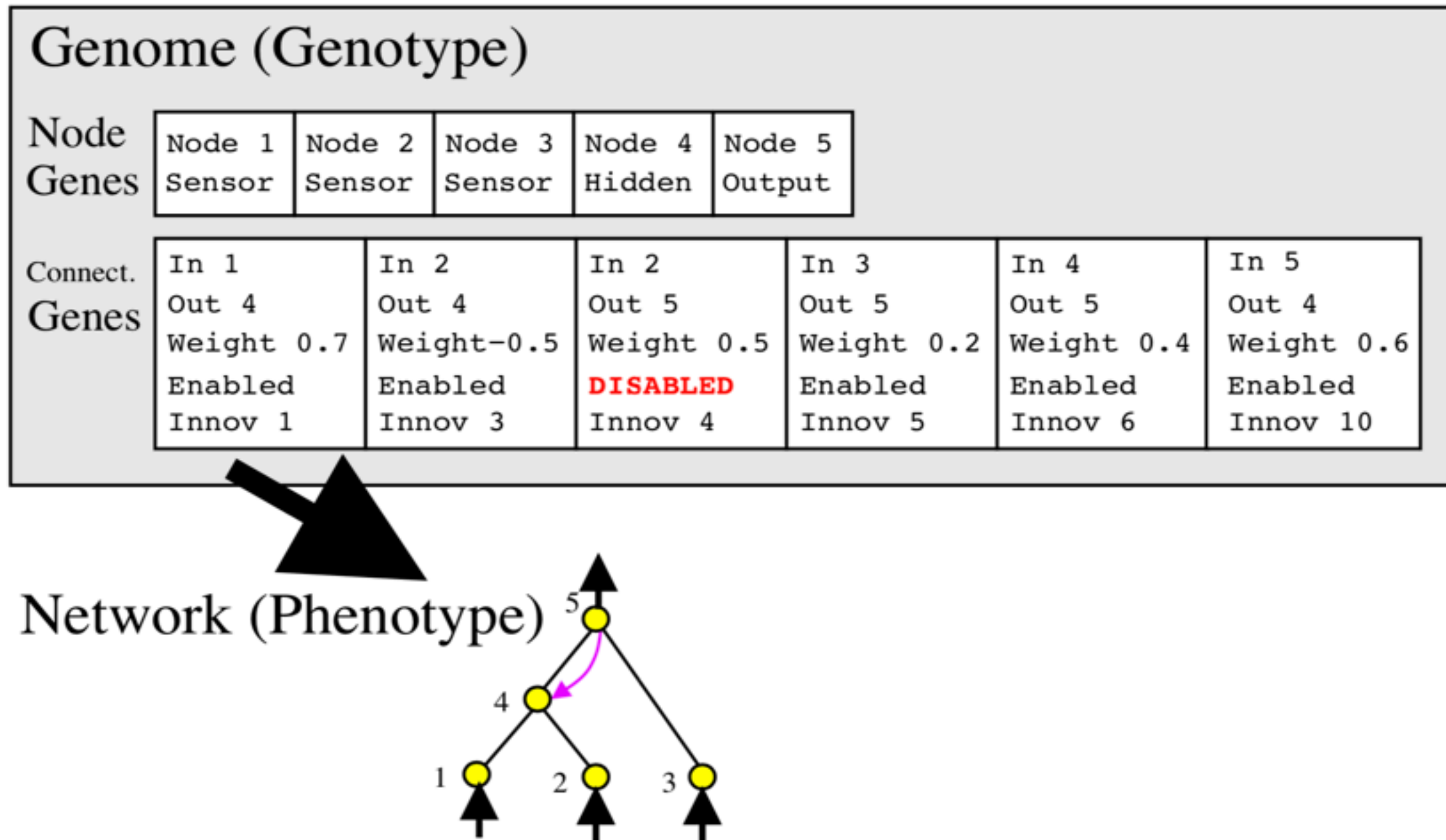
NEAT

NeuroEvolution of Augmenting Topologies

- Features:
 - Direct encoding of the neural network
 - Specific crossover operator to deal with competing convention
 - Speciation-based selection algorithm to protect innovation
 - « Start simple » and mutation operators to augment topology complexity

NEAT

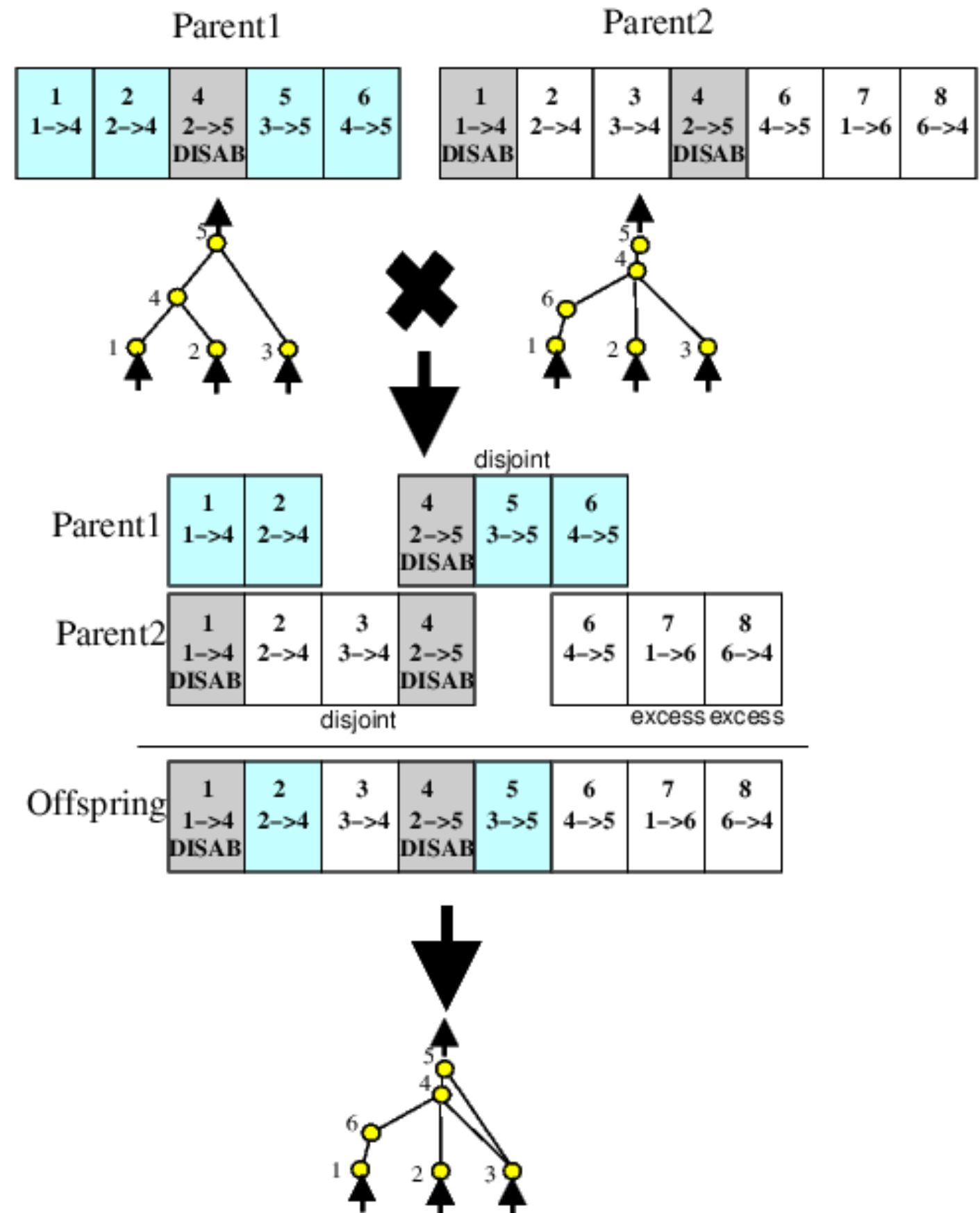
NeuroEvolution of Augmenting Topologies



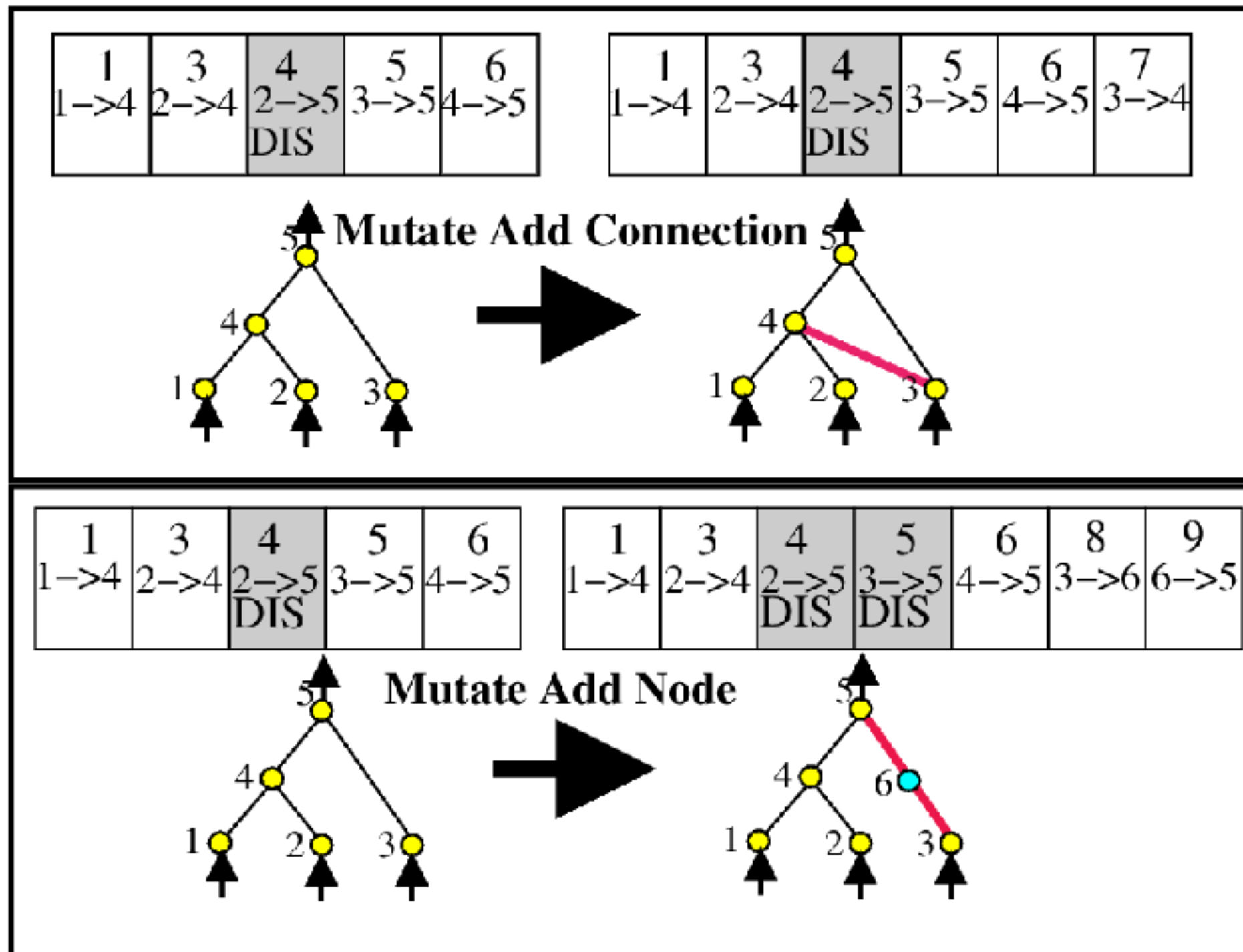
NEAT

Crossover

- Crossover based on an « innovation number »



NEAT: mutation



NEAT: protecting innovation

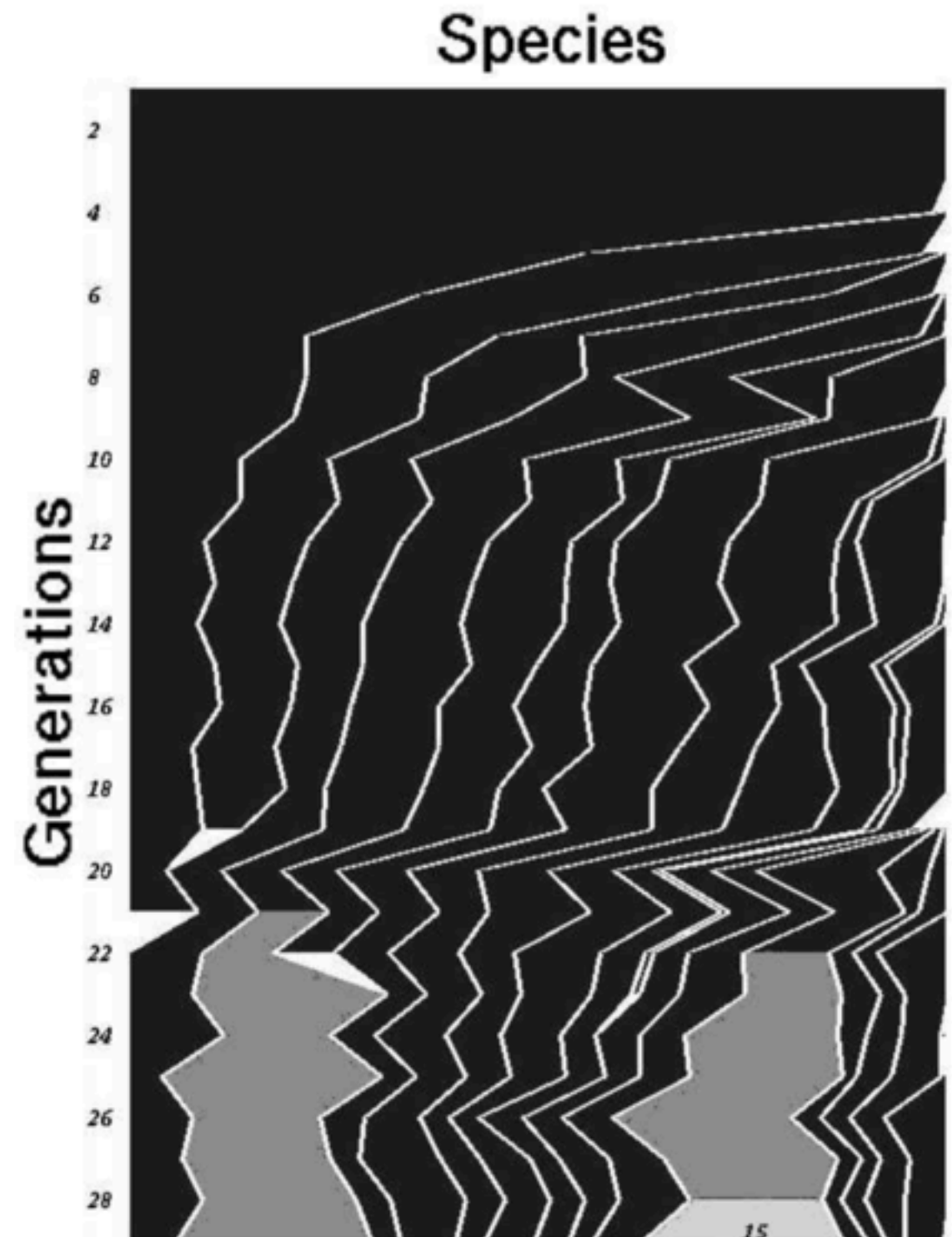
- Speciation mechanism:

$$\delta = \frac{c_1 E}{N} + \frac{c_2 D}{N} + c_3 \overline{W}$$

Threshold : δ_t

- Fitness sharing:

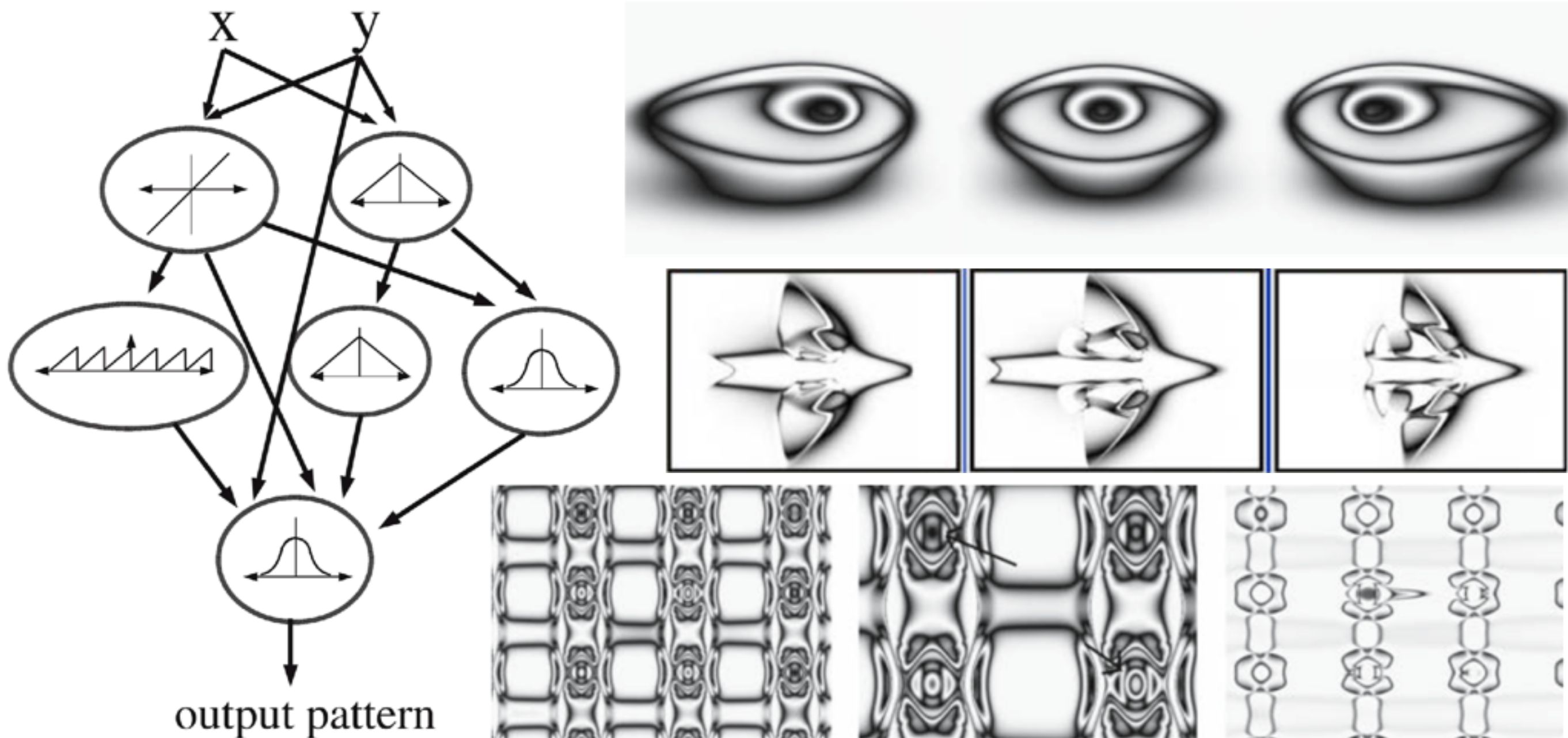
$$f'_i = \frac{f_i}{\sum_{j=1}^n sh(\delta(i, j))}$$



Neuroevolution: NEAT

Complex Networks
of Simple Neurons
for Bipedal Locomotion

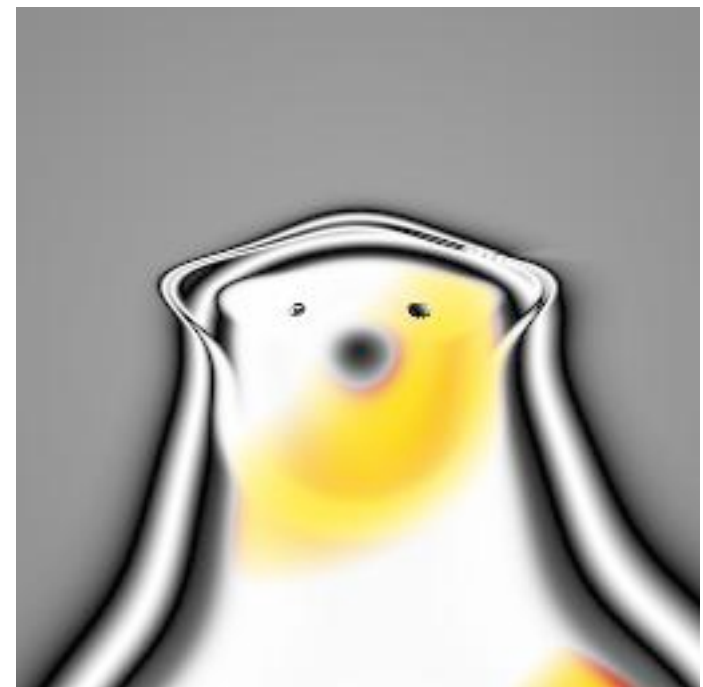
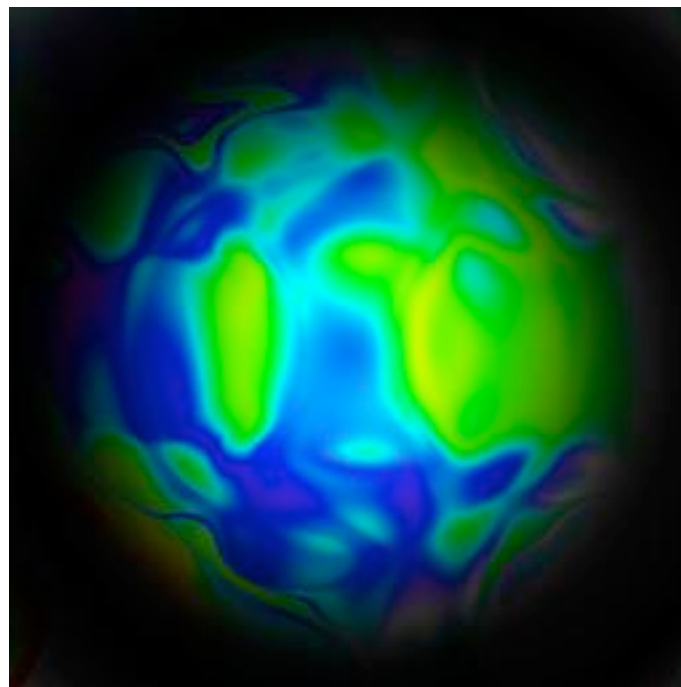
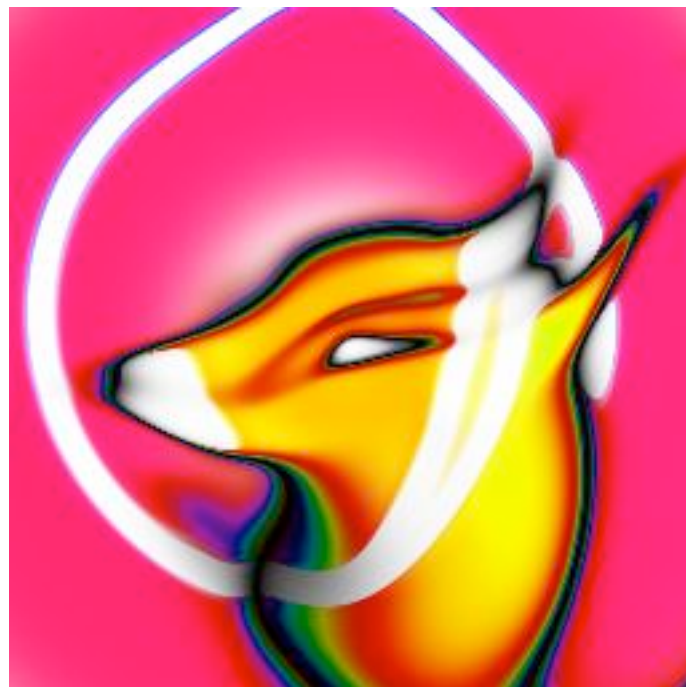
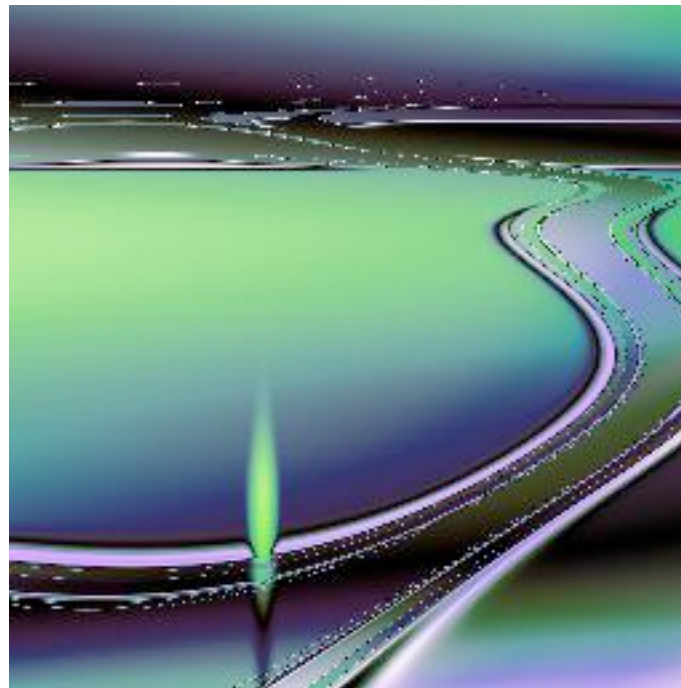
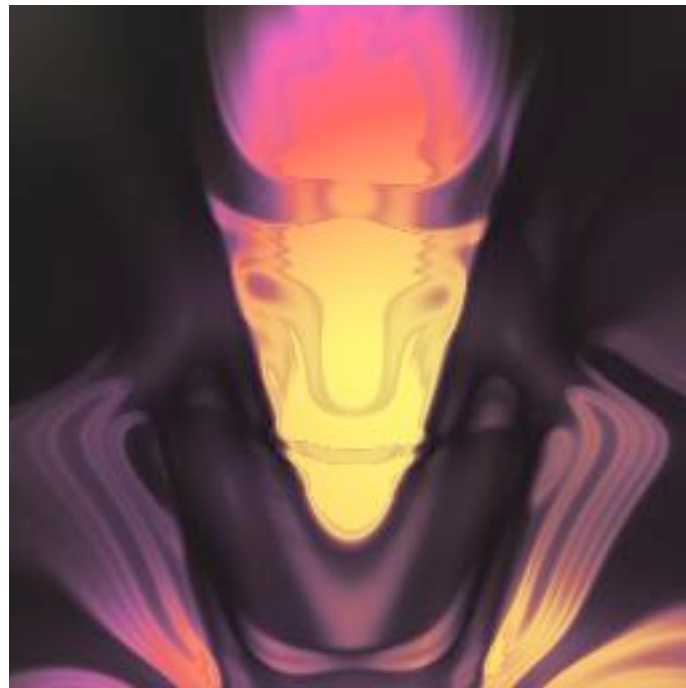
Continuous Pattern Producing Networks: CPPN



Stanley, K. O. (2007). Compositional pattern producing networks: A novel abstraction of development. *Genetic programming and evolvable machines*, 8(2), 131-162.

Examples of pictures generated with CPPNs

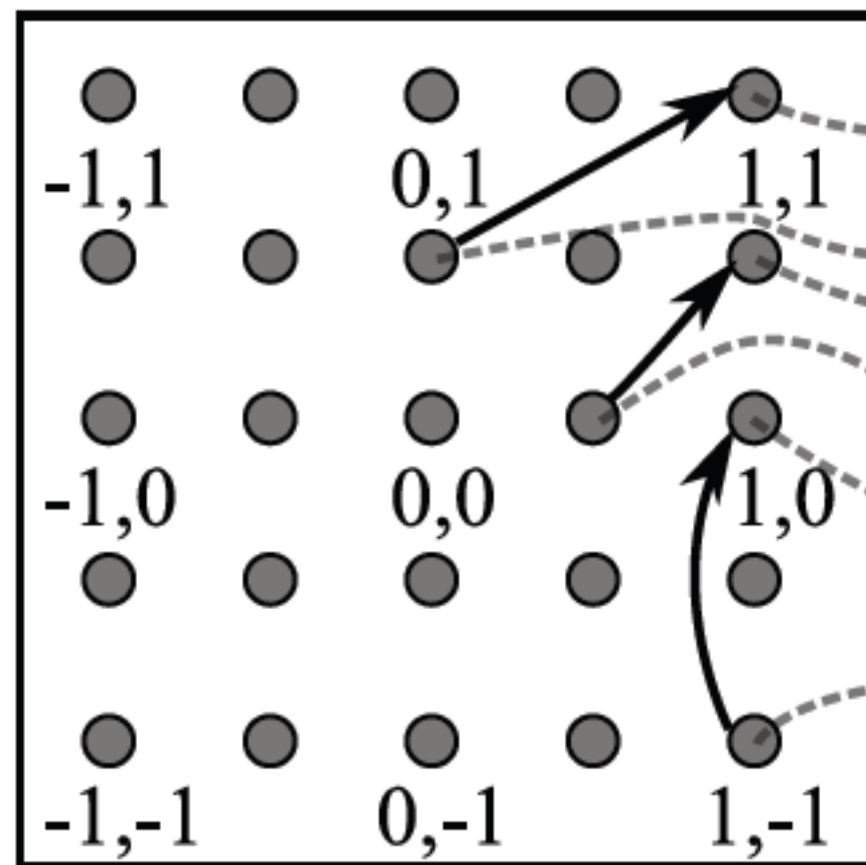
<http://picbreeder.org>



J. Secretan, N. Beato, D. B. D'Ambrosio, A. Rodriguez, A. Campbell, J. T. Folsom-Kovarik, and K. O. Stanley
(2011) **Picbreeder: A Case Study in Collaborative Evolutionary Exploration of Design Space**
In: Evolutionary Computation journal, MIT Press, 2011

HyperNEAT

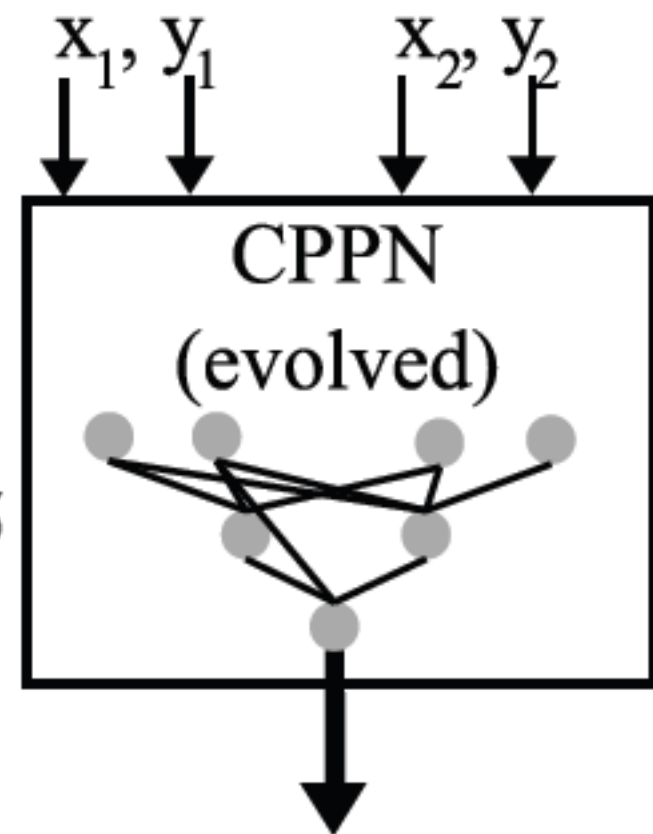
1) Query each potential connection on substrate



Substrate

$x_1, y_1 \rightarrow x_2, y_2$

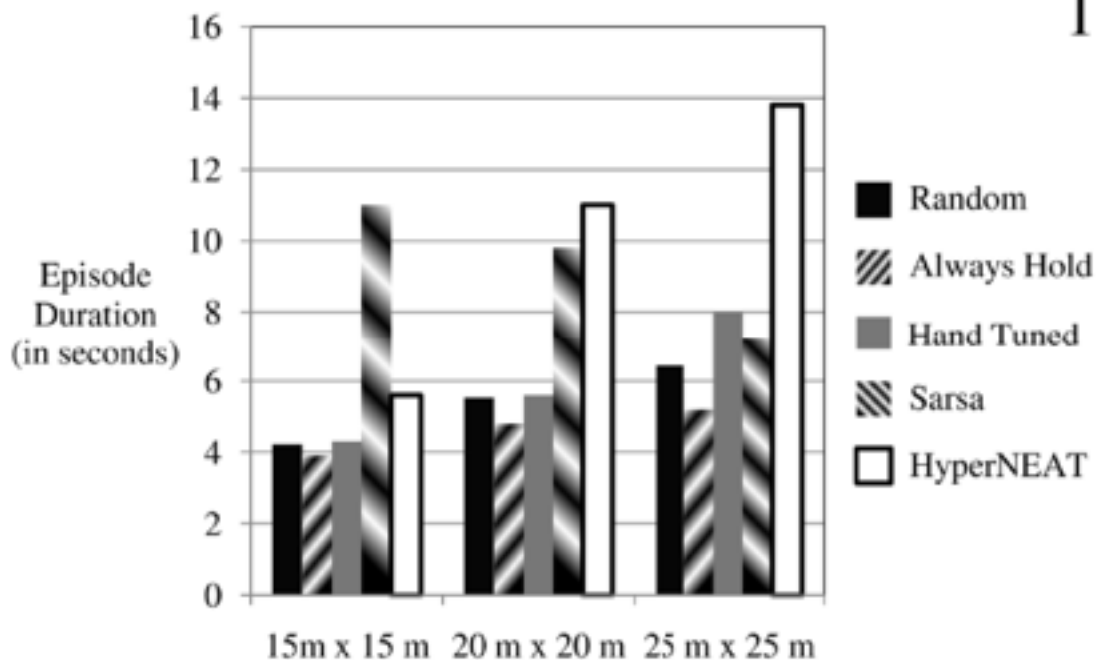
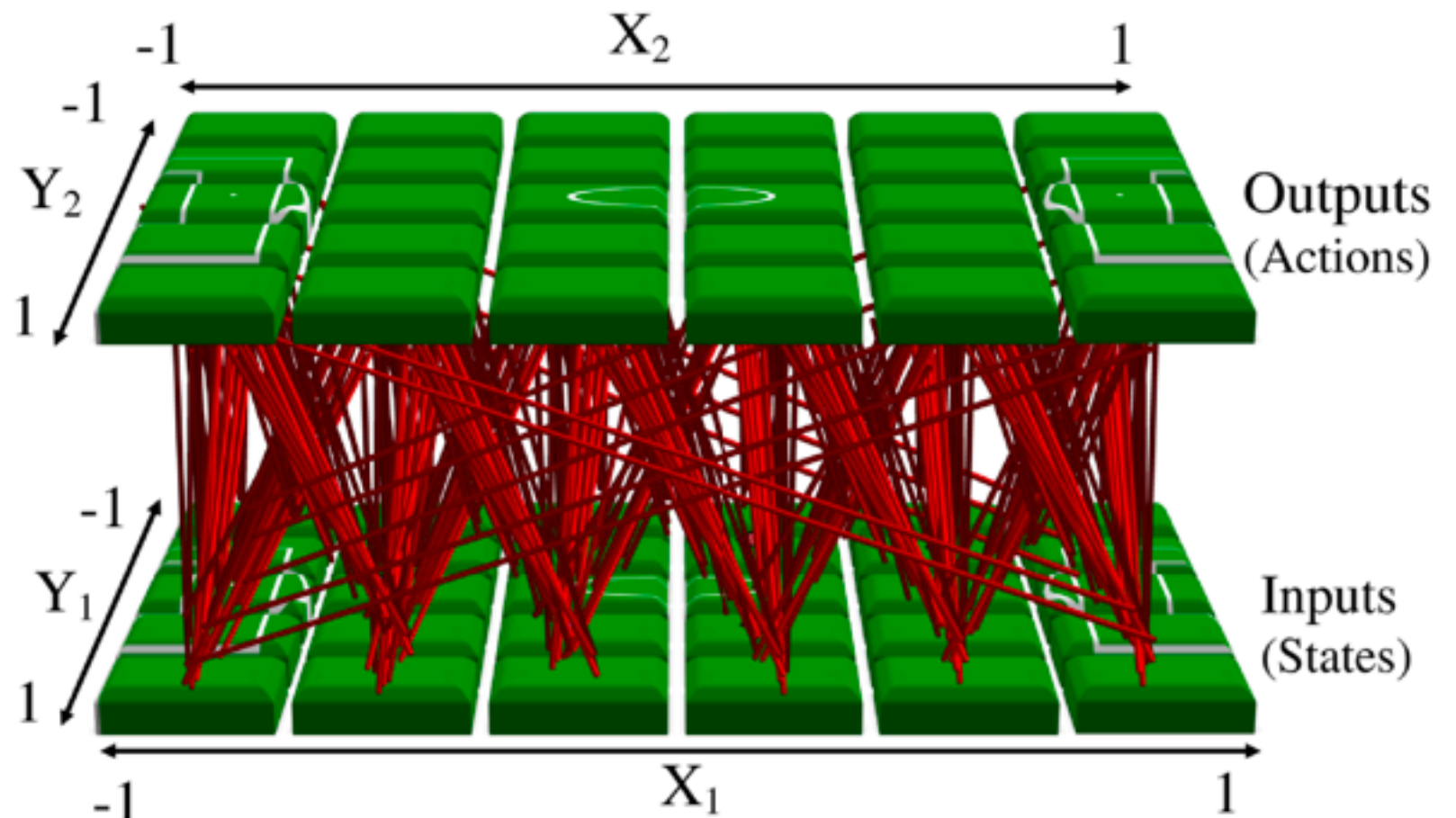
2) Feed each coordinate pair into CPPN



3) Set weight of connection between (x_1, y_1) and (x_2, y_2) to value of output

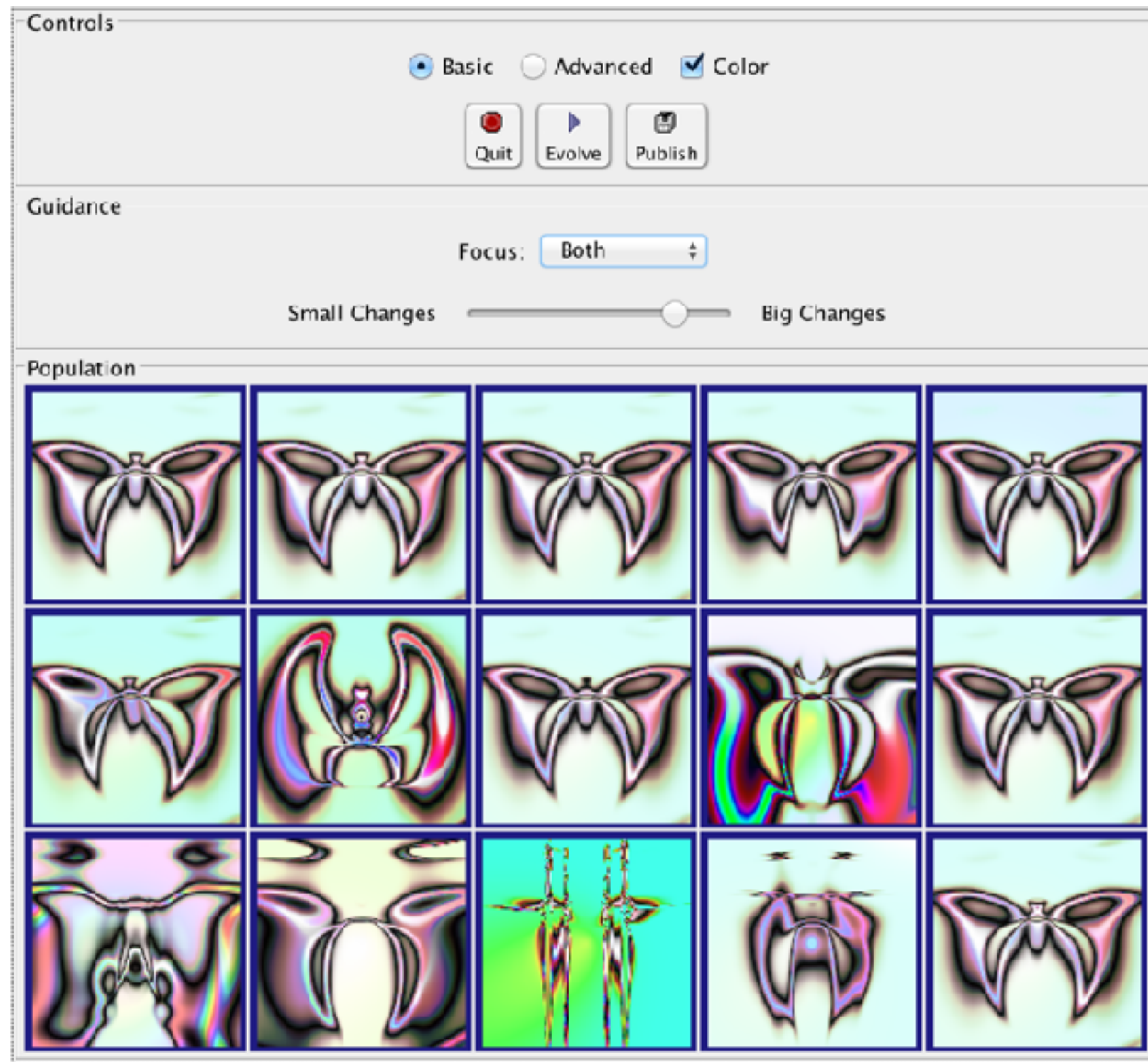
HyperNEAT: Bird's eye view

- Keepaway task
- Can easily transfer to larger field size with retraining



METHOD	AVERAGE HOLD TIME
HYPERNEAT BEV	15.4s
EANT	14.9s
NEAT	14.0s
SARSA	12.5s
HAND-TUNED BENCHMARK	8.3s
ALWAYS HOLD BENCHMARK	7.5s
RANDOM BENCHMARK	3.4s

Back to PicBreeder



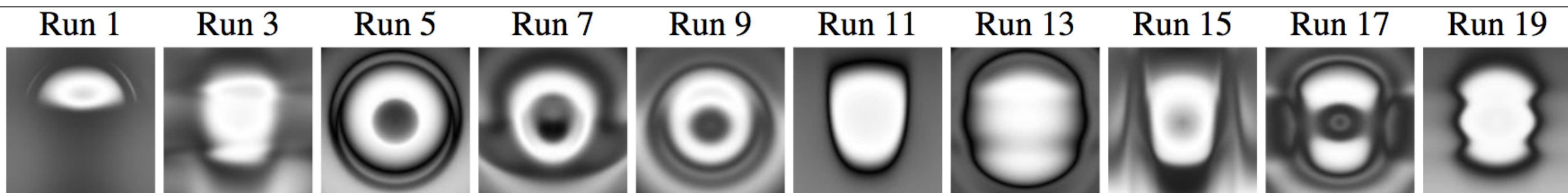
Secretan, J., Beato, N., D'Ambrosio, D. B., Rodriguez, A., Campbell, A., Folsom-Kovarik, J. T., & Stanley, K. O. (2011). **Picbreeder: A case study in collaborative evolutionary exploration of design space.** *Evolutionary Computation*, 19(3), 373-403.

Skull

Goal:



Results, after 30 000 generations with a population size of 150:

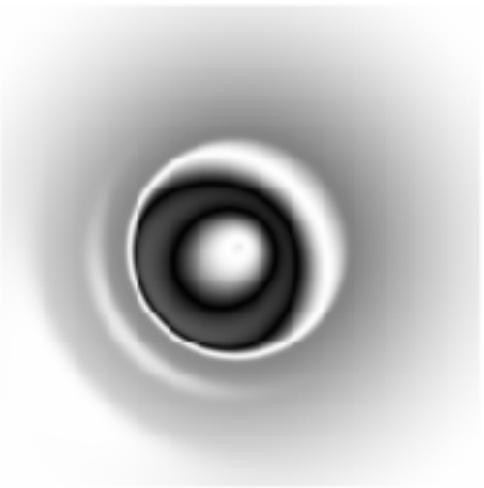


- Brian G Woolley and Kenneth O Stanley. **On the deleterious effects of a priori objectives on evolution and representation.** In Proceedings of the 13th annual conference on Genetic and evolutionary computation, pages 957–964. ACM, 2011.

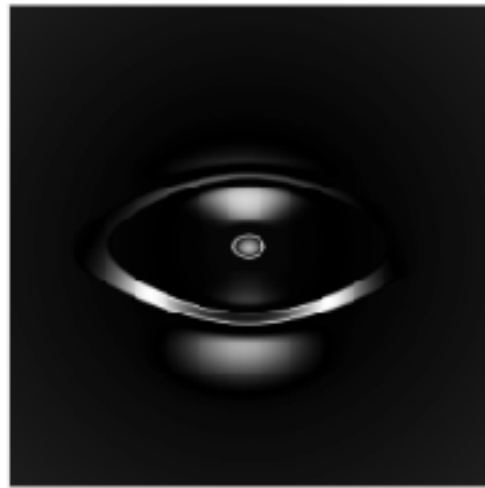
PicBreeder



gen 12



gen 20



gen 36



gen 49



gen 74

Population size = 15

- Brian G Woolley and Kenneth O Stanley. **On the deleterious effects of a priori objectives on evolution and representation.** In Proceedings of the 13th annual conference on Genetic and evolutionary computation, pages 957–964. ACM, 2011.

Quiz

- What are the main features of NEAT ?
 - ☐ Representation of feed-forward networks only
 - ☐ Direct representation of the network
 - ☐ Modular representation
 - ☐ Cross-over included
- Can it be used directly with any evolutionary algorithm (e.g. NSGA-II) ?
 - ☐ Yes
 - ☐ No

Overview

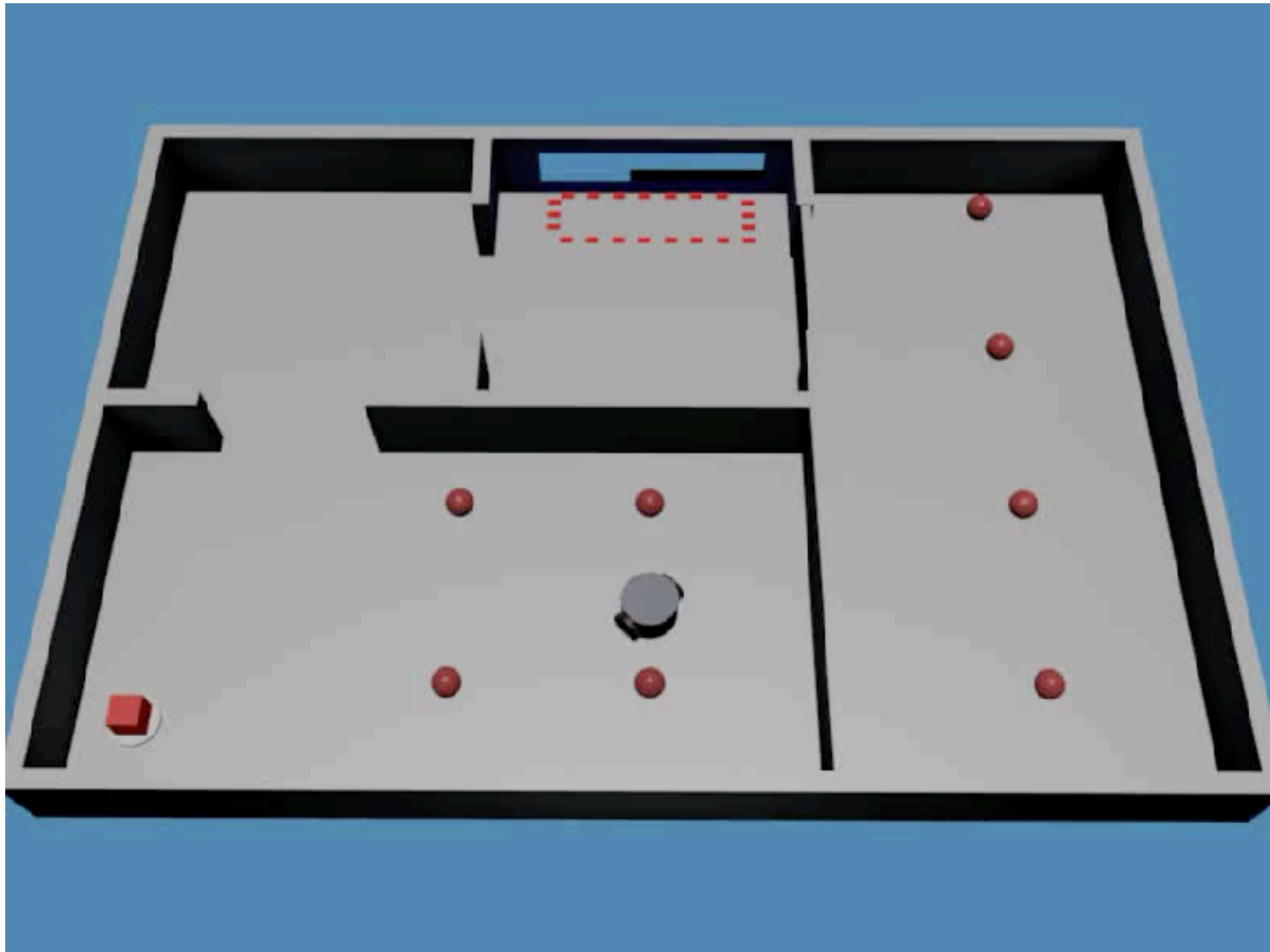


- Evolving Morphologies
- Genetic Programming
- Neuroevolution
- Evolutionary algorithms & selective pressures
- Neuroevolution/GP in Deep learning era
- Building you own representation

Some challenges to deal with when evolving structures

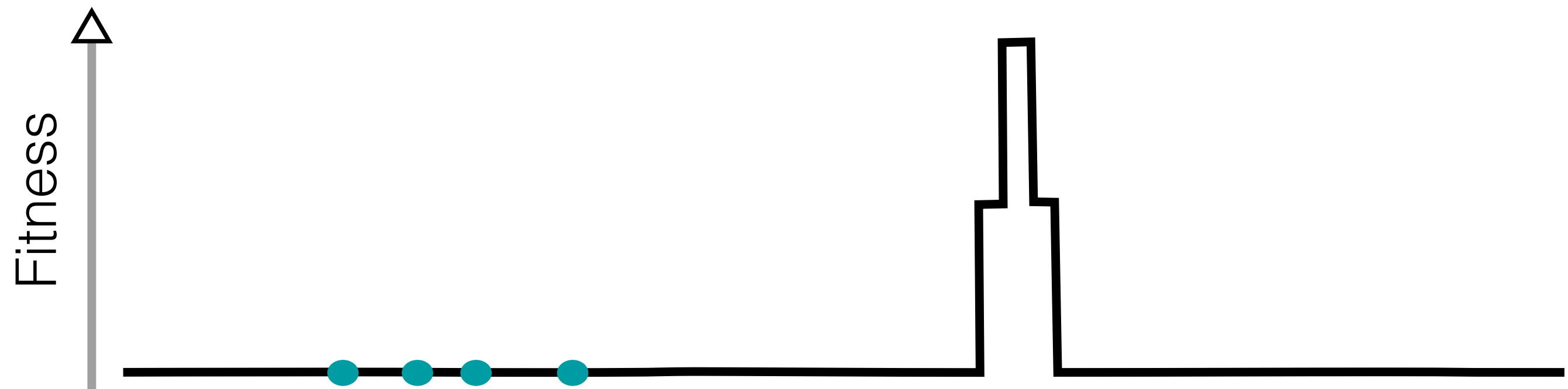
- premature convergence / protecting innovation
- bloat (GP)
- generating neural networks with:
 - a modular structure
 - memory
- **An approach to deal with those challenges: multi-objectivisation**
 - Question: what objective to add for each challenge ?

Premature convergence:
how to push towards exploration?



<https://github.com/doncieux/collectball>

The premature convergence challenge

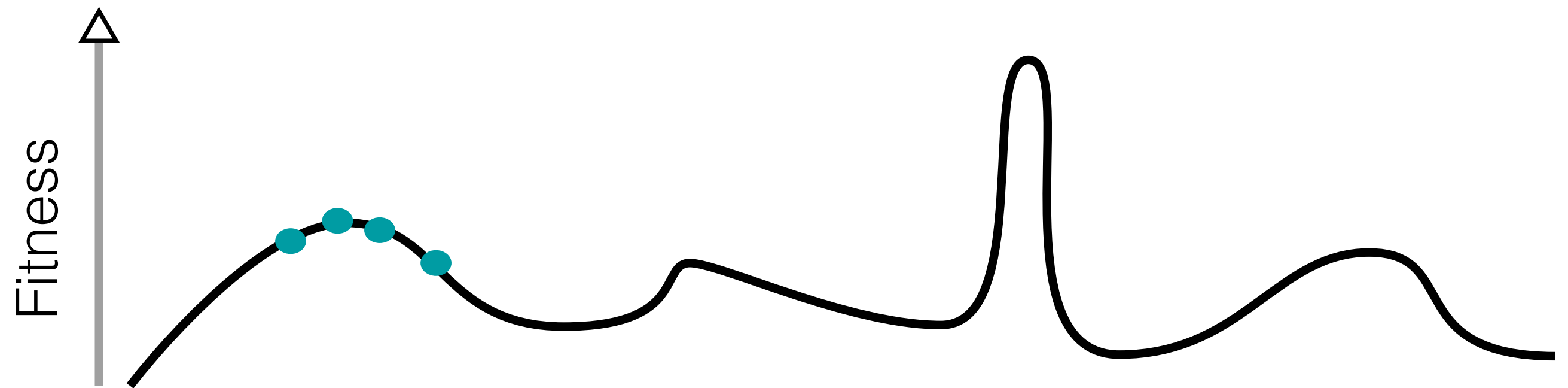


The fitness function has two roles:

- It defines the goal
- It drives the search



The premature convergence challenge



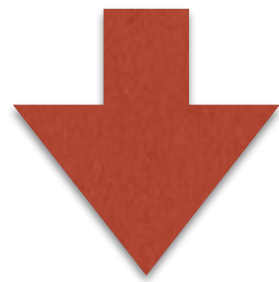
The fitness function has two roles:

- It defines the goal
- It drives the search

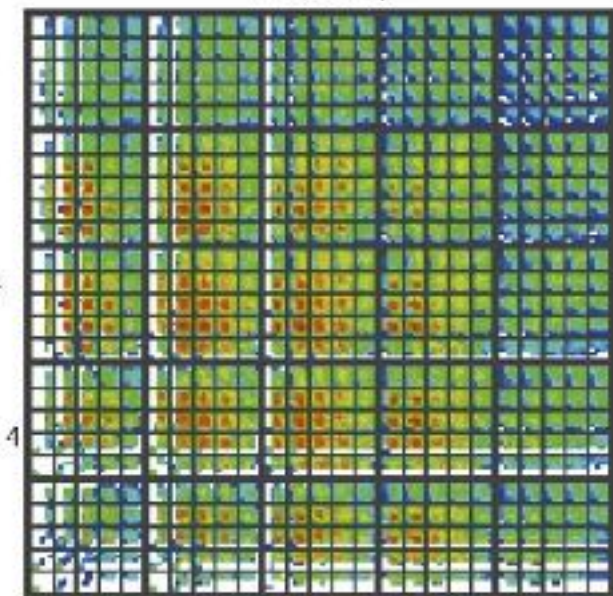
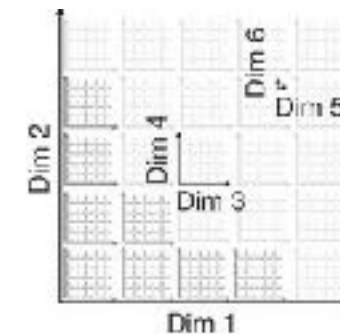
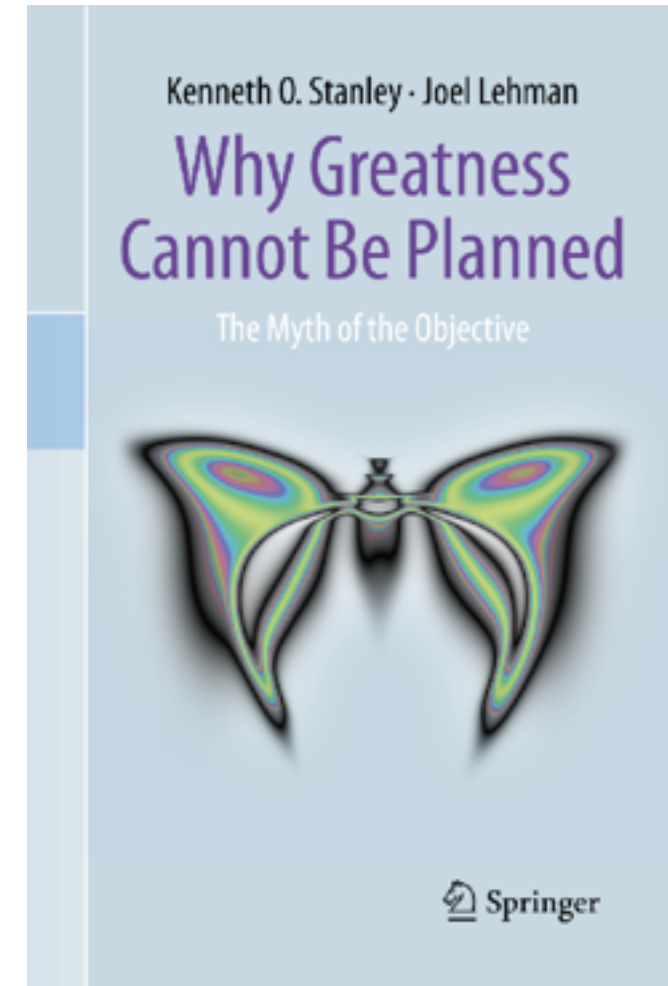


Premature convergence: how to push towards exploration?

Looking for **the**
optimal solution



Looking for novel
or original solutions

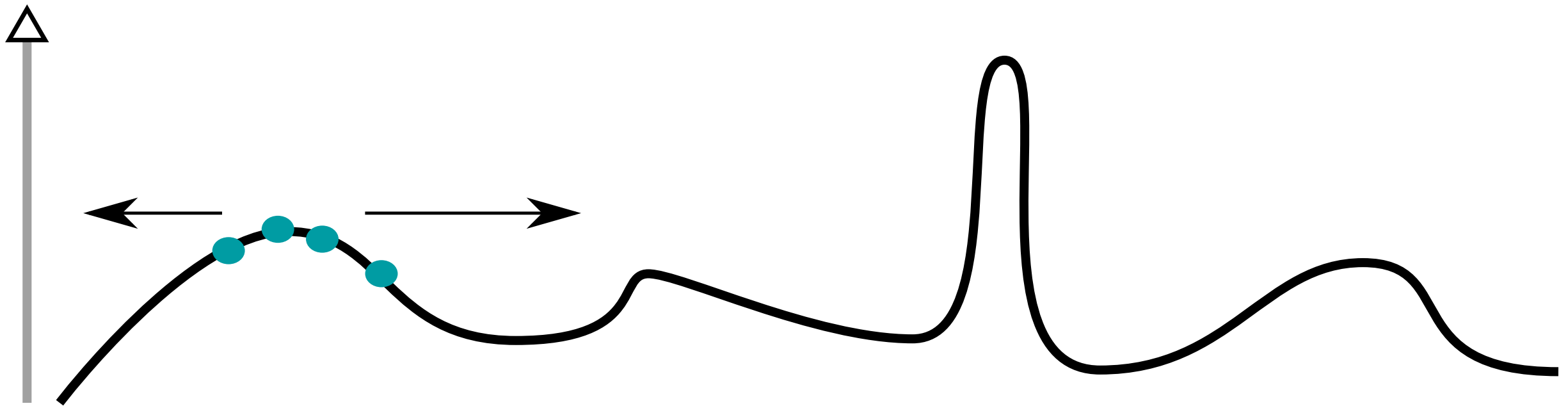


Joel Lehman and Kenneth O. Stanley (2011) **Abandoning Objectives: Evolution Through the Search for Novelty** Alone In: Evolutionary Computation journal (19):2, pages 189-223, Cambridge, MA: MIT Press, 2011

Mouret, J. B., & Clune, J. (2015). **Illuminating search spaces by mapping elites.** arXiv:1504.04909.

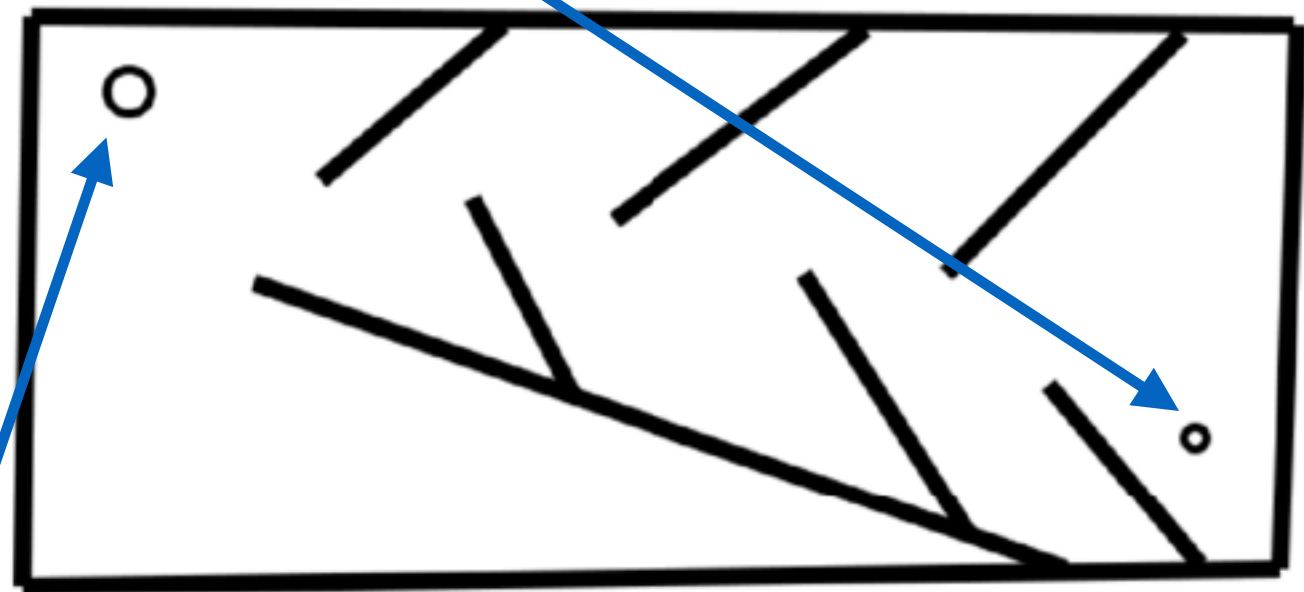
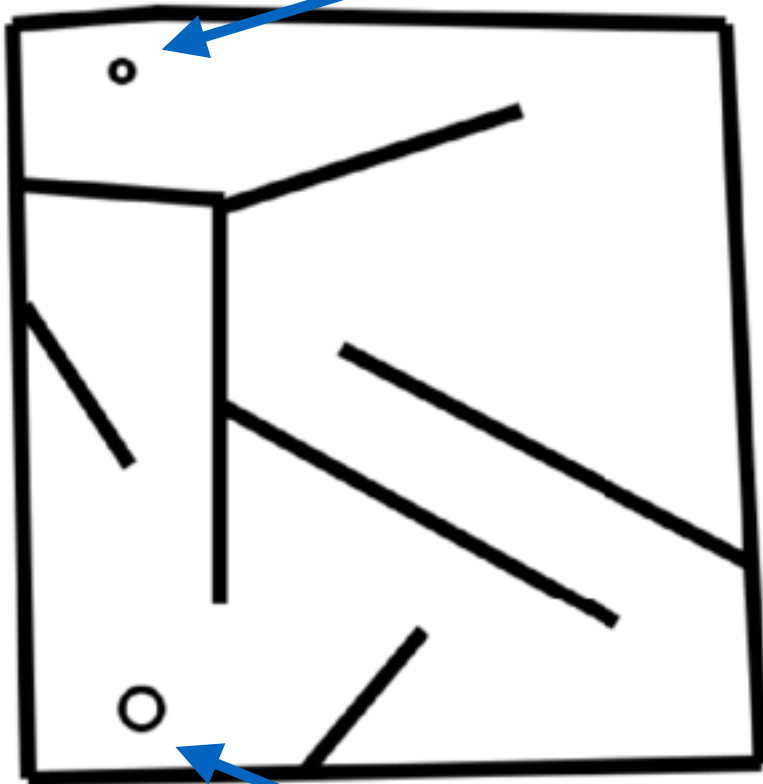
Solution:

add « process helpers »



A **process helper** intends to increase the efficiency of the search process without changing the optimum(s) of the fitness function.

Exit



Starting point

Goal-based approach

- Goal-oriented fitness: $f(x) = \textit{dist}(x, x_e)$

Novelty search

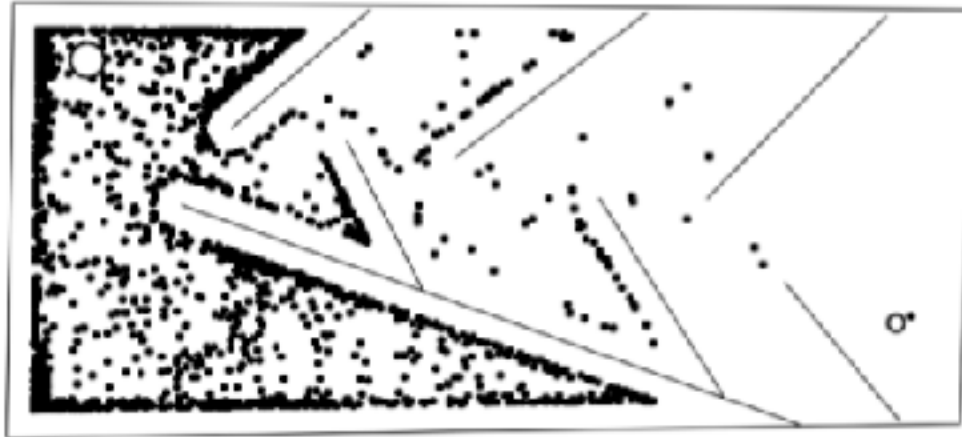
- ~~Goal-oriented fitness: $f(x) = dist(x, x_e)$~~

- Novelty based fitness: $\rho(x) = \frac{1}{k} \sum_{i=0}^k dist(x, \mu_i)$

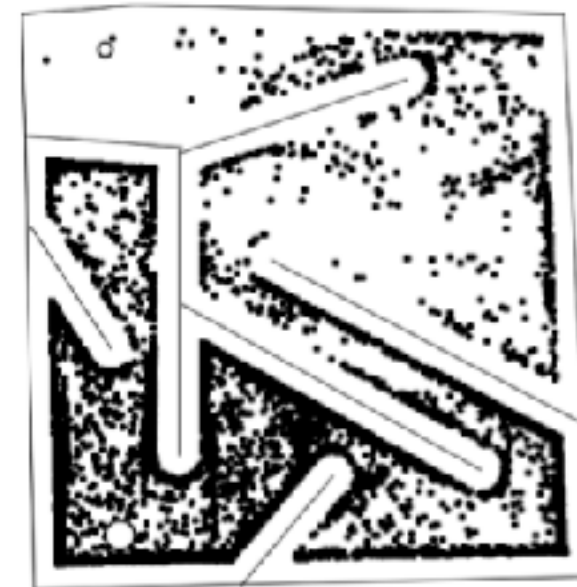
$\{\mu_0, \dots, \mu_{k-1}\}$ are the k-nearest neighbors in pop+archive

- Archive augmented with individuals having a high novelty
- Use a neuroevolution method with an increasing network complexity (NEAT)

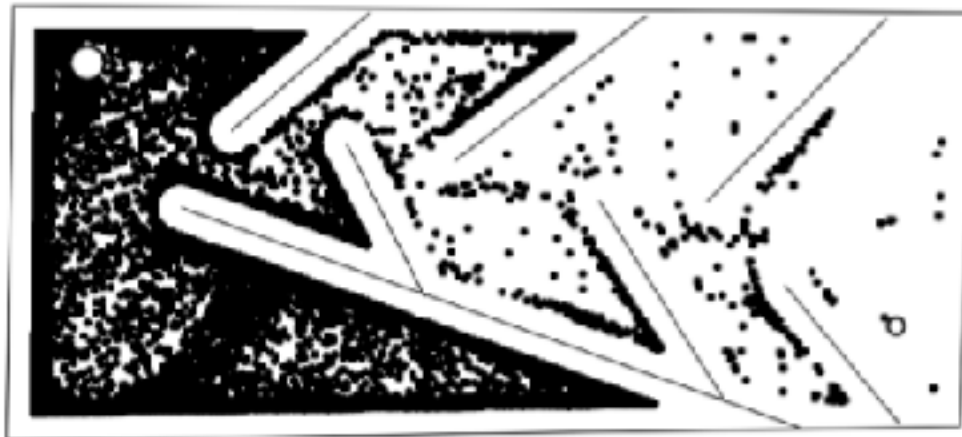
Novelty search



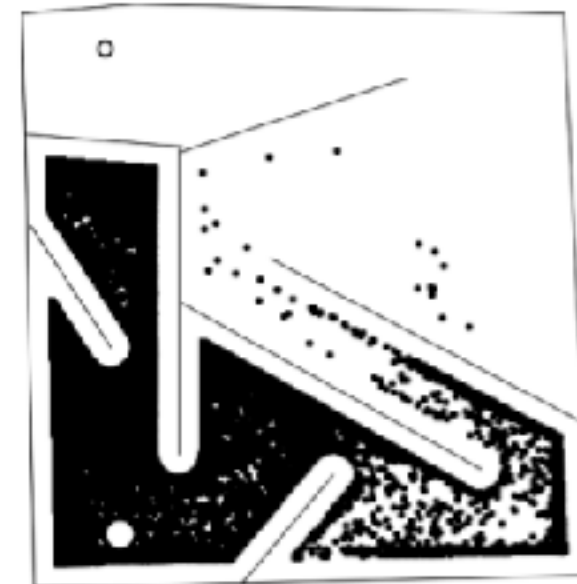
(a) Medium Map Novelty



(b) Hard Map Novelty



(c) Medium Map Fitness



(d) Hard Map Fitness

Novelty Search

$$\rho(x) = \frac{1}{k} \sum_{i=0}^k \text{dist}(x, \mu_i)$$

- Some recommendations:
 - k=15 gives good and robust results
 - Archive type:
 1. adding the λ most novel individuals

Novelty Search

$$\rho(x) = \frac{1}{k} \sum_{i=0}^k \text{dist}(x, \mu_i)$$

- Some recommendations:
 - k=15 gives good and robust results
 - Archive type:
 1. adding the λ most novel individuals
 2. **adding λ random individuals**

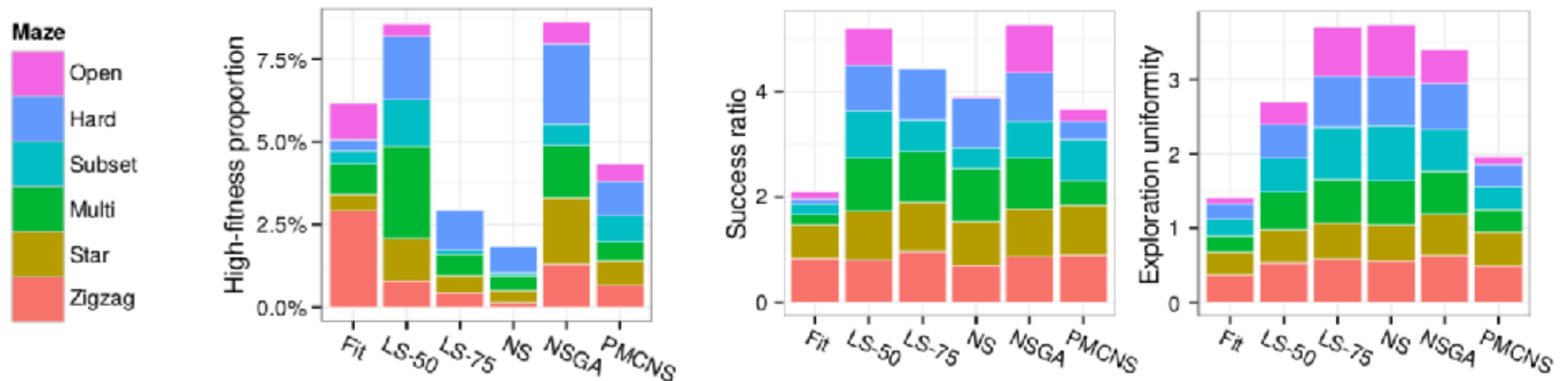
Novelty Search

$$\rho(x) = \frac{1}{k} \sum_{i=0}^k \text{dist}(x, \mu_i)$$

- Some recommendations:
 - k=15 gives good and robust results
 - Archive type:
 1. adding the λ most novel individuals
 2. **adding λ random individuals**
- (NS robust to different λ values)

Novelty as a process helper

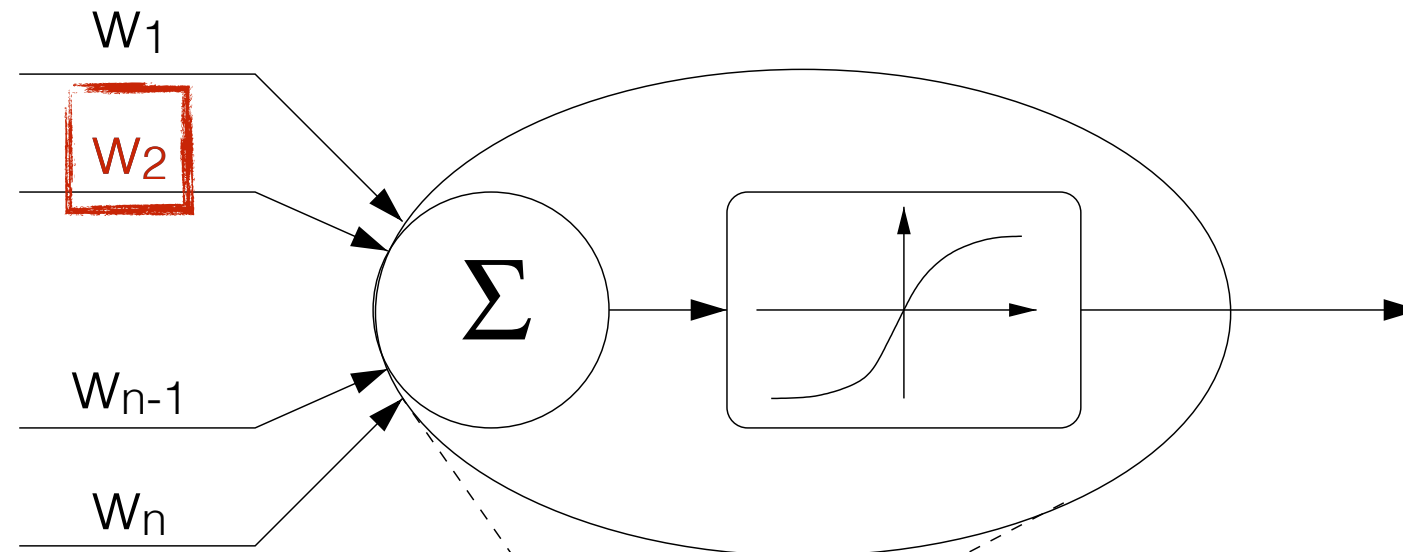
- The novelty criterion can be combined with a fitness criterion ... and it is a good idea to do it with NSGA-II



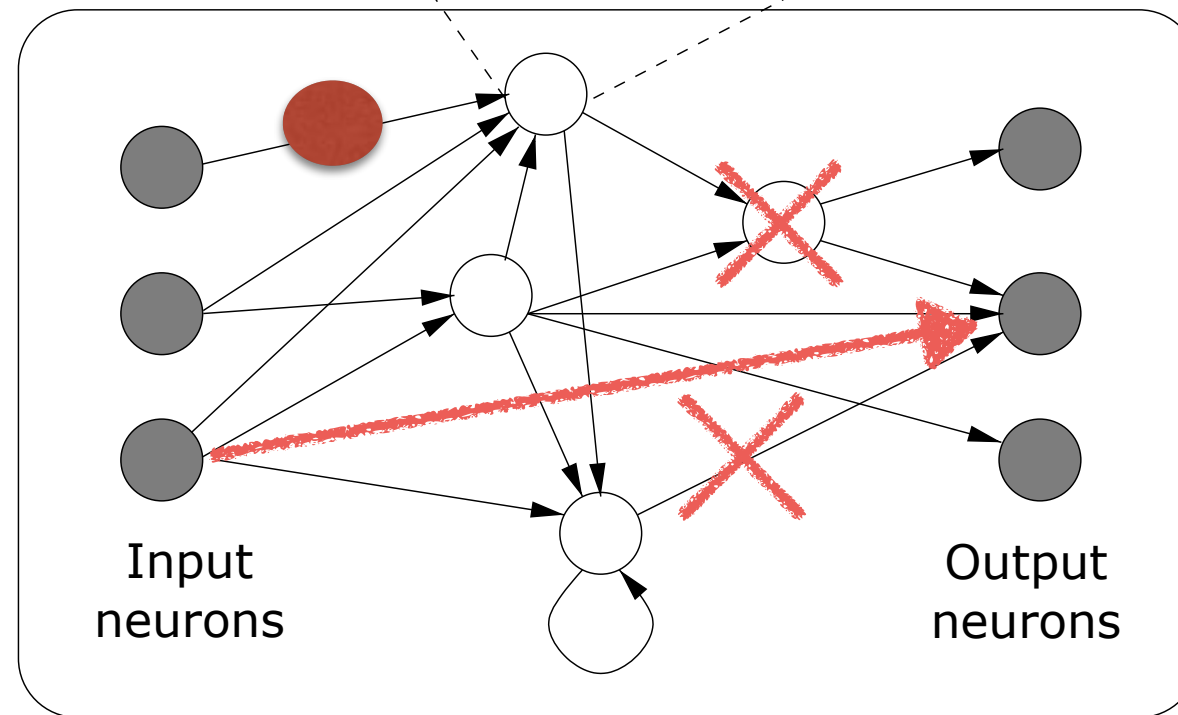
Novelty as a process helper

DNN: simplifying NEAT

Neuron



Mutations



Crossover

Neural network

Novelty as a process helper

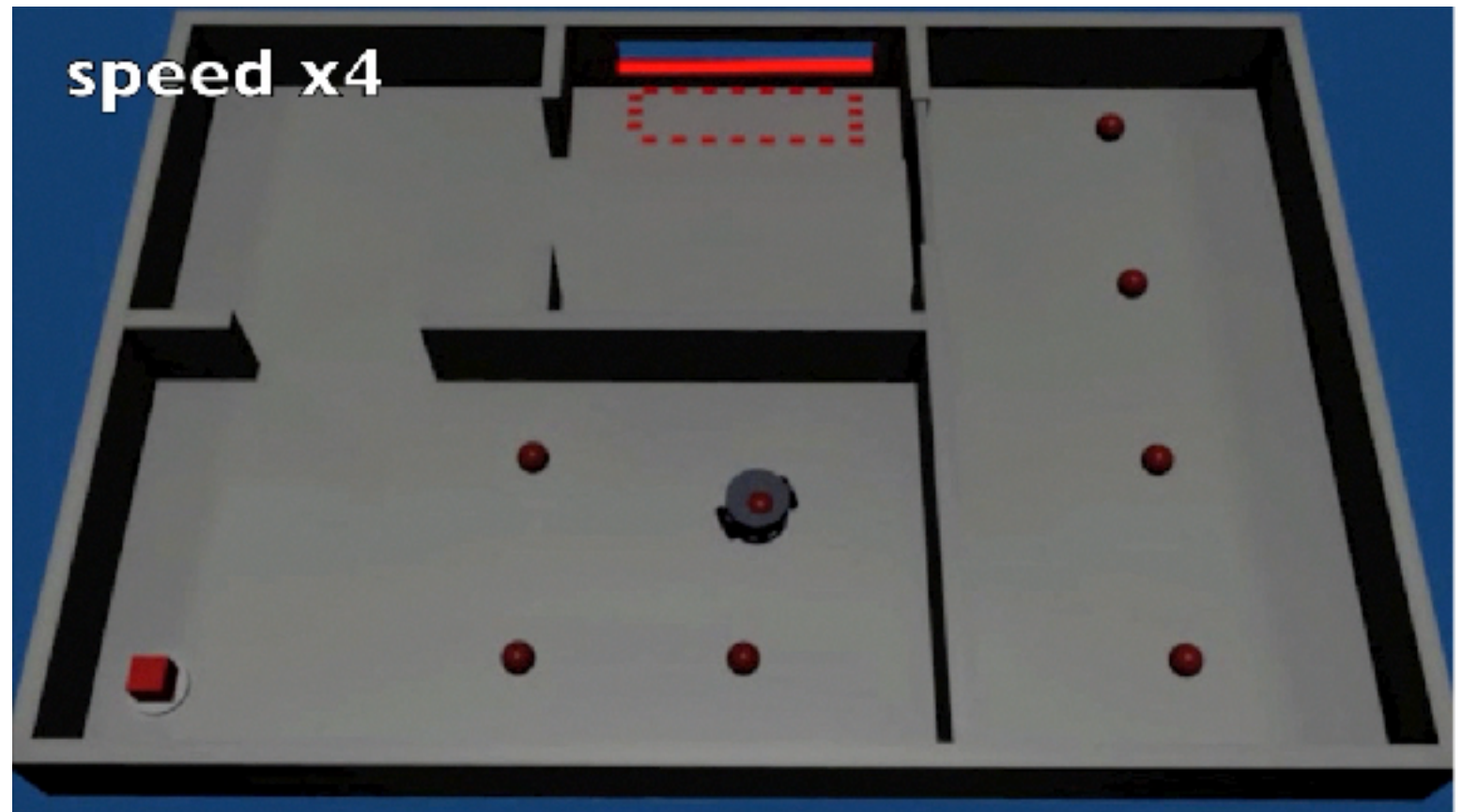
DNN: simplifying NEAT

Fitness objectives:

1. nb_{ball}
2. Behavior diversity
(archive-free novelty)

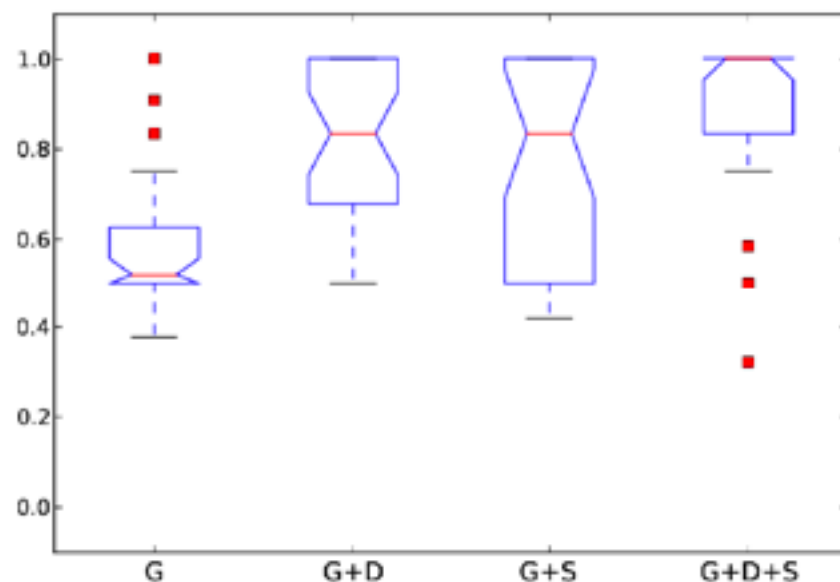
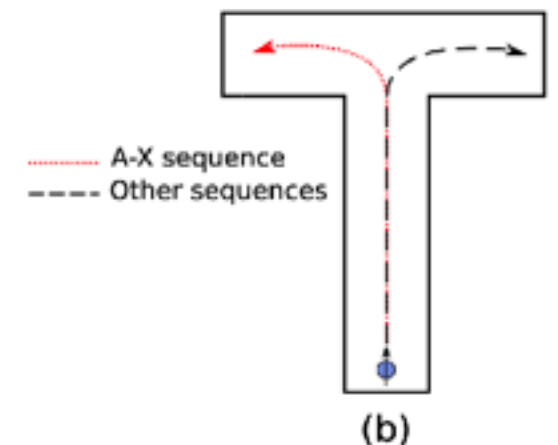
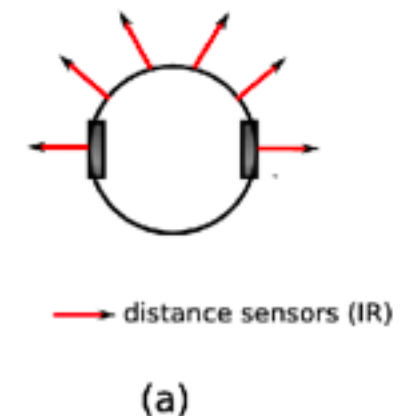
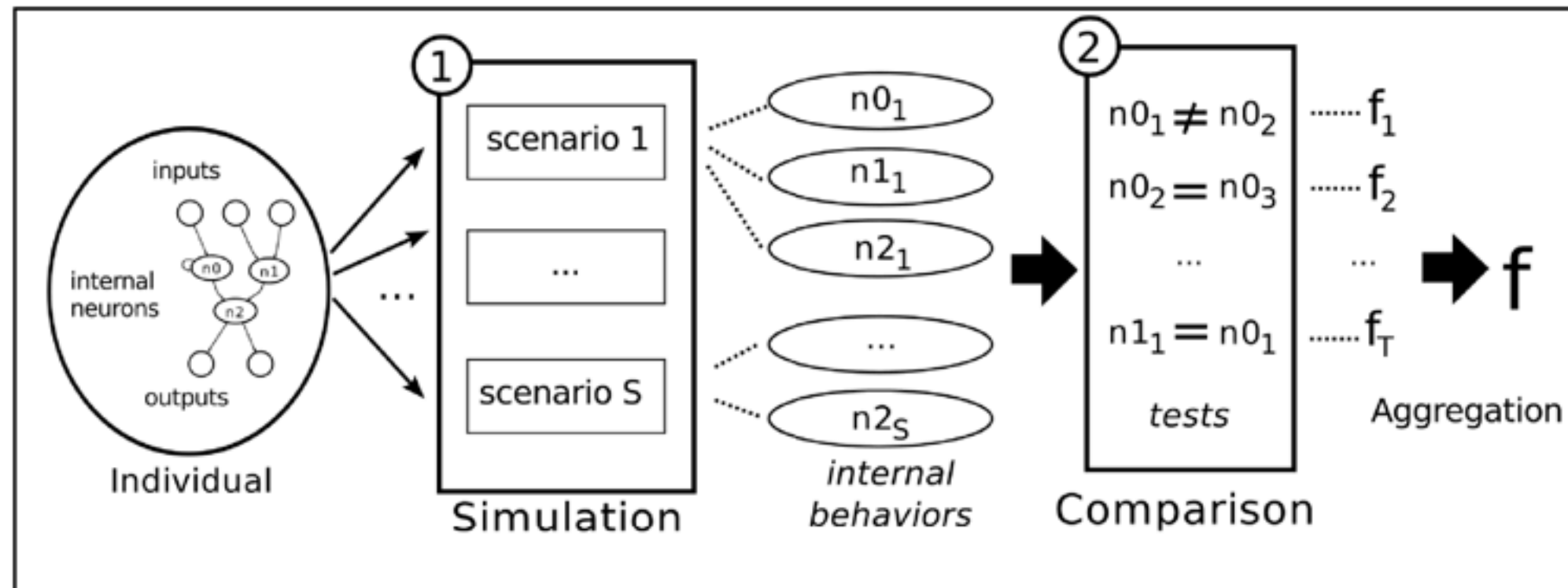
Multi-objective EA:
NSGA-II

Neuroevolution



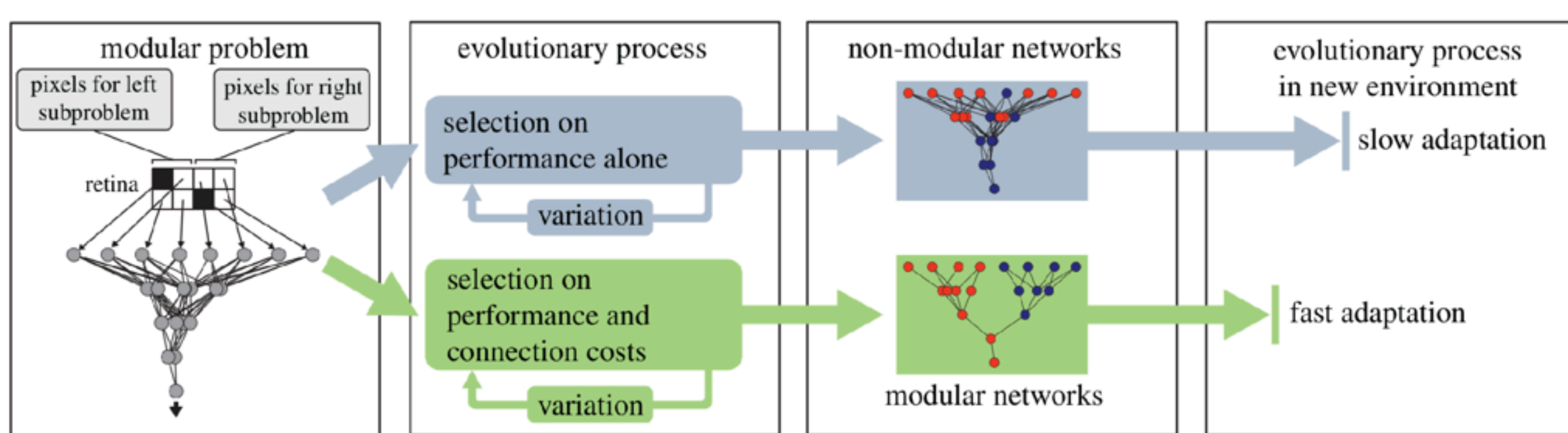
<https://github.com/doncieux/collectball>

Architecture search & Selective pressures: rewarding consistent behaviors

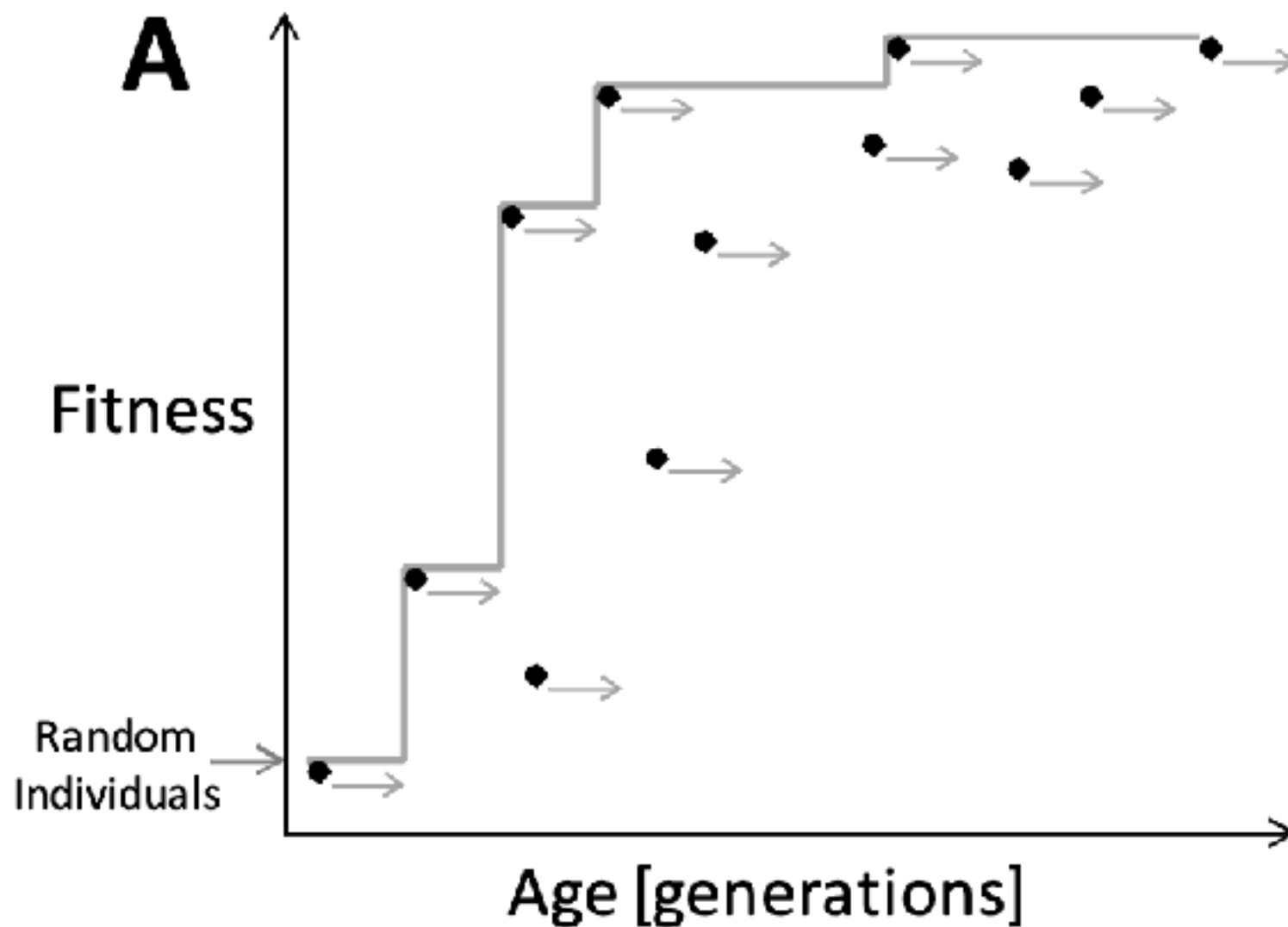


	Setup	Description
1	G	Goal-oriented
2	G + D	Goal-oriented + Diversity
3	G + S	Goal-oriented + Scenario-based
4	G + D + S	Goal-oriented + Diversity + Scenario-based

Adding a connection cost to push towards modular networks



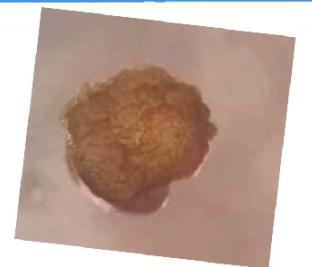
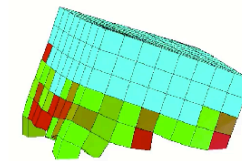
Architecture search & Selective pressures



Used in:

A scalable pipeline for
designing reconfigurable organisms.

Sam Kriegman, Douglas Blackiston, Michael Levin, Josh Bongard
University of Vermont, Tufts University.



Quiz

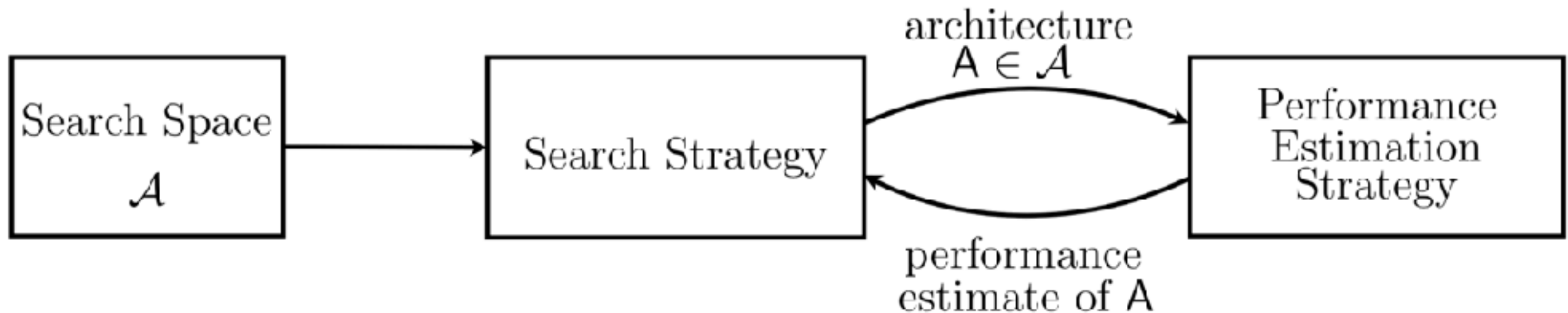
- What is the optimal solution in Novelty Search ?

Overview



- Evolving Morphologies
- Genetic Programming
- Neuroevolution
- Evolutionary algorithms & selective pressures
- Neuroevolution/GP in Deep learning era
- Building you own representation

Neural Architecture Search

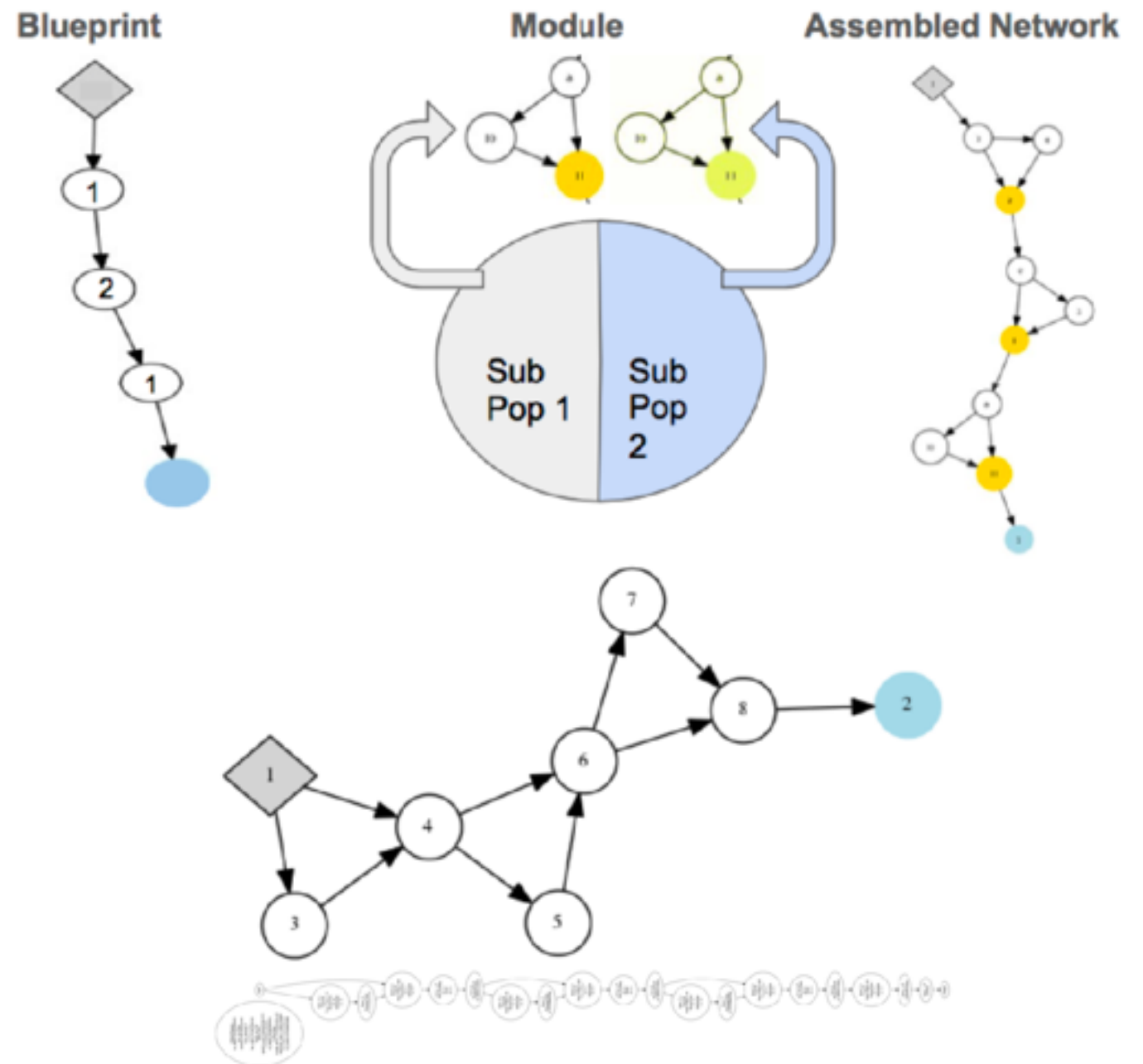


- Many different search strategies:
 - Random search
 - Bayesian optimisation
 - Reinforcement learning
 - Gradient based
 - Evolutionary methods

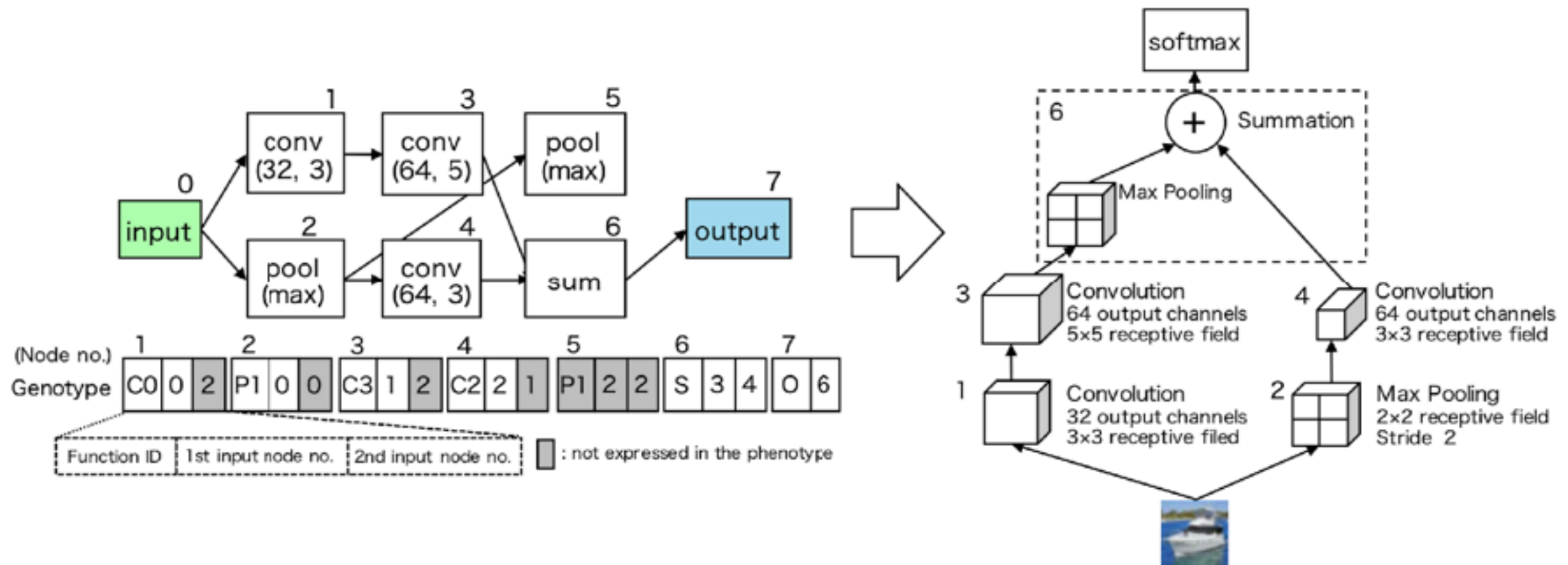
Warning: random search is not that bad, cf Li, L., & Talwalkar, A. (2020, August). Random search and reproducibility for neural architecture search. In *Uncertainty in Artificial Intelligence* (pp. 367-377). PMLR.

Neuroevolution in deep learning era: CoDeepNeat

- NEAT -> DeepNEAT:
 - each node represents a layer
 - Gaussian or bit-flip mutation
- DeepNeat -> CoDeepNEAT
 - SANE/ESP like approach
- Performance: error 7.3% on CIFAR-10 (6% human error rate)



Neuroevolution in deep learning era: CNN with CGP



- Evolving Convolutional Networks with Cartesian GP
 - Structure: Cartesian GP
 - Weight: backpropagation
 - 1+lambda evolution strategy

Model	Error rate	# params ($\times 10^6$)
Maxout [7]	9.38	–
Network in Network [19]	8.81	–
VGG [29] ¹	7.94	15.2
ResNet [10]	6.61	1.7
MetaQNN [1] ²	9.09	3.7
Neural Architecture Search [41]	3.65	37.4
CGP-CNN (ConvSet)	6.75	1.52
CGP-CNN (ResSet)	5.98	1.68

CIFAR - 10

Suganuma, M., Shirakawa, S., & Nagao, T. (2017, July). A genetic programming approach to designing convolutional neural network architectures. In *Proceedings of the Genetic and Evolutionary Computation Conference* (pp. 497-504).

Neuroevolution in deep learning era: Google Brain's approach

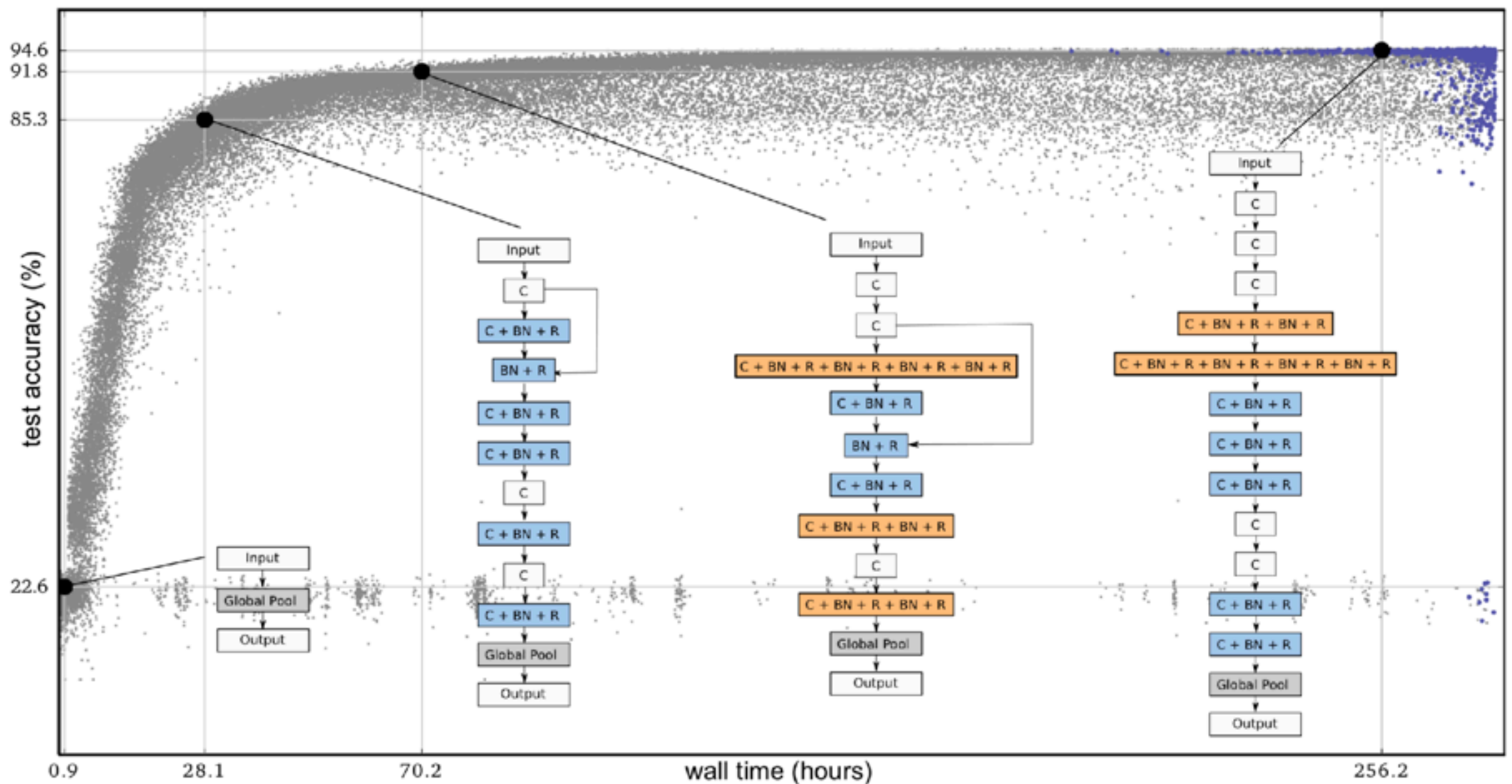
- Tournament-based selection on a massively parallel architecture
- « Start simple » principle
- Learned weights inheritance
- No crossover
- Pop size up to 1000
- Mutation, randomly select one among:
 - alter learning rate
 - identity
 - reset weights
 - insert convolution
 - remove convolution
 - alter stride
 - alter number of channels
 - filter size
 - insert one-to-one
 - add skip
 - remove skip

Neuroevolution in deep learning era: Google Brain's approach

STUDY	STARTING POINT	CONSTRAINTS	POST-PROCESSING	PARAMS.	C10+	C100+
BAYESIAN (SNOEK ET AL., 2012)	3 LAYERS	FIXED ARCHITECTURE, NO SKIPS	NONE	–	90.5%	–
Q-LEARNING (BAKER ET AL., 2016)	–	DISCRETE PARAMS., MAX. NUM. LAYERS, NO SKIPS	TUNE, RETRAIN	11.2 M	93.1%	72.9%
RL (ZOPH & LE, 2016)	20 LAYERS, 50% SKIPS	DISCRETE PARAMS., EXACTLY 20 LAYERS	SMALL GRID SEARCH, RETRAIN	2.5 M	94.0%	–
RL (ZOPH & LE, 2016)	39 LAYERS, 2 POOL LAYERS AT 13 AND 26, 50% SKIPS	DISCRETE PARAMS., EXACTLY 39 LAYERS, 2 POOL LAYERS AT 13 AND 26	ADD MORE FILTERS, SMALL GRID SEARCH, RETRAIN	37.0 M	96.4%	–
EVOLUTION (OURS)	SINGLE LAYER, ZERO CONVS.	POWER-OF-2 STRIDES	NONE	5.4 M 40.4 M ENSEMB.	94.6% 95.6%	77.0%

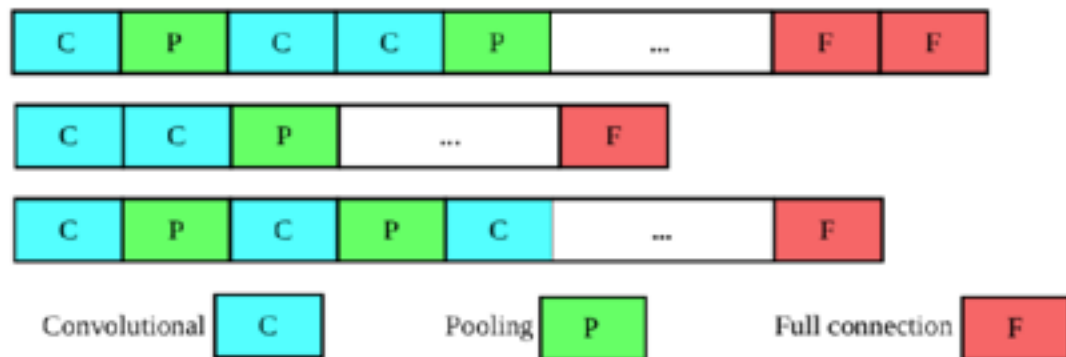
Real, E., Moore, S., Selle, A., Saxena, S., Suematsu, Y. L., Tan, J., ... & Kurakin, A. (2017, August). Large-scale evolution of image classifiers. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70* (pp. 2902-2911). JMLR. org.

Neuroevolution in deep learning era: Google Brain's approach



Real, E., Moore, S., Selle, A., Saxena, S., Suematsu, Y. L., Tan, J., ... & Kurakin, A. (2017, August). Large-scale evolution of image classifiers. In *Proceedings of the 34th International Conference on Machine Learning-Volume 70* (pp. 2902-2911). JMLR. org.

Neuroevolution in deep learning era: EvoCNN

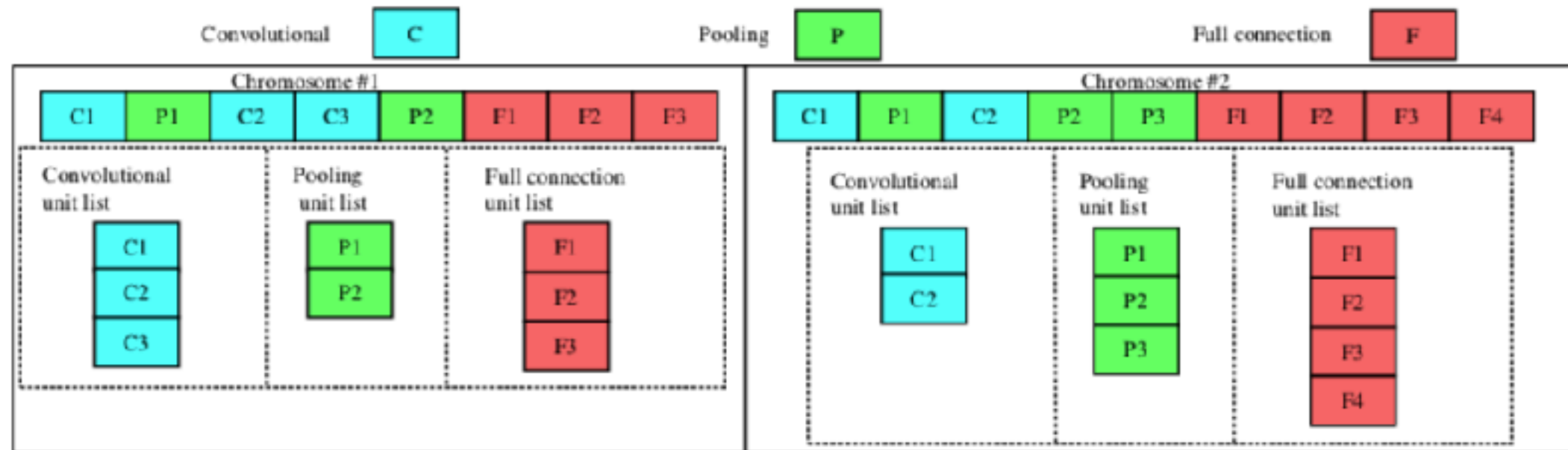


- Mutation: add/remove/modify + polynomial for real parameters
- Crossover: SBX for parameters, see next slide for structure
- Tournament selection

THE CLASSIFICATION ERRORS OF THE PROPOSED EvoCNN METHOD AGAINST THE PEER COMPETITORS ON THE FASHION BENCHMARK DATASET.

classifier	error(%)	# parameters	# epochs
2C1P2F+Drouout	8.40(+)	3.27M	300
2C1P	7.50(+)	100K	30
3C2F	9.30(+)	—	—
3C1P2F+Dropout	7.40(+)	7.14M	150
GRU+SVM+Dropout	10.30(+)	—	100
GoogleNet [41]	6.30(+)	101M	—
AlexNet [3]	10.10(+)	60M	—
SqueezeNet-200 [51]	10.00(+)	500K	200
MLP 256-128-64	10.00(+)	41K	25
VGG16 [52]	6.50(+)	26M	200
EvoCNN (best)	5.47	6.68M	100
EvoCNN (mean)	7.28	6.52M	100

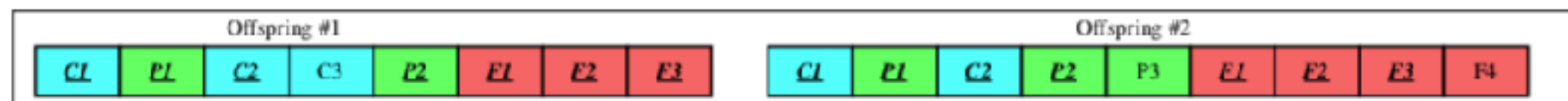
Neuroevolution in deep learning era: EvoCNN



(a) Unit Collection

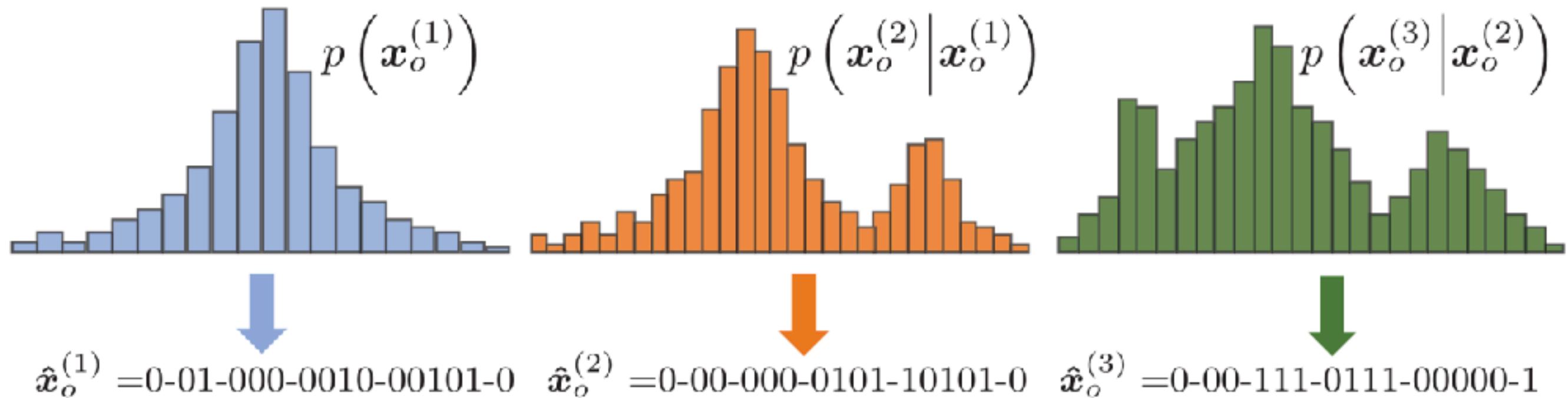
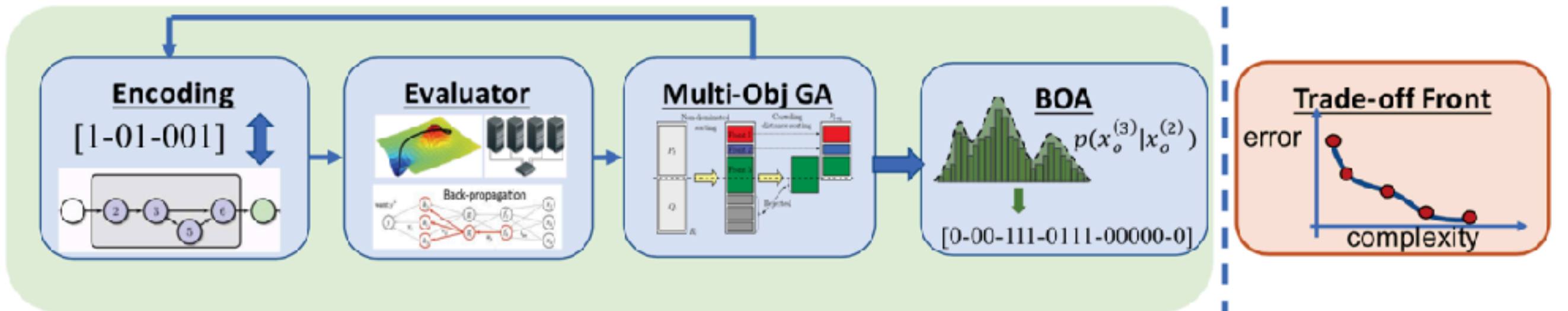


(b) Unit Align and Crossover

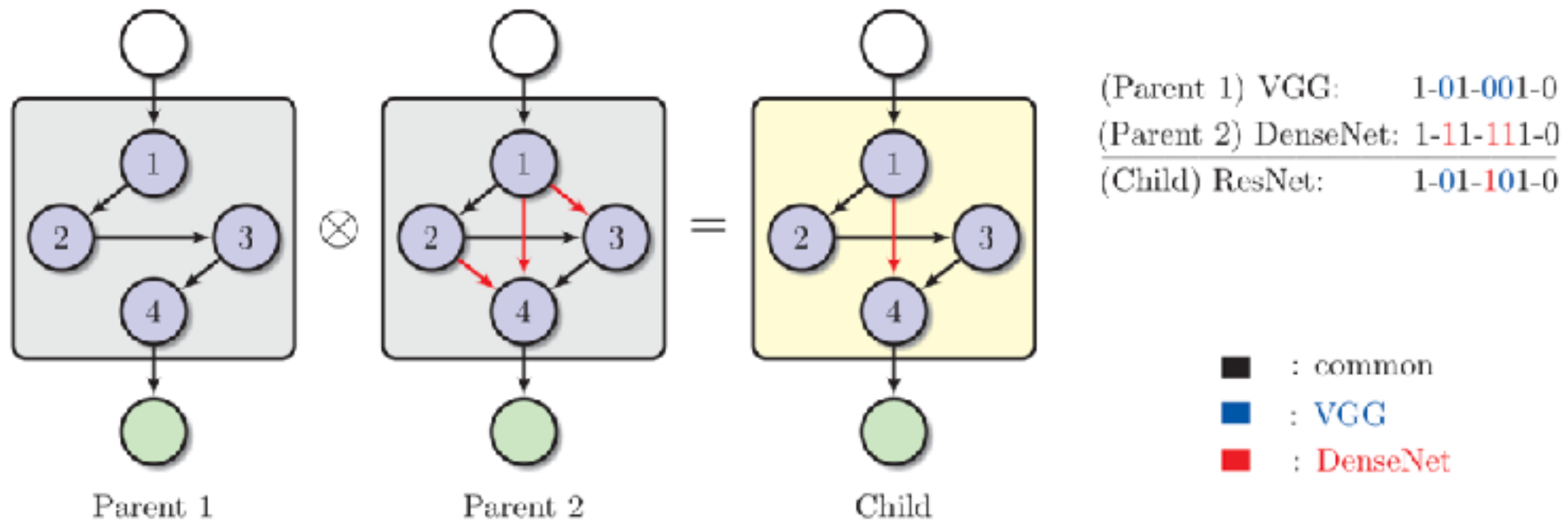
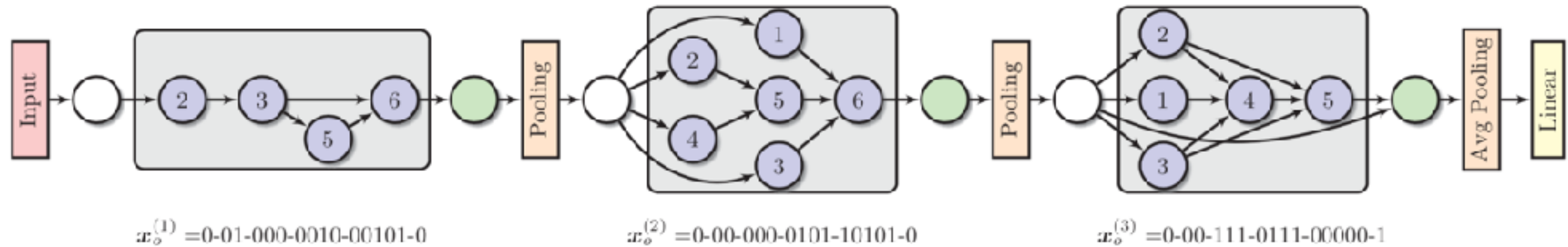


(c) Unit Restore

NSGA-Net



NSGA-Net



NSGA-Net

CIFAR 10

Architectures	Params (M)	Test Error (%)	$\times +$ (M)	Search Cost (GPU-days)	Search Method
Wide ResNet [44]	36.5	4.17	-	-	human experts
DenseNet-BC (k = 40) [17]	25.6	3.47	-	-	human experts
NAS [46]	7.1	4.47	-	3150	RL
NAS + more filters[46]	37.4	3.65	-	3150	RL
ENAS + macro search space [33]	21.3	4.23	-	0.5	RL + weight sharing
ENAS + macro search space + more channels [33]	38.0	3.87	-	0.5	RL + weight sharing
NSGA-Net + macro search space	3.3	3.85	1290	8	evolution
DARTS second order + cutout [27]	3.3	2.76	-	4	gradient-based
DARTS second order (6 @ 576) + cutout [27] [†]	3.3	2.76	547	4	gradient-based
NASNet-A + cutout [47]	3.3	2.65	-	2,000	RL
NASNet-A (6 @ 660) + cutout [47] [†]	3.2	2.91	532	2,000	RL
ENAS + cutout [33]	4.6	2.89	-	0.5	RL + weight sharing
ENAS (6 @ 660) + cutout [33] [†]	3.3	2.75	533	0.5	RL + weight sharing
AmoebaNet-A [35]	3.2	3.34	-	3,150	evolution
AmoebaNet-A (6 @ 444) + cutout [35] [†]	3.3	2.77	533	3,150	evolution
NSGA-Net (6 @ 560) + cutout	3.3	2.75	535	4	evolution
NSGA-Net (7 @ 1536) + cutout	26.8	2.50	4147	4	evolution

Lu, Z., Whalen, I., Boddeti, V., Dhebar, Y., Deb, K., Goodman, E., & Banzhaf, W. (2019, July). Nsga-net: neural architecture search using multi-objective genetic algorithm. In *Proceedings of the Genetic and Evolutionary Computation Conference* (pp. 419-427).

Overview



- Evolving Morphologies
- Genetic Programming
- Neuroevolution
- Evolutionary algorithms & selective pressures
- Neuroevolution/GP in Deep learning era
- Building you own representation

Building your own representation: Radcliffe's recommendations

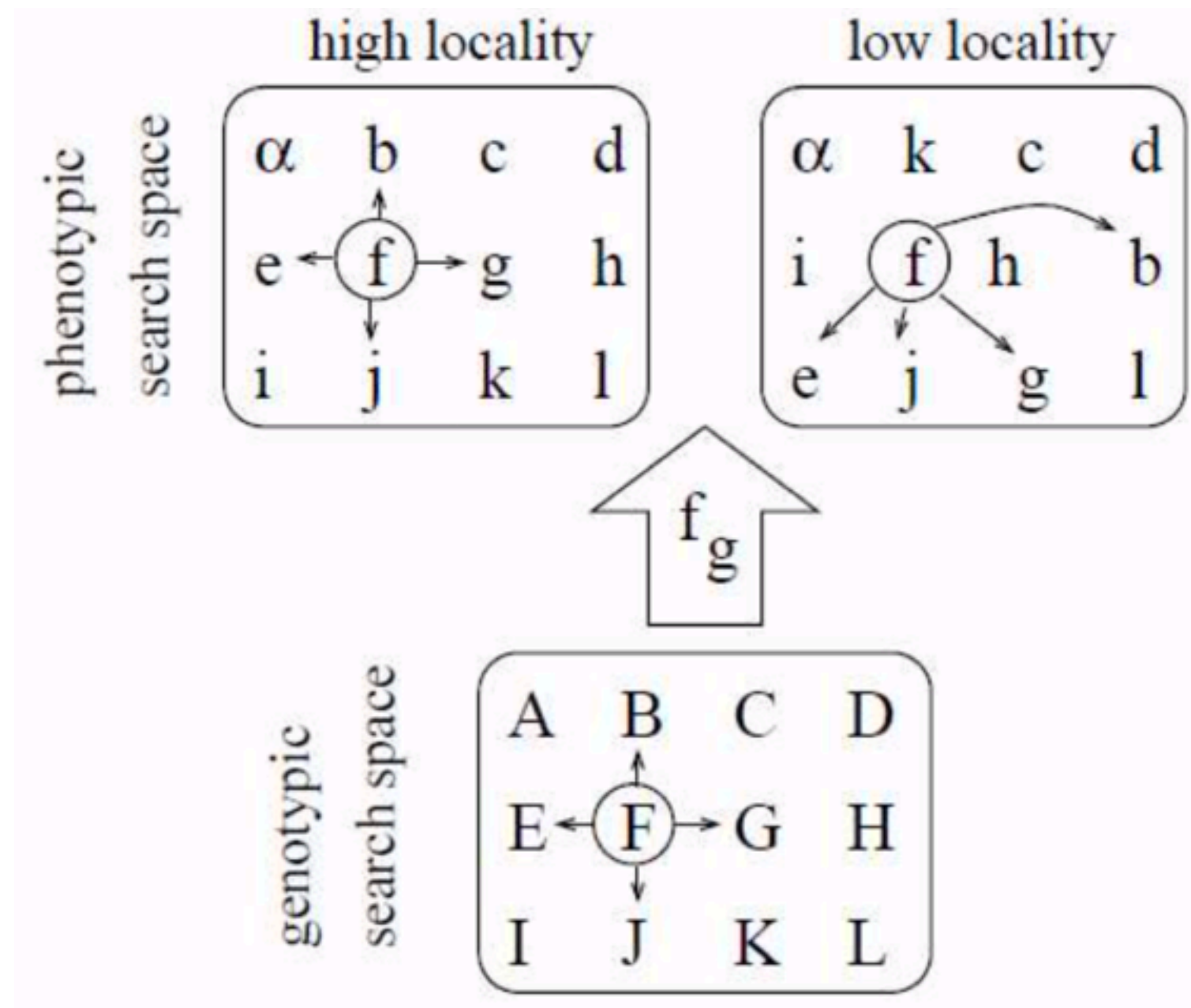
- **Respect:** offspring produced by crossover should have all characteristics that *both* parents have
- **Transmission:** every gene should come from one parent
- **Assortment:** Offspring = any compatible combination of parent features
- **Ergodicity:** repetitive application of operators should reach any part of the search space

Building your own representations: recommendations

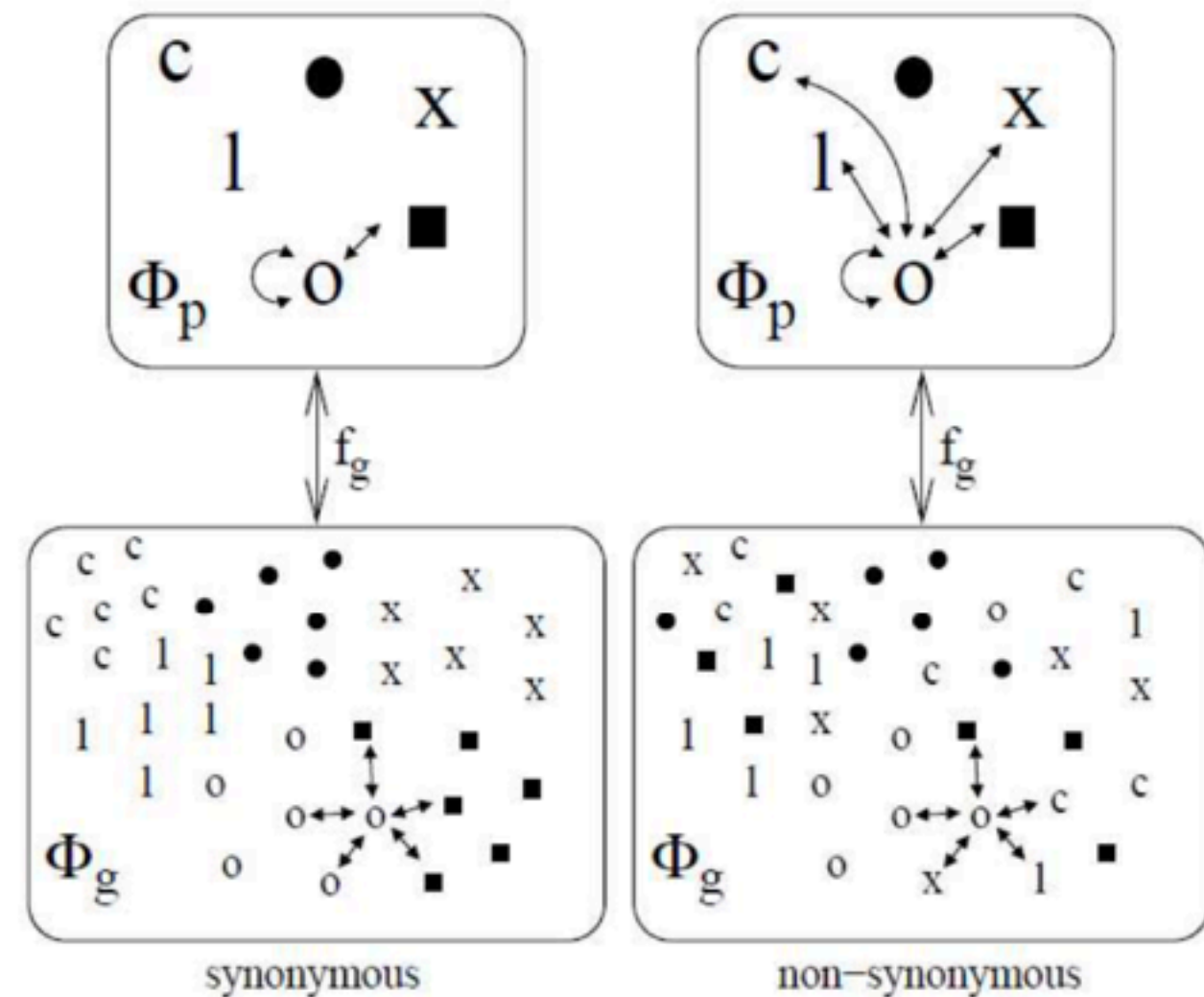
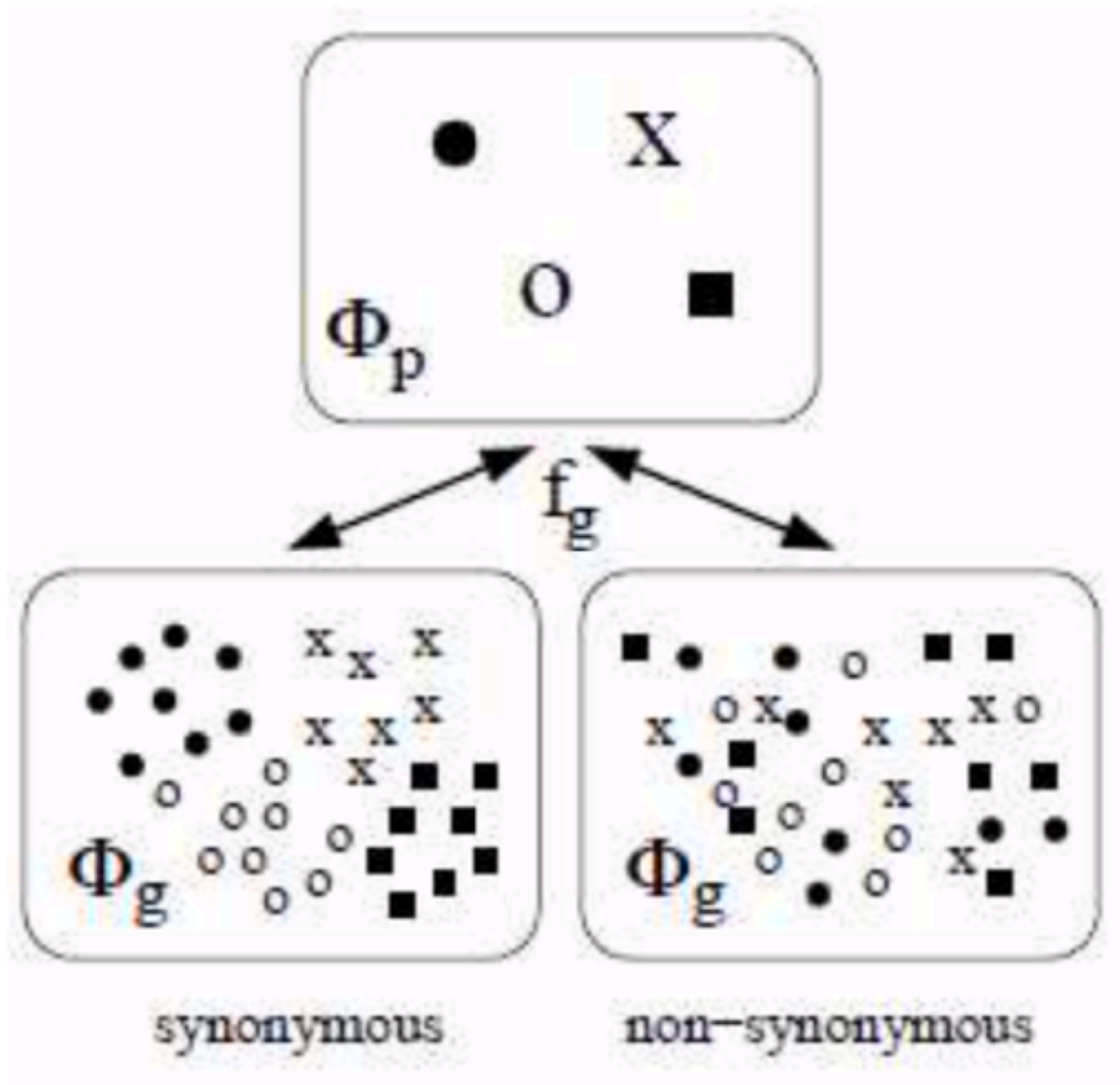
- A mutation should generate limited modifications
- A genotype should combine properties of the parents
- A representation is a mapping from genotype space (explored by the EA) to the phenotype space (generated structure): How does it influence mutation/crossover ?

Rothlauf's recommendations: locality

- Locality depends on the operators
- High locality: neighbours in genotype space remain neighbours in the phenotype space
 - does not change problem complexity

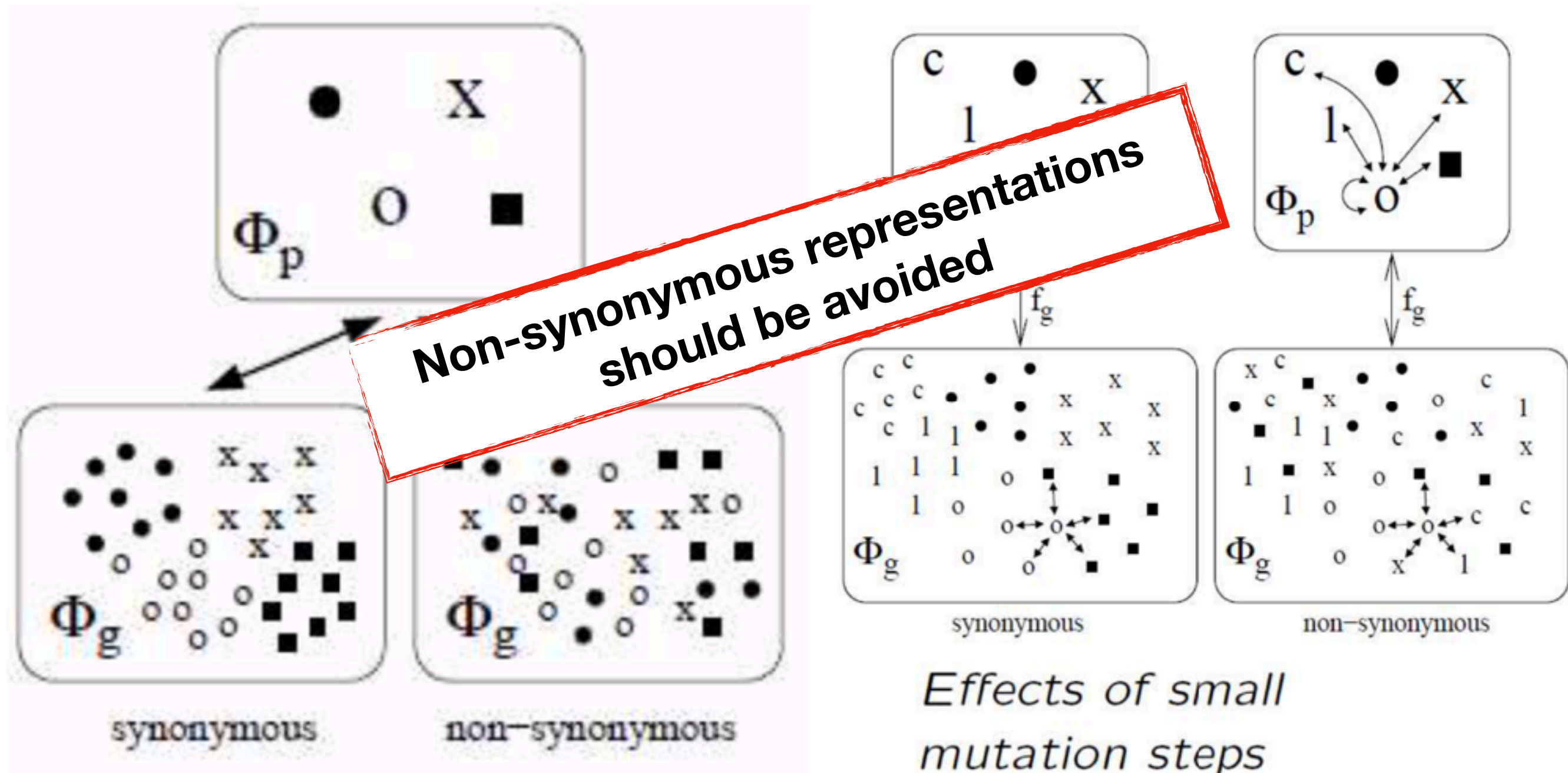


Rothlauf's recommendations: redundancy



*Effects of small
mutation steps*

Rothlauf's recommendations: redundancy



Take home messages

- Possible to evolve complex structures: genetic programming, neuroevolution (NEAT)
- Mutation and crossover operators need to be carefully designed
- And the selective pressure to use is critical

Thank you for listening !



stephane



https://

WARNING, next lecture:
16/04/2026, 8:30-10:30
univsite.fr
mc.fr/sdoncieux/

